

IniFile

1.20

Erzeugt von Doxygen 1.9.1

1 Die Benutzung von IniFile aus Anwendersicht	1
1 Die Benutzung von IniFile aus Anwendersicht	1
2 Ausstehende Aufgaben	4
3 Liste der bekannten Fehler	5
4 Veraltete Elemente	5
5 Datenstruktur-Verzeichnis	5
5.1 Datenstrukturen	5
6 Datei-Verzeichnis	5
6.1 Auflistung der Dateien	5
7 Datenstruktur-Dokumentation	6
7.1 IniEntry Strukturreferenz	6
7.1.1 Ausführliche Beschreibung	6
7.1.2 Dokumentation der Felder	6
8 Datei-Dokumentation	7
8.1 IniFile.c-Dateireferenz	7
8.1.1 Makro-Dokumentation	10
8.1.2 Dokumentation der benutzerdefinierten Typen	13
8.1.3 Dokumentation der Funktionen	13
8.1.4 Variablen-Dokumentation	41
8.2 IniFile.h-Dateireferenz	44
8.2.1 Ausführliche Beschreibung	47
8.2.2 Makro-Dokumentation	48
8.2.3 Dokumentation der benutzerdefinierten Typen	55
8.2.4 Dokumentation der Funktionen	56
Index	81

1 Die Benutzung von IniFile aus Anwendersicht

IniFile dient dazu, flexibel Konfigurationsinformationen zu verwalten, die in Ini-Dateien im Windows-Stil vorliegen. Ein kompatibles, funktional weit gehend identisches Perl-Modul zum gleichen Zweck liegt ebenfalls vor.

Mehrere Ini-Dateien können nacheinander verarbeitet werden, wobei später eingelesene Dateien Vorrang vor früheren haben. Dadurch können bei größeren Projekten allgemein gehaltene Konfigurationsdateien für Standardwerte benutzt werden, die für einzelne Teilanwendungen durch eigene INI-Dateien selektiv überschrieben werden können. IniFile bietet auch die Möglichkeit, mit Platzhaltern in Variablenwerten zu operieren. Weiterhin ist ein Include-Mechanismus für geschachtelte Konfigurationsdateien realisiert.

INI-Dateien enthalten je Zeile ein Wertepaar

```
key = value
```

Leerzeichen am Anfang, vor und hinter dem Gleichheitszeichen sowie am Zeilenende werden ignoriert. Der Werteteil kann in einfache oder doppelte Hochkommata eingeschlossen werden, wenn er Leerzeichen am Anfang oder am Ende enthalten soll. (Diese Hochkommata werden entfernt, bevor sie an ein Anwendungsprogramm weitergegeben werden.)

Fehlt die Angabe von `value`, wird dem Schlüssel der leere String als Wert zugewiesen. In diesem Fall kann auch das Gleichheitszeichen entfallen.

Zusätzlich können Abschnitte verwendet werden:

```
[section]
```

Gleichnamige Schlüssel in verschiedenen Abschnitten sind unabhängig voneinander. Eine Abschnittsangabe gilt bis zur nächsten Abschnittsspezifikation. In den (unbenannten) Standardabschnitt kann in der Ini-Datei durch Angabe des leeren Abschnittsnamens `[]` (oder `[ ]`) zurückgewechselt werden. Auf Schlüssel in benannten Abschnitten wird Bezug genommen (s.u.), indem der Abschnittsname in eckigen Klammern an den Anfang des Schlüsselnamens gestellt wird. Schlüssel im unbenannten Abschnitt haben kein solches Präfix, dürfen jedoch auch mit einem Präfix `[]` (oder `[ ]`) angegeben werden.

Schlüssel und Abschnittsnamen dürfen keine Leerzeichen, Gleichheitszeichen, Doppelpunkte sowie eckige Klammern enthalten.

Eine Konfigurationsdatei kann mit Hilfe eines Pragmas (siehe unten) festlegen, dass ein Abschnitt (üblicherweise der unbenannte) Standardwerte enthält, die dann verwendet werden, wenn ein abgefragter Wert nicht gefunden wird. Dieses Verhalten kann unabhängig vom Inhalt der Konfigurationsdatei auch zur Laufzeit durch einen Funktionsaufruf geändert werden.

Konfigurationsinformationen werden über ihren Schlüssel angesprochen. Konfigurationswerte können Platzhalter enthalten, die erst bei der Benutzung aufgelöst werden. Dadurch ist es möglich, zentrale Informationen (wie beispielsweise ein projektspezifisches Wurzelverzeichnis) nur an einer Stelle zu spezifizieren und an anderen Stellen der INI-Dateien symbolisch darauf Bezug zu nehmen, ohne dass die Anwendung davon etwas merkt. Platzhalter werden ähnlich wie in Windows-Batch-Dateien durch Prozentzeichen geklammert:

```
dir = test/  
  
logfile = %dir%error.log
```

Der Anwendung wird durch `ini_get("logfile", ...)` der Wert `test/error.log` geliefert, sofern der Wert des Schlüssels `dir` nicht in der Zwischenzeit geändert wurde. Der Platzhalter kann am Anfang einen Abschnittsnamen in eckigen Klammern enthalten. Ohne Angabe eines Abschnitts wird stets der unbenannte Abschnitt verwendet. Wird ein leerer oder nur aus Leerzeichen bestehender Abschnittsname angegeben, wird ebenfalls der unbenannte Abschnitt verwendet. Ansonsten sollte der Platzhalter keine Leerzeichen enthalten. Die Referenzen werden dynamisch rekursiv aufgelöst. Unendliche Rekursion wird durch eine Heuristik verhindert.

Konfigurationswerte können auch durch das Programm zur Laufzeit hinzugefügt, geändert oder gelöscht werden. Die verfügbaren Schlüssel und Abschnitte können aufgelistet werden. Konfigurationsdateien können auch geschrieben werden.

Schlüssel- und Abschnittsnamen sind unabhängig von Groß- und Kleinschreibung.

Kommentarzeilen, die mit `#`, `*` oder `;` anfangen, werden ignoriert. Kommentare am Zeilenende werden darüber hinaus nicht unterstützt.

Weiterhin können einige fest vorgegebene Pragmata in Konfigurationsdateien stehen. Sie bestehen aus einem Schlüsselwort, gefolgt von einem Doppelpunkt (kein Gleichheitszeichen!) und dahinter optional einem Wert. Bei Pragmata findet die oben dargestellte Platzhalterersetzung nicht statt. Zurzeit sind drei Pragmata realisiert:

1. `include`: gibt den Namen einer weiteren Konfigurationsdatei an (absolut oder aber relativ zum Pfad der Anwendung), deren Inhalt an der Stelle eingefügt wird, an der die Anweisung auftritt. Geschachtelte Inklusionen sind zulässig. Auch wenn eine Inklusionsanweisung innerhalb eines benannten Abschnitts auftritt, beginnt die inkludierte Datei im unbenannten Abschnitt. Umgekehrt wird beim Erreichen des Dateiendes und bei der Rückkehr in die inkludierende Datei der vorher dort geltende Abschnitt wieder aufgenommen. Nach dem Einlesen ist nicht mehr rekonstruierbar, welche Informationen welcher Datei entstammen.
2. `fallback`: legt den Namen eines Abschnitts fest, in dem Werte gesucht werden, die im explizit oder implizit genannten Abschnitt nicht gefunden werden. Wenn der unbenannte Abschnitt als Fallback verwendet werden soll, muss dies durch einen leeren Namen (zwei direkt aufeinander folgende Anführungszeichen) angegeben werden. Fehlt der Name, wird der Fallback-Mechanismus abgeschaltet. Die Einstellung gilt global, kann also nicht für einzelne Passagen an- und für andere abgeschaltet werden. Bei mehrfachen Angaben gilt die zuletzt gelesene.
3. `resolve_escapes`: Wenn dieses Pragma ohne Wert oder mit einem ganzzahligen Wert ungleich 0 angegeben wird, dann werden Escape-Sequenzen im C-Stil (also `\a`, `\b`, `\f`, `\n`, `\r`, `\t`, `\v`, `\'`, `\"`, `\?`, `\\` sowie `\` gefolgt von bis zu drei Oktalziffern und `\x` gefolgt von bis zu zwei Hexadezimalziffern) in den entsprechenden Byte-Wert umgesetzt. (Zur Angabe eines literalen `\` sollte dieser anschließend jeweils verdoppelt angegeben werden.) Die Angabe eines NULL-Bytes (`\0` oder `\x0`) ist nicht ratsam, da C-Programme mit Zeichenketten, die diesen Wert enthalten, nicht gut umgehen können. Diese Umsetzung findet nur innerhalb von Werten, nicht innerhalb von Schlüsseln und nicht innerhalb von Pragmata, statt, und zwar bereits beim Lesen der Datei. Die Einstellung gilt von der nächsten Zeile an, nicht jedoch in inkludierten Dateien. Tritt die Anweisung in einer inkludierten Datei auf, gilt sie nur bis zum Ende dieser Datei. Wird als Wert des Pragmas 0 angegeben, wird die Ersetzung ab der folgenden Zeile beendet.

Zusätzlich sind in dem Library-Modul Hilfsroutinen enthalten, die plattformunabhängige Möglichkeiten zum Aufbau von Dateipfaden unterstützen.

Schließlich sind einige Makros in der öffentlichen Schnittstelle enthalten, die Low-Level-Funktionen zur Speicher-verwaltung usw. ähnlich wie `stdlib.h` bieten (`INI_MALLOC`, `INI_REALLOC`, `INI_STRALLOC`, `INI_STREALLOC`, `INI_FREE`, `INI_COPYSTR`).

Alle Funktionen sind mit dem Präfix `ini_` versehen, um die Namensraumverschmutzung zu reduzieren. Alle Makros tragen das Präfix `INI_`.

Bemerkungen

Im Normalfall wird bei der Kompilierung eine linkbare Objektdatei erzeugt. Alternativ kann bei der Kompilierung das Präprozessor-Symbol `INI_TEST` definiert werden. Dann wird eine einfache `main()`-Funktion mitkompiliert, die beim Linken ein ausführbares Testprogramm ergibt. Diesem Testprogramm können beim Aufrufen die Namen einer oder mehrerer INI-Dateien (zum Beispiel eine der beigelegten Test-INI-Dateien) mitgegeben werden. Die enthaltenen Projektdefinitionsdateien für Microsoft Visual Studio definieren dieses Symbol in der Debug-Konfiguration (jedoch nicht in der Release-Konfiguration).

Zu beachten

Die Implementierung benutzt sortierte verkettete Listen von Wertepaaren. Bei typischen Initialisierungsdateigrößen hat dies den Vorteil, schlanken Code mit geringem Overhead zu ermöglichen. Bei sehr großen Mengen von Einträgen kann die Laufzeit sich jedoch deutlich verschlechtern. Für diesen Fall wäre eine Implementierung mit Hashes vorzuziehen. Eine solche Umstellung ist jedoch nicht geplant. Gleichwohl sollten Anwendungen die innere Datenstruktur als opak betrachten.

Eine Implementierung als Klasse könnte je nach Anwendungsfall sinnvoll sein. Eine solche Umstellung ist jedoch ebenfalls nicht geplant.

Achtung

Es gibt nur eine Instanz des Konfigurationshandlers pro Prozess. Es ist derzeit nicht möglich, mehrere Konfigurationsdateien gleichzeitig unabhängig voneinander zu verwalten.

Die Funktionen sind außerdem nicht threadsafe.

Noch zu erledigen Heuristik zur Verhinderung unendlicher Rekursionen ersetzen durch explizite Prüfung darauf, ob bei der Expansion einer Referenz dieselbe Referenz erneut auftritt. Möglicherweise diesen Mechanismus ebenso wie den zur Verhinderung unendlicher Inklusions-Rekursion auf Stack-(oder Hash-?)Technik umstellen. (Niedrige Priorität)

Noch zu erledigen Bereitstellen als DLL?

\$Id: IniFile.c 430 2021-07-09 23:56:33Z gisbert \$

Änderungen:

Datum	Kürzel	Version	Beschreibung
2021-07-19	gws	1.20	Segmentnamen aus lauter Leerzeichen sind jetzt äquivalent zum unbenannten Segment. Ersetzung von Platzhaltern mit leerem Wert repariert. Funktionen ini_comments() , ini_ncomments() , ini_add_comment() und iniget_license() hinzugefügt. Dokumentation ergänzt. Testfälle systematisiert.
2021-03-12	gws	1.12	Fehlerhafte Behandlung bei fehlender Schlüsselangabe korrigiert. Probleme mit isspace() , toupper() usw. bei 8-Bit-Zeichen umgangen.
2021-02-01	gws	1.11	Fehlercode korrigiert, wenn Ini-Datei nicht gefunden Secure-Funktionen/-Makros eingeführt (aber noch nicht konsequent eingesetzt, nur ganz am Ende).
2007-10-25	gws	1.1	Pragma resolve_escapes eingeführt.
2007-06-25	gws	1.01	INI_COPYSTR verfeinert, INI_STRLEN eingeführt.
2007-04-02	gws	1.0	Dokumentation angepasst, Parametertypen genauer gefasst, ini_get_version() hinzugefügt.
2007-03-11	gws	1.0rc2	Unendliche Rekursion bei Include verhindert.
2007-02-05	gws	1.0rc1	Include-Mechanismus sowie 2 Pragmata implementiert.
2007-02-02	gws	0.95	Fallback-Abschnitt und ini_catfile eingeführt.
2006-04-25	gws	0.942	Deprecation Warnings von Visual Studio 2005 abgewürgt.
2005-09-27	gws	0.941	Prototyp für ini_destroy() korrigiert.
2005-05-08	gws	0.94	Minimale Erweiterungen.
2005-05-01	gws	0.2	Verarbeitung von Abschnitten voll integriert.
2004-11-04	gws	0.12	Minimale Funktionserweiterung.
1998-xx-xx	gws	0.1x	Erste Produktionsversion.

2 Ausstehende Aufgaben

page **Die Benutzung von IniFile aus Anwendersicht**

Heuristik zur Verhinderung unendlicher Rekursionen ersetzen durch explizite Prüfung darauf, ob bei der Expansion einer Referenz dieselbe Referenz erneut auftritt. Möglicherweise diesen Mechanismus ebenso wie den zur Verhinderung unendlicher Inklusions-Rekursion auf Stack-(oder Hash-?)Technik umstellen. (Niedrige Priorität)

Bereitstellen als DLL?

3 Liste der bekannten Fehler

Global [ini_comments](#) (char **commentlist, int maxcomment)

Bei Abbruch wegen Speichermangel wird bereits allozierter Speicher nicht wieder freigegeben.

Global [ini_keys](#) (char **keylist, int maxkeys)

Bei Abbruch wegen Speichermangel wird bereits allozierter Speicher nicht wieder freigegeben.

Global [ini_keys_section](#) (char **keylist, int maxkeys, const char *sectionname)

Bei Abbruch wegen Speichermangel wird bereits allozierter Speicher nicht wieder freigegeben.

Global [ini_sections](#) (char **sectionlist, int maxsections)

Bei Abbruch wegen Speichermangel wird bereits allozierter Speicher nicht wieder freigegeben.

4 Veraltete Elemente

Global [ini_add_dir_delim](#) (char *dirpath, const unsigned char option)

Diese Aufgabe wird von [ini_catfile\(\)](#) besser gemacht.

5 Datenstruktur-Verzeichnis

5.1 Datenstrukturen

Hier folgt die Aufzählung aller Datenstrukturen mit einer Kurzbeschreibung:

[IniEntry](#)

Struktur zum Verwalten von Ini-Einträgen (Schlüssel-Wert-Paare) in einer verketteten Liste

6

6 Datei-Verzeichnis

6.1 Auflistung der Dateien

Hier folgt die Aufzählung aller Dateien mit einer Kurzbeschreibung:

[IniFile.c](#)

7

[IniFile.h](#)

Header-Datei für den Konfigurationsdateihandler IniFile

44

7 Datenstruktur-Dokumentation

7.1 IniEntry Strukturreferenz

Struktur zum Verwalten von Ini-Einträgen (Schlüssel-Wert-Paare) in einer verketteten Liste.

```
#include <IniFile.h>
```

Zusammengehörigkeiten von IniEntry:



Datenfelder

- char * [key](#)
Schlüssel.
- struct [IniEntry](#) * [next](#)
Pointer auf den nächsten Eintrag.
- char * [value](#)
Wert.

7.1.1 Ausführliche Beschreibung

Struktur zum Verwalten von Ini-Einträgen (Schlüssel-Wert-Paare) in einer verketteten Liste.

Definiert in Zeile 77 der Datei IniFile.h.

7.1.2 Dokumentation der Felder

7.1.2.1 **key** char* IniEntry::key

Schlüssel.

Definiert in Zeile 78 der Datei IniFile.h.

7.1.2.2 value `char* IniEntry::value`

Wert.

Definiert in Zeile 79 der Datei IniFile.h.

7.1.2.3 next `struct IniEntry* IniEntry::next`

Pointer auf den nächsten Eintrag.

Definiert in Zeile 80 der Datei IniFile.h.

Die Dokumentation für diese Struktur wurde erzeugt aufgrund der Datei:

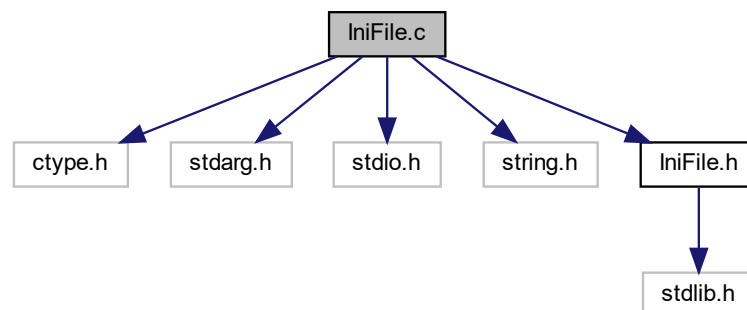
- [IniFile.h](#)

8 Datei-Dokumentation

8.1 IniFile.c-Dateireferenz

```
#include <ctype.h>
#include <stdarg.h>
#include <stdio.h>
#include <string.h>
#include "IniFile.h"
```

Include-Abhängigkeitsdiagramm für IniFile.c:



Makrodefinitionen

- #define `BUFSIZE` (65500)
Größe des I/O-Buffers.
- #define `COPYRIGHT`
Copyright-Hinweis.
- #define `DEFAULTOPTION` ("0")
- #define `ERROR_RETURN`(ininame, lineno, retval)
Hilfs-Makro für die Fehlerbehandlung in `ini_read()`.
- #define `JUMP_EMPTY_SECTION`(key)
Hilfs-Makro zum Überspringen von Referenzen auf das unbenannte Segment.
- #define `MAKE_SPECIAL_FILE_NAME`(fn, temp_ptr, ext)
Hilfs-Makro für die Erzeugung der speziellen .log- ind .err-Dateinamen.
- #define `MINIBUFSIZE` (1024)
Größe des Speichers zum Lesen von Zeilen.
- #define `VERSION` (0x0114)
Versionsnummer 2 Bytes binär (Major Version: oberes Byte, Minor Version: unteres Byte).
- #define `VERSIONSTRING` ("1.20")
Version als String (nicht unbedingt rein numerisch).

Typdefinitionen

- typedef unsigned char `uchar`

Funktionen

- int `handle_pragma` (char *pragma, char *value, int *npragma, int *pragmaval)
Verarbeitung einer Pragma-Anweisung in einer Ini-Datei (interne Funktion).
- int `ini_add` (const char *key, const char *const value)
Fügt einen Konfigurationseintrag (Schlüssel/Wert) hinzu.
- int `ini_add_comment` (const char *text)
Fügt eine Kommentarzeile hinzu.
- char * `ini_add_dir_delim` (char *dirpath, const unsigned char option)
Hängt einen Verzeichnistrenner an, falls nicht schon vorhanden.
- char * `ini_catfile` (char *components,...)
Baut einen Dateipfad aus einzelnen Komponenten zusammen.
- char ** `ini_comments` (char **commentlist, int maxcomment)
Liefert eine Liste der Kommentare.
- int `ini_delete` (const char *key)
Löscht einen Schlüssel, falls es ihn gibt.
- void `ini_destroy` (void)
Löscht alle Schlüssel, löscht alle Strukturen.
- char * `ini_fix_dir_delim` (char *path, const unsigned char option)
Setzt alle Vorkommen von Verzeichnistrennern auf Unix- oder DOS-Stil um.
- char * `ini_get` (char *value, const char *key, const char *const defaultvalue, const size_t maxlen)
Holt den Wert, der zu einem gegebenen Konfigurations-Schlüssel gehört.
- char `ini_get_dir_delimiter` (const unsigned char option)
Gibt das Verzeichnistrennzeichen im Unix- oder DOS-Stil zurück.
- char * `ini_get_error_filename` (void)
Liefert ggf.

- int __inline ini_get_error_linenumber (void)
Liefert ggf.
- char * ini_get_fallbacksection (void)
Holt den Namen des Fallback-Abschnitts für im angegebenen Abschnitt nicht gefundene Einträge.
- char * ini_get_license (void)
Liefert den Lizenz- und Copyright-Text.
- unsigned short __inline ini_get_version (void)
Liefert die aktuelle Version der Library als Binärwert.
- const char __inline * ini_get_version_string (void)
Liefert die aktuelle Version der Library als String.
- char ** ini_keys (char **keylist, int maxkeys)
Liefert eine Liste der Schlüssel.
- char ** ini_keys_section (char **keylist, int maxkeys, const char *sectionname)
Liefert eine Liste der Schlüssel in einem gegebenen Abschnitt.
- int __inline ini_ncomments (void)
Liefert die Anzahl der Kommentare zurück.
- int __inline ini_nkeys (void)
Liefert die Anzahl der Konfigurationseinträge zurück.
- int ini_nkeys_section (const char *sectionname)
Liefert die Anzahl der Konfigurationseinträge in einem gegebenen Abschnitt zurück.
- int ini_nsections (void)
Liefert die Anzahl der Konfigurationsabschnitte zurück.
- int ini_read (const char *const inifilename)
Einlesen einer Konfigurationsdatei.
- void __inline * ini_safemalloc (const size_t size)
Sichere (safe) Allokation von Speicherplatz.
- void __inline * ini_saferealloc (void *memp, const size_t size)
Sichere (safe) Neu-Allokation von Speicherplatz.
- char ** ini_sections (char **sectionlist, int maxsections)
Liefert eine Liste der Konfigurationsabschnitte zurück.
- int __inline ini_set (const char *key, const char *const value)
Fügt einen Konfigurationseintrag (Schlüssel/Wert) hinzu; Synonym für ini_add().
- int ini_set_fallbacksection (const char *sectionname)
Setzt den Namen des Fallback-Abschnitts für im angegebenen Abschnitt nicht gefundene Einträge.
- int ini_write (const char *const inifilename, const char *comment)
Schreibt die aktuellen Ini-Werte in eine Datei.
- void __inline * inis_saferealloc (void *memp, const size_t newsize, const size_t oldsize)
Sichere (safe&secure) Neu-Allokation von Speicherplatz.
- void __inline inis_wipe_memory (void *memp, const size_t size)
Sicheres (safe und secure) Löschen von Speicherbereichen für sicherheitsrelevante Anwendungen.
- char * isolate_string (char *str, int resolve_escapes)
Isoliert den relevanten Inhalt eines Strings (interne Funktion).
- char * make_section_name (const char *str)
Erzeugt aus einem Namen einen sauberen Abschnittsnamen (interne Funktion).

Variablen

- static int `commentcount` = 0
Anzahl der Kommentarzeilen (stets aktuell).
- static `TIniEntry` * `comments` = NULL
Anfang der verketteten Liste von Ini-Kommentar-Einträgen.
- static int `entrycount` = 0
Anzahl der Konfigurationseinträge (stets aktuell).
- static char * `errname` = NULL
Sonderfall: Name der Standard-Fehlerprotokolldatei.
- static char * `errorininame` = NULL
Name der Konfigurationsdatei, in der ein Fehler auftrat (oder NULL).
- static int `errorlinenumber` = -1
Nummer der Konfigurationsdateizeile, bei der ein Fehler auftrat.
- static char * `fallbacksection` = NULL
Fallback-Abschnitt für im angegebenen Abschnitt nicht gefundene Einträge.
- static int `got_license` = 0
Flag, ob Lizenz-/Copyright-Text schon abgefragt wurden.
- static `TIniEntry` * `inilist` = NULL
Anfang der verketteten Liste von Ini-Einträgen.
- static `TIniEntry` * `ininames` = NULL
Sammlung aktiver Include-Namen, um Rekursion zu vermeiden

- static char * `logname` = NULL
Sonderfall: Name der Standard-Protokolldatei.
- static int `recursion` = -1
Merker für Rekursion bei der Platzhalter-Ersetzung.
- static int `sectioncount` = 0
Anzahl der Abschnitte (gecached).
- static char * `tmpstr` = NULL
Platz für temporär erzeugte Strings für Rückgabe aus Funktionen.

8.1.1 Makro-Dokumentation

8.1.1.1 VERSION `#define VERSION ( 0x0114 )`

Versionsnummer 2 Bytes binär (Major Version: oberes Byte, Minor Version: unteres Byte).

Definiert in Zeile 253 der Datei `IniFile.c`.

8.1.1.2 VERSIONSTRING `#define VERSIONSTRING ( "1.20" )`

Version als String (nicht unbedingt rein numerisch).

Definiert in Zeile 254 der Datei `IniFile.c`.

8.1.1.9 DEFAULTOPTION `#define DEFAULTOPTION ("0")`

8.1.2 Dokumentation der benutzerdefinierten Typen

8.1.2.1 `uchar` `typedef unsigned char uchar`

Definiert in Zeile 260 der Datei IniFile.c.

8.1.3 Dokumentation der Funktionen

8.1.3.1 `ini_safemalloc()` `void __inline* ini_safemalloc ( const size_t size )`

Sichere (safe) Allokation von Speicherplatz.

Der Speicherplatz (sofern genügend verfügbar) wird mit 0 initialisiert.

Parameter

<code>in</code>	<code>size</code>	Angeforderte Speichergröße in Bytes.
-----------------	-------------------	--------------------------------------

Rückgabe

Pointer auf den Speicherplatz. NULL, falls nicht genügend Speicher vorhanden.

Definiert in Zeile 304 der Datei IniFile.c.

```

307                                     {
308   char *memp = (char *)malloc(size);
309   if (memp) inis_wipe_memory( mem, size );

```

8.1.3.2 `ini_saferealloc()` `void __inline* ini_saferealloc ( void * memp, const size_t size )`

Sichere (safe) Neu-Allokation von Speicherplatz.

Sichere (safe) Re-Allokation von Speicherplatz.

Falls die Größe 0 und der Pointer NULL ist, ist dies ein NOOP. Falls die Größe 0 ist, ist diese Funktion identisch mit `free( )`. Falls der angegebene Pointer NULL ist, ist diese Funktion identisch mit `ini_safemalloc( )`. Falls die neue Speichergröße größer als die alte ist, wird der neue Teil nicht initialisiert. Falls der neue Speicherbereich kleiner als der alte ist, wird der alte Restspeicher ebenfalls nicht ausgenullt.

Parameter

in, out	<i>memp</i>	Pointer auf den zu verändernden Speicher. Kann NULL sein.
in	<i>size</i>	Angeforderte Speichergröße in Bytes.

Rückgabe

Pointer auf den Speicherplatz. NULL, falls nicht genügend Speicher vorhanden.

Definiert in Zeile 328 der Datei IniFile.c.

```
331                                     {
332     if ( size == 0 ) {
333         if (memp) {
334             free(memp);
335             memp = NULL;
336         }
337     } else {
338         memp = (memp) ? realloc(memp, size) : INI_MALLOC(size);
339     }
```

8.1.3.3 inis_saferealloc() void __inline* inis_saferealloc (

void * *memp*,

const size_t *newsize*,

const size_t *oldsize*)

Sichere (safe&secure) Neu-Allokation von Speicherplatz.

Falls die neue Größe 0 und der Pointer NULL ist, ist dies ein NOOP. Falls die neue Größe 0 ist, ist diese Funktion identisch mit [INIS_FREE\(\)](#). Falls der angegebene Pointer NULL ist, ist diese Funktion identisch mit [ini_safemalloc\(\)](#). Falls die neue Speichergröße größer als die alte ist, wird der neue Teil nicht initialisiert. Falls der neue Speicherbereich kleiner als der alte ist, wird der alte Restspeicher ebenfalls nicht ausgenullt.

Parameter

in, out	<i>memp</i>	Pointer auf den zu verändernden Speicher. Kann NULL sein.
in	<i>newsize</i>	Angeforderte Speichergröße in Bytes.
in	<i>oldsize</i>	Bisherige Speichergröße in Bytes.

Rückgabe

Pointer auf den Speicherplatz. NULL, falls nicht genügend Speicher vorhanden.

Definiert in Zeile 359 der Datei IniFile.c.

```
362                                     {
363     char *newmemp = NULL;
364     if ( newsize == 0 ) {
365         INIS_FREE( memp, oldsize );
366     } else if ( memp ) {
367         /* Speicher neu allozieren, neuen Bereich initialisieren, alten löschen.
368          * Das ist u.U. deutlich aufwändiger als ein (non-secure) realloc.
369          */
370         newmemp = (char *)malloc(newsize);
371         if (newmemp) {
372             if ( newsize >= oldsize ) {
373                 memcpy( newmemp, memp, oldsize );
374                 if ( newsize > oldsize ) inis_wipe_memory( newmemp+oldsize, newsize-oldsize );
375             } else {
```

```

376     memcpy( newmemp, mem, newsize );
377 }
378     INIS_FREE( mem, oldsize );
379 }
380 } else {
381     /* Speicher-Neuallokation */
382     newmemp = INI_MALLOC(newsize);

```

8.1.3.4 inis_wipe_memory() void __inline inis_wipe_memory (

```

    void * mem,
    const size_t size )

```

Sicheres (safe und secure) Löschen von Speicherbereichen für sicherheitsrelevante Anwendungen.

Sicheres (secure) Löschen von Speicherplatz.

Der angegebene Speicherbereich wird so sicher (secure) wie möglich gelöscht. Stark angelehnt an die ARGON2-Source.

Parameter

in	<i>mem</i>	Pointer auf den zu löschenden Speicher. Kann NULL sein.
in	<i>size</i>	Speichergröße in Bytes.

Definiert in Zeile 393 der Datei IniFile.c.

```

396                                     {
397     if ( mem && size ) {
398     #if defined(_MSC_VER) && ( _MSC_VER >= 1400 ) || defined(__MINGW32__)
399         SecureZeroMemory( mem, size );
400     #elif defined memset_s
401         memset_s( mem, size, 0, size );
402     #elif defined(__OpenBSD__) || ( defined(__GLIBC__) && defined(__GLIBC_PREREQ) && __GLIBC_PREREQ(2,25) )
403         explicit_bzero( mem, size );
404     #else
405         static void * ( *const volatile memset_sec ) ( void *, int, size_t ) = &memset;
406         memset_sec( mem, 0, size );

```

8.1.3.5 isolate_string() char* isolate_string (

```

    char * str,
    int resolve_escapes )

```

Isoliert den relevanten Inhalt eines Strings (interne Funktion).

Führende Blanks werden übersprungen, Blanks am Ende werden ausgeblendet. Wenn der String in einfache oder doppelte Hochkommata eingeschlossen ist, werden diese ebenfalls ausgeblendet.

Parameter

in	<i>str</i>	Pointer auf den String. Der String kann durch die Funktion destruktiv verändert werden.
in	<i>resolve_escapes</i>	Boolescher Wert, der angibt, ob Escape-Sequenzen im C-Stil aufgelöst werden sollen.

Rückgabe

Pointer auf den isolierten String.

Definiert in Zeile 420 der Datei IniFile.c.

```

423                                     {
424     char *temp, *templ;
425     unsigned char n;
426     size_t k;
427
428     if (str) {
429         /* Ggf. Whitespace am Stringanfang überspringen: */
430         while ( *str && INI_ISSPACE(*str) ) str++;
431         /* Ggf. Whitespace am Stringende entfernen: */
432         if ( strlen(str) ) {
433             for ( temp=str+strlen(str)-1; INI_ISSPACE(*temp); temp-- ) *temp = '\0';
434             /* ggf. Anführungszeichen um Wert entfernen: */
435             if ( ( ( k=strlen(str) ) > 1 ) && ( ( *str == '"' ) || ( *str == '\\' ) ) && ( *(str+k-1) == *str ) ) {
436                 *(str+k-1) = '\0';
437                 str++;
438             }
439         }
440     }
441
442     if (resolve_escapes) {
443         templ = str;
444         while ( (temp = strchr( templ, '\\\' ) ) && ( strlen(temp) > 1 ) ) {
445             k = 1;
446             switch ( *(temp+1) ) {
447                 case 'a' : *temp = 7; break;
448                 case 'b' : *temp = 8; break;
449                 case 't' : *temp = 9; break;
450                 case 'n' : *temp = 10; break;
451                 case 'v' : *temp = 11; break;
452                 case 'f' : *temp = 12; break;
453                 case 'r' : *temp = 13; break;
454                 case '\\\' : *temp = '\\\' ; break;
455                 case '?' : *temp = '?'; break;
456                 case '\"' : *temp = '\"' ; break;
457                 case '\"' : *temp = '\"' ; break;
458                 case '0' : case '1' : case '2' : case '3' :
459                 case '4' : case '5' : case '6' : case '7' :
460                     /* Oktalzahl mit maximal 3 Stellen */
461                     n = 0;
462                     templ = temp + 1;
463                     while ( ( k <= 3 ) && strlen(templ) && ( *templ >= '0' ) && ( *templ <= '7' ) ) {
464                         n = 8*n + ( *templ - '0' );
465                         k++;
466                         templ++;
467                     }
468                     k--;
469                     memcpy( temp, &n, 1 );
470                     break;
471                 case 'x' : /* Hexadezimalzahl mit maximal 2 Stellen */
472                     n = 0;
473                     templ = temp + 2;
474                     while ( ( k <= 2 ) && strlen(templ) && INI_ISXDIGIT(*templ) ) {
475                         n = 16*n + ( INI_ISDIGIT(*templ) ? ( *templ - '0' ) : ( INI_CHAR_TOUPPER(*templ) -
476                             'A' + 10 ) );
477                         k++;
478                         templ++;
479                     }
480                     if ( k > 1 ) {
481                         memcpy( temp, &n, 1 );
482                     } else {
483                         /* Entartungsfall: kein Hex-Zeichen hinter \x gefunden.
484                         * Dann müssen wir die Pointer so umsetzen, dass anschließend nur \x überschrieben
485                         * wird,
486                         * ohne dass etwas dafür eingefügt würde.
487                         */
488                         temp--;
489                         k++;
490                     }
491                     break;
492                 default : k = 0;
493             }
494             if (k) memmove( temp+1, temp+k+1, strlen( temp + 2 ) + 1 );
495             templ = temp + 1;
496         }
497     }

```

8.1.3.6 make_section_name() char* make_section_name (

const char * str)

Erzeugt aus einem Namen einen sauberen Abschnittsnamen (interne Funktion).

Wenn der Eingabeparameter leer oder NULL oder "[" oder "]" ist, wird ein leerer String zurückgeliefert. Sonst wird ein String zurückgeliefert, der garantiert in eckigen Klammern steht und nur Großbuchstaben enthält.

Parameter

in	<i>str</i>	Ein Pointer auf einen Kandidat für einen Abschnittsnamen, aber möglicherweise unsauber. Die Eingabe bleibt garantiert unverändert.
----	------------	--

Rückgabe

Der saubere Abschnittsname in einem frisch allozierten Speicherbereich. NULL, falls nicht genug Speicherplatz vorhanden.

Definiert in Zeile 511 der Datei IniFile.c.

```

514                                     {
515     char *sectionname;
516     const char *temp;
517     size_t len;
518
519     /* Zunächst führende Blanks und eckige Klammern überspringen: */
520     while ( str && *str && ( INI_ISSPACE(*str) || ( *str == '[' ) ) ) str++;
521
522     /* Dann Blanks und eckige Klammern am Ende überspringen: */
523     if ( str && *str && ( *str != ']' ) ) {
524         temp = str + strlen(str) - 1;
525         while ( (temp > str) && ( INI_ISSPACE(*temp) || ( *temp == ']' ) ) ) temp--;
526         /* Der Name steht jetzt von str bis temp einschließlich, und zwar ohne eckige Klammern. */
527         /* Wir wissen auch, dass der String nicht leer ist, also temp >= str. */
528         len = (size_t)( temp - str + 1 );
529         sectionname = INI_STRALLOC( len+2 );
530         if ( !sectionname ) return NULL;
531         *sectionname = '[';
532         memcpy( sectionname+1, str, len );
533         *( sectionname+len+1 ) = ']';
534         *( sectionname+len+2 ) = '\0';
535         INI_STRUPPER( sectionname+1 );
536     } else {
537         /* Sonderfall: leere Eingabe bzw. leerer Abschnittsname. */
538         sectionname = INI_STRALLOC(0);
539         if (sectionname) *sectionname = '\0';
540     }

```

8.1.3.7 handle_pragma() int handle_pragma (

```

    char * pragma,
    char * value,
    int * npragma,
    int * pragma_val )

```

Verarbeitung einer Pragma-Anweisung in einer Ini-Datei (interne Funktion).

Verarbeitet einen Pragma-Eintrag in einer Ini-Datei und ruft die dafür notwendigen Funktionen auf.

Parameter

in	<i>pragma</i>	Pointer auf den Namen des Pragmas. Der String kann durch die Funktion destruktiv verändert werden.
in	<i>value</i>	Pointer auf den Wert. Kann leer sein. Hochkomma- und Escape-Handling müssen bereits erledigt sein. Der String kann durch die Funktion destruktiv verändert werden.
out	<i>npragma</i>	0, falls INCLUDE gefunden wurde; 1 für FALLBACK; 2 für RESOLVE_ESCAPES; -1 bei unbekanntem Pragma. (Nur, falls kein NULL übergeben wurde.)
out	<i>pragma_val</i>	Bei RESOLVE_ESCAPES der Boolesche Wert der Setzung. Sonst 0.

Rückgabe

0, falls OK; sonst einer der von `ini_read()` zurückgegebenen Werte. `INI_ERR_NOMEMORY`, falls nicht genügend Speicherplatz verfügbar. `INI_ERR_NOSUCHPRAGMA`, falls die Konfigurationsdatei ein unbekanntes Pragma enthält.

Definiert in Zeile 560 der Datei `IniFile.c`.

```

563                                     {
564     int n;
565
566     /* Pragma-Namen auf Großschreibung umsetzen: */
567     INI_STRUPPER(pragma);
568     if (pragmaval) *pragmaval = 0;
569
570     if ( INI_STREQUAL( pragma, "INCLUDE" ) ) {
571         /* Include-Datei, also ini_read() aufrufen. (Das ist rekursiv!) */
572         if (npragma) *npragma = 0;
573         n = ini_read(value);
574     } else if ( INI_STREQUAL( pragma, "FALLBACK" ) ) {
575         /* Fallback-Abschnitt setzen: */
576         if (npragma) *npragma = 1;
577         ini_set_fallbacksection(value);
578         n = 0;
579     } else if ( INI_STREQUAL( pragma, "RESOLVE_ESCAPES" ) ) {
580         /* Ab hier beim Einlesen Escape-Sequenzen im C-Stil umsetzen; bzw. Umsetzung beenden: */
581         if (npragma) *npragma = 2;
582         if (pragmaval) *pragmaval = ( value && strlen(value) ) ? atol(value) : 1;
583         n = 0;
584     } else {
585         /* Unbekanntes Pragma: */
586         if (npragma) *npragma = -1;
587         n = INI_ERR_NOSUCHPRAGMA;
588     }

```

8.1.3.8 `ini_read()` `int ini_read (`
`const char *const inifilename )`

Einlesen einer Konfigurationsdatei.

Die angegebene Konfigurationsdatei wird gelesen, ihre Einträge werden intern in einer sortierten verketteten Liste gespeichert. Konfligierende Einträge aus ggf. vorher eingelesenen Konfigurationsdateien werden überschrieben. Wenn die angegebene Datei nicht zum Lesen geöffnet werden kann, ist dies kein Fehler.

Parameter

<code>in</code>	<code>inifilename</code>	Name der einzulesenden Konfigurationsdatei.
-----------------	--------------------------	---

Rückgabe

Anzahl der nach dem Einlesen verfügbaren Einträge. `INI_ERR_NOMEMORY`, falls nicht genügend Speicherplatz verfügbar. `INI_ERR_NOSUCHPRAGMA`, falls die Konfigurationsdatei ein unbekanntes Pragma enthält.

Definiert in Zeile 641 der Datei `IniFile.c`.

```

644                                     {
645
646     FILE *inifile = NULL;
647     TIniEntry *ininameentry = NULL;
648     char minibuf[MINIBUFSIZE];
649     char *sectionname = NULL;
650     char *inbuffer = NULL;
651     char *temp1, *temp2, *temp3;
652     size_t sectionnamemaxlen, i;
653     int npragma, pragmaval, n, lineno = 0, resolve_escapes = 0;
654
655     errorlinenumber = -1;

```

```

656
657 if ( !INI_STRLen(inifilename) ) return ini_nkeys( );
658
659 /* Wir wollen keine unendlichen Include-Schleifen und müssen uns daher merken, wo wir schon waren.
660 * Mehrere Verarbeitungen derselben Ini-Datei sind zulässig, aber rekursive Aufrufe nicht.
661 * Wir speichern die gerade aktiven Namen in der Linked List ininames, wobei der neueste am Anfang
662   steht.
663 */
664 ininameentry = ininames;
665 while ( ininameentry ) {
666     /* Checken, ob unser aktueller Dateiname uns bekannt vorkommt. */
667     if ( INI_STREQUAL( ininameentry->value, inifilename ) ) return ini_nkeys( );
668     /* Hier waren wir schon einmal, daher hier nicht mehr erneut rekurrieren. */
669     ininameentry = ininameentry->next;
670 }
671
672 /* Wir sind beruhigt: bisher keine Rekursion. Aber unseren aktuellen Namen merken: */
673 ininameentry = (TIniEntry *)INI_MALLOC( sizeof( *ininameentry ) );
674 if ( !ininameentry ) return INI_ERR_NOMEMORY;
675 ininameentry->key = NULL;
676 ininameentry->value = (char *)inifilename;
677 ininameentry->next = ininames;
678 ininames = ininameentry;
679
680 /* Spezialfälle vorbereiten: Name der Standard-Protokolldatei
681 * (wie Ini-Datei, aber mit Extension .log) und der Standard-Fehlerprotokolldatei
682 * (wie Ini-Datei, aber mit Extension .err).
683 */
684 if ( !logname || !errname ) {
685     /* Basisnamen bestimmen: */
686     templ = INI_STRALLOC( strlen(inifilename) );
687     if (!templ) ERROR_RETURN( inifilename, lineno, INI_ERR_NOMEMORY );
688     (void)strcpy( templ, inifilename );
689     temp2 = strrchr( templ, '.' );
690     if (temp2) *temp2 = '\0';
691
692     MAKE_SPECIAL_FILE_NAME( logname, templ, ".log" ); /* Protokolldateiname */
693     MAKE_SPECIAL_FILE_NAME( errname, templ, ".err" ); /* Fehlerprotokolldateiname */
694
695     INI_FREE(templ);
696 }
697
698 /* Ini-Datei öffnen, I/O-Buffer setzen, falls möglich: */
699 inifile = fopen( inifilename, "r" );
700 if ( !inifile ) {
701     /* Kein Fehler, einfach diese Datei ignorieren. Aber auch aus unserem Ini-Call-Stack entfernen. */
702     ininameentry = ininames;
703     ininames = ininameentry->next;
704     INIS_FREE( ininameentry, sizeof( *ininameentry ) );
705     return ini_nkeys( );
706 }
707 inbuffer = (char *)INI_MALLOC(BUFSIZE);
708 if (inbuffer) setvbuf( inifile, inbuffer, _IOFBF, BUFSIZE );
709
710 /* Abschnittsnamen initialisieren: */
711 sectionnamemaxlen = 0;
712 sectionname = INI_STRALLOC(sectionnamemaxlen);
713 if ( !sectionname ) ERROR_RETURN( inifilename, lineno, INI_ERR_NOMEMORY );
714 *sectionname = '\0';
715
716 /* Alle Zeilen durchgehen: */
717 while ( fgets( minibuf, MINIBUFSIZE, inifile ) ) {
718     lineno++;
719     for ( temp1=minibuf; INI_ISSPACE(*temp1) && *temp1; temp1++ ) ; /* Whitespace am Zeilenanfang
720     überspringen. */
721     if (!*temp1) continue;
722     for ( temp2=temp1+strlen(temp1)-1; INI_ISSPACE(*temp2); temp2-- ) *temp2 = '\0'; /* Whitespace am
723     Zeilenende ignorieren. */
724
725     if ( strchr( "##;", *temp1 ) ) {
726         /* Kommentar in einer gesonderten Liste speichern. */
727         n = ini_add_comment( temp1+1 );
728         if ( n == INI_ERR_NOMEMORY ) ERROR_RETURN( inifilename, lineno, INI_ERR_NOMEMORY );
729         continue;
730     }
731
732     if ( ( *temp1 == '[' ) && ( *temp2 == ']' ) ) {
733         /* Abschnittsname gefunden. */
734         for ( temp3=temp1+1; INI_ISSPACE(*temp3) && ( temp3 < temp2 ); temp3++ ) ;
735         if ( temp3 == temp2 ) {
736             /* Sonderfall: leerer Abschnitt (oder Abschnittsname nur aus Blanks) => unbenannter Abschnitt */
737             *sectionname = '\0';
738         } else {
739             /* Normalfall: ein echter Abschnittsname */
740             i = (size_t)( temp2 - temp1 + 1 );
741             if ( i > sectionnamemaxlen ) {

```

```

740     sectionnamemaxlen = i;
741     sectionname = INI_STRREALLOC( sectionname, sectionnamemaxlen );
742     if ( !sectionname ) ERROR_RETURN( inifilename, lineno, INI_ERR_NOMEMORY );
743 }
744 INI_COPYSTR( sectionname, temp1, i );
745 }
746 } else {
747     temp2 = strchr( temp1, '=' ); /* Prüfung auf normalen Ini-Eintrag */
748     temp3 = strchr( temp1, ':' ); /* Prüfung auf Pragma */
749     if ( temp3 && ( !temp2 || ( temp3 < temp2 ) ) ) {
750         /* Pragma gefunden */
751         *( temp3++ ) = '\0';
752         n = handle_pragma( temp1, isolate_string( temp3, 0 ), &npragma, &pragmaval );
753         /* In Pragmata keine Escape-Ersetzung! */
754         if ( n < 0 ) {
755             /* Kann nur NOSUCHPRAGMA oder NOMEMORY sein. In beiden Fällen die weitere Bearbeitung
756              * dieser Ini-Datei einstellen und das Problem nach oben melden.
757              */
758             if ( errorlinenumber < 1 ) {
759                 /* Markiere, wo der Fehler aufgefallen ist: */
760                 ERROR_RETURN( inifilename, lineno, n );
761             } else {
762                 /* Wir reichen nur eine bereits eingetragene Meldung weiter nach oben. */
763                 return n;
764             }
765         }
766         if ( npragma == 2 ) resolve_escapes = pragmaval; /* falls dies das resolve_escapes-Pragma war */
767     } else {
768         /* Kein Pragma. */
769         if ( temp2 != temp1 ) {
770             /* kein leerer Schlüssel */
771             if ( !temp2 ) { /* kein Gleichheitszeichen, also: leerer Wert */
772                 temp2 = temp1 + strlen(temp1);
773             } else { /* Gleichheitszeichen gefunden; Wert kann immer noch leer sein. */
774                 /* Den Schlüssel isolieren: */
775                 for ( temp3=temp2-1; INI_ISSPACE(*temp3) && (temp3>=temp1); temp3-- ) *temp3 = '\0';
776                 if ( *temp2 == '=' ) *temp2 = '\0';
777                 temp2++;
778             }
779             /* temp1 zeigt jetzt auf den Key (möglicherweise leerer String!),
780              * temp2 auf den Wert (möglicherweise leerer String!)
781              */
782             temp2 = isolate_string( temp2, resolve_escapes );
783             /* ggf. Anführungszeichen um Wert entfernen: */
784             if ( ( strlen(temp2) > 1 ) && ( ( *temp2 == '"' ) || ( *temp2 == '\'' ) ) &&
785                 ( *( temp2+strlen(temp2)-1 ) == *temp2 ) ) {
786                 temp2++;
787                 *(temp2+strlen(temp2)-1) = '\0';
788             }
789             if ( strlen(sectionname) ) {
790                 /* Abschnittsname vorhanden: */
791                 temp3 = INI_STRALLOC( strlen(sectionname)+strlen(temp1) );
792                 if ( !temp3 ) ERROR_RETURN( inifilename, lineno, INI_ERR_NOMEMORY );
793                 INI_COPYSTR( temp3, sectionname, strlen(sectionname) );
794                 (void)strcat( temp3, temp1 );
795                 if ( !ini_add( temp3, temp2 ) ) ERROR_RETURN( inifilename, lineno, INI_ERR_NOMEMORY );
796                 INI_FREE(temp3);
797             } else {
798                 /* Ein wahrscheinlich häufiger Sonderfall: kein benannter Abschnitt, daher schnelleres
799                  * Handling: */
800                 if ( !ini_add( temp1, temp2 ) ) ERROR_RETURN( inifilename, lineno, INI_ERR_NOMEMORY );
801             }
802         }
803     }
804 }
805 fclose(inifile);
806 INI_FREE(inbuffer);
807 INI_FREE(sectionname);
808
809 /* Nun unseren Namen aus der ininames-Liste streichen, damit wir nicht fälschlich Rekursion vermelden:
810  */
811 ininames = ininameentry->next;
812 INIS_FREE( ininameentry, sizeof( *ininameentry ) );

```

8.1.3.9 ini_add() int ini_add (

 const char * key,

 const char *const value)

Fügt einen Konfigurationseintrag (Schlüssel/Wert) hinzu.

Parameter

in	key	Der Schlüssel (darf auch leer oder NULL sein). Der Schlüssel darf am Anfang in eckigen Klammern einen Abschnittsnamen enthalten.
in	value	Der Wert (darf leer sein). Falls NULL, wird der entsprechende Eintrag gelöscht.

Rückgabe

Anzahl der nach dem Hinzufügen verfügbaren Einträge. INI_ERR_NOMEMORY, falls nicht genügend Speicherplatz verfügbar.

Definiert in Zeile 825 der Datei IniFile.c.

```

828                                     {
829     TIniEntry *inilistptr = inilist;
830     TIniEntry *previousinilistptr = NULL;
831     TIniEntry *newinilistentry;
832     char *tempkey;
833
834     if ( !got_license ) {
835         fprintf( stderr, "This software uses the IniFile library.\n%s", COPYRIGHT );
836         got_license = 1;
837     }
838
839     if ( !value ) return ini_delete(key); /* NULL-Wert: dann löschen. */
840
841     /* Eintrag grafieunabhängig suchen in der Liste: */
842     JUMP_EMPTY_SECTION(key);
843     tempkey = INI_STRALLOC( INI_STRLen(key)); /* NULL-Schlüssel wie leeren Schlüssel behandeln. */
844     if ( !tempkey ) return INI_ERR_NOMEMORY;
845     INI_COPYSTRUPPER( tempkey, key );
846
847     while ( inilistptr && ( strcmp( tempkey, inilistptr->key ) > 0 ) ) {
848         previousinilistptr = inilistptr;
849         inilistptr = inilistptr->next;
850     }
851
852     if ( inilistptr && INI_STREQUAL( tempkey, inilistptr->key ) ) {
853         /* gefunden; also alten Wert überschreiben */
854         inilistptr->value = INI_STRREALLOC( inilistptr->value, strlen(value) );
855         if ( !inilistptr->value ) {
856             INI_FREE(tempkey);
857             return INI_ERR_NOMEMORY;
858         }
859         (void)strcpy( inilistptr->value, value );
860         INI_FREE(tempkey);
861     } else {
862         /* Neuen Eintrag hinzufügen. */
863         newinilistentry = (TIniEntry *)INI_MALLOC( sizeof(*newinilistentry) );
864         if ( !newinilistentry ) {
865             INI_FREE(tempkey);
866             return INI_ERR_NOMEMORY;
867         }
868         newinilistentry->key = tempkey;
869         newinilistentry->value = INI_STRALLOC( strlen(value) );
870         if ( !newinilistentry->value ) {
871             INI_FREE(tempkey);
872             INI_FREE(newinilistentry);
873             return INI_ERR_NOMEMORY;
874         }
875         (void)strcpy( newinilistentry->key, tempkey );
876         (void)strcpy( newinilistentry->value, value );
877         /* Nachfolger des neuen Eintrags wird der in der Sortierung nächstgrößere Alteintrag. */
878         newinilistentry->next = inilistptr;
879
880         if ( !previousinilistptr ) {
881             /* Einfügung am Anfang, also Anfangspointer neu setzen. */
882             inilist = newinilistentry;
883         } else {
884             /* Der in der Sortierung voran gehende Eintrag erhält den neuen als Nachfolger. */
885             previousinilistptr->next = newinilistentry;
886         }
887         entrycount++;
888     }
889
890     sectioncount = 0; /* Gecachten Wert invalidieren */

```

8.1.3.10 ini_set() `int __inline ini_set (`
 `const char * key,`
 `const char *const value )`

Fügt einen Konfigurationseintrag (Schlüssel/Wert) hinzu; Synonym für [ini_add\(\)](#).

Synonym für [ini_add\(\)](#):

Parameter

<code>in</code>	<code>key</code>	Der Schlüssel (darf auch leer oder NULL sein). Der Schlüssel darf am Anfang in eckigen Klammern einen Abschnittsnamen enthalten.
<code>in</code>	<code>value</code>	Der Wert (darf leer sein). Falls NULL, wird der entsprechende Eintrag gelöscht.

Rückgabe

Anzahl der nach dem Einlesen verfügbaren Einträge. INI_ERR_NOMEMORY, falls nicht genügend Speicherplatz verfügbar.

Siehe auch

[ini_add\(\)](#)

Definiert in Zeile 906 der Datei IniFile.c.

```
909 { return ini_add(key, value); }
```

8.1.3.11 ini_nkeys() `int __inline ini_nkeys (`
 `void )`

Liefert die Anzahl der Konfigurationseinträge zurück.

Rückgabe

Die Anzahl der Konfigurationseinträge.

Definiert in Zeile 914 der Datei IniFile.c.

```
917 { return entrycount; }
```

8.1.3.12 ini_nkeys_section() `int ini_nkeys_section (`
 `const char * sectionname )`

Liefert die Anzahl der Konfigurationseinträge in einem gegebenen Abschnitt zurück.

Parameter

<code>in</code>	<code>sectionname</code>	Name des Abschnitts. Kann auch NULL, leer oder [] sein. Begrenzende eckige Klammern dürfen fehlen.
-----------------	--------------------------	--

Rückgabe

Die Anzahl der Konfigurationseinträge in dem genannten Abschnitt.

Definiert in Zeile 925 der Datei IniFile.c.

```

928                                     {
929     TIniEntry *inilistptr;
930     char *mysectionname;
931     size_t sectionnamelen;
932     int keycount = 0;
933
934     mysectionname = make_section_name(sectionname);
935     if ( !mysectionname ) return INI_ERR_NOMEMORY;
936     sectionnamelen = strlen(mysectionname);
937
938     /* Alle Einträge durchgehen, dabei nur die passenden berücksichtigen. */
939     for ( inilistptr=inilist; inilistptr; inilistptr=inilistptr->next ) {
940         if ( ( ( sectionnamelen != 0 ) &&
941             ( strlen( inilistptr->key ) >= sectionnamelen ) &&
942             ( memcmp( inilistptr->key, mysectionname, sectionnamelen ) == 0 )
943             ) ||
944             ( ( sectionnamelen == 0 ) &&
945             ( ( strlen(inilistptr->key) == 0 ) || ( *(inilistptr->key) != '[' ) )
946             )
947         ) keycount++;
948     }
949     INI_FREE(mysectionname);

```

8.1.3.13 ini_nsections() int ini_nsections (

void)

Liefert die Anzahl der Konfigurationsabschnitte zurück.

Rückgabe

Die Anzahl der Abschnitte (inkl. des unbenannten Abschnitts).

Definiert in Zeile 957 der Datei IniFile.c.

```

960                                     {
961     TIniEntry *inilistptr;
962     char *sectionname, *temp;
963     size_t sectionnamemaxlen, i;
964     int foundunbenannt = 0;
965
966     if (sectioncount) return sectioncount; /* Gecacheter Wert liegt vor, alle sind froh. */
967
968     sectionnamemaxlen = 0;
969     sectionname = INI_STRALLOC(sectionnamemaxlen);
970     if ( !sectionname ) return INI_ERR_NOMEMORY;
971
972     *sectionname = '\0';
973
974     /* Alle Einträge durchgehen, dabei Sektionsnamen zählen.
975     * Wir wissen: Die Einträge sind grafieunabhängig sortiert, brauchen also keine
976     * Historie der gesehenen Einträge nachzuhalten. */
977     for ( inilistptr=inilist; inilistptr; inilistptr=inilistptr->next ) {
978         if ( strlen(inilistptr->key) ) {
979             if ( *(inilistptr->key) == '[' ) {
980                 /* Eintrag mit benanntem Abschnitt */
981                 temp = strchr( inilistptr->key, ']' );
982                 if (temp) { /* Sicherheitshalber; alles andere wäre sehr merkwürdig. */
983                     i = (size_t)( temp - inilistptr->key + 1 );
984                     if ( ( i != strlen(sectionname) ) || ( memcmp( sectionname, inilistptr->key, i ) != 0 ) ) {
985                         /* Neuer Abschnittsname gefunden! */
986                         sectioncount++;
987                         if ( i > sectionnamemaxlen ) {
988                             sectionnamemaxlen = i;
989                             sectionname = INI_STRREALLOC( sectionname, sectionnamemaxlen );
990                             if ( !sectionname ) return INI_ERR_NOMEMORY;
991                         }
992                         INI_COPYSTR( sectionname, inilistptr->key, i );
993                     }
994                 }

```



```

995     } else {
996         /* Unbenannter Abschnitt. */
997         if ( !foundunbenannt ) {
998             foundunbenannt = 1;
999             sectioncount++;
1000         }
1001         *sectionname = '\0';
1002     }
1003 }
1004 }
1005 INI_FREE(sectionname);

```

8.1.3.14 ini_keys() char** ini_keys (
char ** keylist,
int maxkeys)

Liefert eine Liste der Schlüssel.

Liefert eine Liste aller definierten Schlüssel zurück.

Parameter

out	<i>keylist</i>	Pointer auf ein Array von Strings, das die Schlüssel aufnimmt. Schlüssel in benannten Abschnitten enthalten am Anfang den Abschnittsnamen in eckigen Klammern. Wird terminiert durch einen NULL-Pointer, sofern Platz dafür vorhanden ist. Platz für die Strings wird dynamisch alloziert. Kann NULL sein; dann wird Speicher dynamisch alloziert.
in	<i>maxkeys</i>	Maximal zurückzuliefernde Anzahl von Schlüssel. Kann ≤ 0 sein, dann keine Begrenzung. keylist muss (falls ungleich NULL) jedenfalls groß genug sein, um diese Einträge und den terminierenden NULL-Eintrag aufzunehmen.

Rückgabe

Pointer auf das Array von Strings; oder NULL, falls nicht genug Speicherplatz.

Siehe auch

[ini_nkeys\(\)](#)

Fehler Bei Abbruch wegen Speichermangel wird bereits allozierter Speicher nicht wieder freigegeben.

Definiert in Zeile 1031 der Datei IniFile.c.

```

1034                                     {
1035
1036     TIniEntry *inilistptr = inilist;
1037     char **keyptr;
1038     int keycount;
1039
1040     if ( !keylist ) {
1041         /* Speicherplatz allozieren. */
1042         keycount = ini_nkeys() + 1;
1043         if ( ( maxkeys > 0 ) && ( keycount > maxkeys ) ) keycount = maxkeys;
1044         if ( maxkeys <= 0 ) maxkeys = keycount;
1045         keylist = (char **)INI_MALLOC( ( (unsigned)keycount+1 ) * sizeof(char *) );
1046         if ( !keylist ) return NULL;
1047     }
1048
1049     /* Alle Einträge bis zur erlaubten Maximalzahl eintragen. */
1050     for ( keycount=0, keyptr=keylist; inilistptr && (keycount < maxkeys); keycount++,
        inilistptr=inilistptr->next ) {
1051         *keyptr = INI_STRALLOC( strlen(inilistptr->key) );
1052         if ( !keyptr ) return NULL;
1053         (void)strcpy( *(keyptr++), inilistptr->key );
1054     }
1055
1056     if ( keycount < maxkeys ) *keyptr = (char *)NULL; /* Terminierenden NULL-Pointer eintragen */

```

8.1.3.15 ini_keys_section() `char** ini_keys_section (`
`char ** keylist,`
`int maxkeys,`
`const char * sectionname )`

Liefert eine Liste der Schlüssel in einem gegebenen Abschnitt.

Liefert eine Liste aller definierten Schlüssel in einem vorgegebenen Abschnitt zurück.

Parameter

out	<i>keylist</i>	Pointer auf ein Array von Strings, das die Schlüssel aufnimmt. Schlüssel in benannten Abschnitten enthalten am Anfang den Abschnittsnamen in eckigen Klammern. Wird terminiert durch einen NULL-Pointer, sofern Platz dafür vorhanden ist. Platz für die Strings wird dynamisch alloziert. Kann NULL sein; dann wird Speicher dynamisch alloziert.
in	<i>maxkeys</i>	Maximal zurückzuliefernde Anzahl von Schlüssel. Kann ≤ 0 sein, dann keine Begrenzung. keylist muss (falls ungleich NULL) jedenfalls groß genug sein, um diese Einträge und den terminierenden NULL-Eintrag aufzunehmen.
in	<i>sectionname</i>	Name des Abschnitts. Kann auch NULL, leer oder [] sein. Begrenzende eckige Klammern dürfen fehlen.

Rückgabe

Pointer auf das Array von Strings; oder NULL, falls nicht genug Speicherplatz.

Siehe auch

[ini_nkeys_section\(\)](#)

Fehler Bei Abbruch wegen Speichermangel wird bereits allozierter Speicher nicht wieder freigegeben.

Definiert in Zeile 1084 der Datei IniFile.c.

```

1087
1088
1089     TIniEntry *inilistptr = inilist;
1090     char **keyptr;
1091     char *mysectionname;
1092     size_t sectionnamelen;
1093     int keycount;
1094
1095     mysectionname = make_section_name(sectionname);
1096     if ( !mysectionname ) return NULL;
1097     sectionnamelen = strlen(mysectionname);
1098
1099     if ( !keylist ) {
1100         /* Speicherplatz allozieren. */
1101         keycount = ini_nkeys_section(mysectionname) + 1;
1102         if ( ( maxkeys > 0 ) && ( keycount > maxkeys ) ) keycount = maxkeys;
1103         if ( maxkeys <= 0 ) maxkeys = keycount;
1104         keylist = (char **)INI_MALLOC( ( (unsigned)keycount+1 ) * sizeof(char *) );
1105         if ( !keylist ) return NULL;
1106     }
1107
1108     /* Alle Einträge durchgehen, dabei nur die passenden berücksichtigen, und höchstens bis zur
1109     Maximalzahl. */
1109     for ( inilistptr=inilist, keyptr=keylist, keycount = 0; inilistptr && ( keycount < maxkeys );
1110           inilistptr=inilistptr->next ) {
1111         if ( ( ( sectionnamelen != 0 ) &&
1112               ( strlen(inilistptr->key) >= sectionnamelen ) &&
1113               ( memcmp( inilistptr->key, mysectionname, sectionnamelen ) == 0 )
1114               ) ||
1114           ( ( sectionnamelen == 0 ) && ( ( strlen(inilistptr->key) == 0 ) || ( *(inilistptr->key) != '['
1115           ) ) ) ) {

```

```

1116     *keyptr = INI_STRALLOC( strlen(inilistptr->key) );
1117     if ( !*keyptr ) return NULL;
1118     (void)strcpy( *(keyptr++), inilistptr->key );
1119     keycount++;
1120 }
1121 }
1122
1123 if ( keycount < maxkeys ) *keyptr = (char *)NULL; /* Terminierenden NULL-Pointer eintragen */

```

8.1.3.16 ini_sections() char** ini_sections (

char ** sectionlist,

int maxsections)

Liefert eine Liste der Konfigurationsabschnitte zurück.

Liefert eine Liste aller definierten Abschnitte (Sections) zurück.

Parameter

out	<i>sectionlist</i>	Pointer auf ein Array von Strings, das die Abschnittsnamen aufnimmt. Die Abschnittsnamen enthalten begrenzende eckige Klammern, der unbenannte Abschnitt ist jedoch ein leerer String. Wird terminiert durch einen NULL-Pointer, sofern Platz dafür vorhanden ist. Platz für die Strings wird dynamisch alloziert. Kann NULL sein; dann wird Speicher dynamisch alloziert.
in	<i>maxsections</i>	Maximal zurückzuliefernde Anzahl von Abschnitten. Kann ≤ 0 sein, dann keine Begrenzung. sectionlist muss (falls ungleich NULL) jedenfalls groß genug sein, um diese Einträge und den terminierenden NULL-Eintrag aufzunehmen.

Rückgabe

Pointer auf das Array von Strings; oder NULL, falls nicht genug Speicherplatz.

Siehe auch

[ini_nsections\(\)](#)

Fehler Bei Abbruch wegen Speichermangel wird bereits allozierter Speicher nicht wieder freigegeben.

Definiert in Zeile 1149 der Datei IniFile.c.

```

1152                                     {
1153     TIniEntry *inilistptr;
1154     char **sectionptr;
1155     char *sectionname, *temp;
1156     size_t sectionnamemaxlen, i;
1157     int mysectioncount = 0;
1158     int foundunbenannt = 0;
1159     int isNewSection;
1160
1161     if ( !sectioncount ) {
1162         /* Nötigenfalls die Anzahl der Abschnitte neu ermitteln */
1163         if ( ini_nsections() < 0 ) return NULL;
1164     }
1165
1166     if ( !sectionlist ) {
1167         /* Speicherplatz allozieren. */
1168         mysectioncount = sectioncount + 1;
1169         if ( ( maxsections > 0 ) && ( mysectioncount > maxsections ) ) mysectioncount = maxsections;
1170         if ( maxsections <= 0 ) maxsections = mysectioncount;
1171         sectionlist = (char **)INI_MALLOC( ( (unsigned)mysectioncount+1 ) * sizeof(char * ) );
1172         if ( !sectionlist ) return NULL;
1173     }

```

```

1174
1175  /* Abschnittsnamen initialisieren: */
1176  sectionnamemaxlen = 0;
1177  sectionname = INI_STRALLOC(sectionnamemaxlen);
1178  if ( !sectionname ) return NULL;
1179  *sectionname = '\0';
1180
1181  /* Alle Einträge durchgehen und bei jedem neuen Abschnittsnamen diesen in die Liste übernehmen.
1182   * Wir wissen: Die Einträge sind grafieunabhängig sortiert, brauchen also keine
1183   * Historie der gesehenen Einträge zu checken. */
1184  for ( inilistptr=inilist, sectionptr=sectionlist; inilistptr; inilistptr=inilistptr->next ) {
1185      if ( strlen(inilistptr->key) ) {
1186          isNewSection = 0;
1187          if ( *(inilistptr->key) == '[' ) {
1188              /* Eintrag mit benamstem Abschnitt */
1189              temp = strchr( inilistptr->key, ']' );
1190              if (temp) { /* Sicherheitshalber; alles andere wäre sehr merkwürdig. */
1191                  i = (size_t)( temp - inilistptr->key + 1 );
1192                  if ( (i != strlen(sectionname)) || ( memcmp( sectionname, inilistptr->key, i ) != 0 ) ) {
1193                      /* Neuer Abschnittsnamen gefunden! */
1194                      if ( i > sectionnamemaxlen ) {
1195                          sectionnamemaxlen = i;
1196                          sectionname = INI_STRREALLOC( sectionname, sectionnamemaxlen );
1197                          if ( !sectionname ) return NULL;
1198                      }
1199                      INI_COPYSTR( sectionname, inilistptr->key, i );
1200                      isNewSection = 1;
1201                  }
1202              }
1203          } else {
1204              /* Unbenannter Abschnitt. */
1205              if ( !foundunbenannt ) {
1206                  foundunbenannt = 1;
1207                  isNewSection = 1;
1208              }
1209              *sectionname = '\0';
1210          }
1211          if (isNewSection) {
1212              /* Neuen Eintrag in die Liste übernehmen */
1213              *sectionptr = INI_STRALLOC( strlen(sectionname) );
1214              if ( !*sectionptr ) return NULL;
1215              (void)strcpy( *(sectionptr++), sectionname );
1216          }
1217      }
1218  }
1219
1220  if (mysectioncount < maxsections) *sectionptr = (char *)NULL; /* Terminierenden NULL-Pointer
eintragen */

```

8.1.3.17 ini_ncomments() int __inline ini_ncomments (

void)

Liefert die Anzahl der Kommentare zurück.

Rückgabe

Die Anzahl der Kommentare.

Definiert in Zeile 1228 der Datei IniFile.c.

```
1231 { return commentcount; }
```

8.1.3.18 ini_comments() char** ini_comments (

char ** commentlist,

int maxcomment)

Liefert eine Liste der Kommentare.

Liefert eine Liste aller Kommentare zurück.

Parameter

out	<i>commentlist</i>	Pointer auf ein Array von Strings, das die Kommentare aufnimmt. Wird terminiert durch einen NULL-Pointer, sofern Platz dafür vorhanden ist. Platz für die Strings wird dynamisch alloziert. Kann NULL sein; dann wird Speicher dynamisch alloziert.
in	<i>maxcomment</i>	Maximal zurückzuliefernde Anzahl von Kommentaren. Kann ≤ 0 sein, dann keine Begrenzung. commentlist muss (falls ungleich NULL) jedenfalls groß genug sein, um diese Einträge und den terminierenden NULL-Eintrag aufzunehmen.

Rückgabe

Pointer auf das Array von Strings; oder NULL, falls nicht genug Speicherplatz.

Siehe auch

[ini_ncomments\(\)](#)

Fehler Bei Abbruch wegen Speichermangel wird bereits allozierter Speicher nicht wieder freigegeben.

Definiert in Zeile 1252 der Datei IniFile.c.

```

1255                                     {
1256
1257     TIniEntry *inicommentptr = comments;
1258     char **commentptr;
1259     int commentcount;
1260
1261     if ( !commentlist ) {
1262         /* Speicherplatz allozieren. */
1263         commentcount = ini_ncomments( ) + 1;
1264         if ( ( maxcomment > 0 ) && ( commentcount > maxcomment ) ) commentcount = maxcomment;
1265         if ( maxcomment <= 0 ) maxcomment = commentcount;
1266         commentlist = (char **)INI_MALLOC( ( (unsigned)commentcount+1 ) * sizeof(char *) );
1267         if ( !commentlist ) return NULL;
1268     }
1269
1270     /* Alle Kommentare bis zur erlaubten Maximalzahl eintragen. */
1271     for ( commentcount=0, commentptr=commentlist; inicommentptr && (commentcount < maxcomment);
1272           commentcount++, inicommentptr=inicommentptr->next ) {
1273         *commentptr = INI_STRALLOC( strlen(inicommentptr->value) );
1274         if ( !*commentptr ) return NULL;
1275         (void)strcpy( *(commentptr++), inicommentptr->value );
1276     }
1277     if ( commentcount < maxcomment ) *commentptr = (char *)NULL; /* Terminierenden NULL-Pointer eintragen
1278     */

```

8.1.3.19 ini_add_comment() int ini_add_comment (
const char * text)

Fügt eine Kommentarzeile hinzu.

Parameter

in	<i>text</i>	Der Kommentar (ohne einleitendes Kommentarzeichen; darf leer sein).
----	-------------	---

Rückgabe

Anzahl der nach dem Hinzufügen verfügbaren Kommentarzeilen. INI_ERR_NOMEMORY, falls nicht genügend Speicherplatz verfügbar.

Definiert in Zeile 1288 der Datei IniFile.c.

```

1291                                     {
1292     TIniEntry *commentptr = comments;
1293     TIniEntry *previouscommentptr = NULL;
1294     TIniEntry *newcomment;
1295
1296     if ( !text ) return commentcount;
1297
1298     /* Neuen Eintrag hinzufügen. */
1299     newcomment = (TIniEntry *)INI_MALLOC( sizeof(*newcomment) );
1300     if ( !newcomment ) return INI_ERR_NOMEMORY;
1301     newcomment->key = NULL;
1302     newcomment->next = NULL;
1303     newcomment->value = INI_STRALLOC( strlen(text) );
1304     if ( !newcomment->value ) {
1305         INI_FREE(newcomment);
1306         return INI_ERR_NOMEMORY;
1307     }
1308     (void)strcpy( newcomment->value, text );
1309
1310     if ( commentptr ) {
1311         /* Wir haben schon mindesetns einen Kommentar.
1312          * Den neuen Eintrag ans Ende der Kommentarliste anhängen:
1313          */
1314         while ( commentptr->next ) commentptr = commentptr->next;
1315         commentptr->next = newcomment;
1316     } else {
1317         /* Dies ist unser erster Kommentar. */
1318         comments = newcomment;
1319     }

```

8.1.3.20 ini_delete() int ini_delete (

const char * key)

Löscht einen Schlüssel, falls es ihn gibt.

Parameter

in	key	Der Schlüssel (darf auch leer oder NULL sein). Der Schlüssel darf am Anfang in eckigen Klammern einen Abschnittsnamen enthalten.
----	-----	--

Rückgabe

Anzahl der nach dem Einlesen verfügbaren Einträge. INI_ERR_NOMEMORY, falls nicht genügend Speicherplatz verfügbar.

Definiert in Zeile 1331 der Datei IniFile.c.

```

1334                                     {
1335     TIniEntry *inilistptr = inilist;
1336     TIniEntry *previousinilistptr = NULL;
1337     char *tempkey;
1338
1339     /* Eintrag grafieunabhängig suchen in der Liste: */
1340     JUMP_EMPTY_SECTION(key);
1341     tempkey = INI_STRALLOC( INI_STRLEN(key) ); /* NULL-Schlüssel wie leeren Schlüssel behandeln. */
1342     if ( !tempkey ) return INI_ERR_NOMEMORY;
1343     INI_COPYSTRUPPER(tempkey, key);
1344
1345     while ( inilistptr && ( strcmp( tempkey, inilistptr->key ) > 0 ) ) {
1346         previousinilistptr = inilistptr;
1347         inilistptr = inilistptr->next;
1348     }
1349

```

```

1350 if ( inilistptr && INI_STREQUAL( tempkey, inilistptr->key ) ) {
1351     /* gefunden; also löschen. */
1352     if ( !previousinilistptr ) {
1353         /* Es war der erste EWintrag der Liste; also Listenanfang neu setzen. */
1354         inilist = inilistptr->next;
1355     } else {
1356         /* Im voran gehenden Eintrag den Pointer auf das nächste Element umsetzen. */
1357         previousinilistptr->next = inilistptr->next;
1358     }
1359     /* Den von diesem Eintrag belegten Speicher freigeben. */
1360     INI_FREE(inilistptr->key);
1361     INI_FREE(inilistptr->value);
1362     INI_FREE(inilistptr);
1363     entrycount--;
1364 }
1365 INI_FREE(tempkey);
1366
1367 sectioncount = 0; /* Gecacheten Wert invalidieren */

```

8.1.3.21 ini_write() int ini_write (

const char *const inifilename,

const char * comment)

Schreibt die aktuellen Ini-Werte in eine Datei.

Schreibt den aktuellen Gesamtbestand der Schlüssel-/Werte-Paare in eine Datei, wahlweise mit einem Kommentar. Platzhalter werden nicht aufgelöst, sondern wörtlich ausgeschrieben.

Parameter

in	<i>inifilename</i>	Name der zu schreibenden Datei. Wenn NULL oder leer, wird nach stdout geschrieben.
in	<i>comment</i>	An den Anfang der Datei zu schreibender Kommentar. Wenn ein solcher angegeben ist, werden bisher vorhandene (eingeliesene oder mit ini_add_comment() hinzugefügte) Kommentare ignoriert. Kann mehrzeilig sein. Kann NULL sein.

Rückgabe

0, falls OK. INI_ERR_NOMEMORY, falls nicht genügend Speicherplatz verfügbar.

Siehe auch

[ini_nkeys\(\)](#)

Definiert in Zeile 1390 der Datei IniFile.c.

```

1393
1394
1395 FILE *inifile;
1396 TIniEntry *inilistptr;
1397 char *outbuffer = NULL;
1398 static const char * const delims[ ] = { " ", "\"", "'" };
1399 static const char * const nonprintables =
1400     "\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20\x21\x22\x23\x24\x25\x26\x27\x28\x29\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x40\x41\x42\x43\x44\x45\x46\x47\x48\x49\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x60\x61\x62\x63\x64\x65\x66\x67\x68\x69\x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x80\x81\x82\x83\x84\x85\x86\x87\x88\x89\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x00";
1401 char minibuf[MINIBUFSIZE];
1402 char *temp, *temp2, *keyptr, *sectionname;
1403 const char *templ;
1404 size_t sectionnamemaxlen, i;
1405 int isError, isNewSection, hadNamedSection, hasNonprintables, resolve_escapes=0;
1406
1407 isError = 0;
1408 if ( INI_STRLEN(inifilename) ) {
1409     inifile = fopen( inifilename, "w" );
1410     if (!inifile) return INI_ERR_NOFILE;
1411     outbuffer = (char *)INI_MALLOC(BUFSIZE);
1412     if (outbuffer) setvbuf( inifile, outbuffer, _IOFBF, BUFSIZE );

```

```

1412 } else {
1413     inifile = stdout;
1414 }
1415
1416 if ( INI_STRLen(comment) ) {
1417     /* Explizit angegebene neue Kommentare schreiben; bisher vorhandene Kommentare ignorieren. */
1418     while ( INI_STRLen(comment) ) {
1419         templ = strchr(comment, '\n');
1420         if (!templ) templ = comment + strlen(comment);
1421         if ( fprintf( inifile, "# " ) < 0 ) isError = 1;
1422         if ( !isError ) {
1423             if ( fwrite( comment, sizeof(char), (size_t)( templ-comment ), inifile ) < (size_t)(
templ-comment ) ) isError = 1;
1424         }
1425         if ( !isError ) {
1426             if ( fprintf( inifile, "\n" ) < 0 ) isError = 1;
1427         }
1428         if (isError) break;
1429         comment = ( *templ == '\0' ) ? templ : (templ+1);
1430     }
1431 } else {
1432     /* Ggf. bereits vorher vorhandene Kommentare schreiben: */
1433     inilistptr = comments;
1434     while (inilistptr) {
1435         if ( fprintf( inifile, "# %s\n", inilistptr->value ) < 0 ) isError = 1;
1436         inilistptr = inilistptr->next;
1437     }
1438 }
1439
1440 /* Abschnittsnamen initialisieren: */
1441 sectionnamemaxlen = 0;
1442 sectionname = INI_STRALLOC(sectionnamemaxlen);
1443 if ( !sectionname ) return INI_ERR_NOMEMORY;
1444 *sectionname = '\0';
1445 hadNamedSection = 0;
1446
1447 /* Alle Einträge schreiben. */
1448 for ( inilistptr=inilist; inilistptr && !isError; inilistptr=inilistptr->next ) {
1449     keyptr = inilistptr->key ? inilistptr->key : delims[0];
1450     /* Sicherheitshalber Zeilenende-Zeichen auf Blank umsetzen. (Inplace -- you get what you pay for.)
1451     */
1452     while ( ( temp = strchr( keyptr, '\n' ) ) != NULL ) *temp = ' ';
1453     while ( ( temp = strchr( keyptr, '\r' ) ) != NULL ) *temp = ' ';
1454
1455     /* Prüfung auf Abschnittswchsel: */
1456     isNewSection = 0;
1457     if ( *keyptr == '[' ) {
1458         /* Eintrag mit benamstem Abschnitt */
1459         temp = strchr( keyptr, '[' );
1460         if (temp) { /* Sicherheitshalber; alles andere wäre sehr merkwürdig. */
1461             i = (size_t)( temp - keyptr + 1 );
1462             if ( ( i != strlen(sectionname) ) || ( memcmp( sectionname, keyptr, i ) != 0 ) ) {
1463                 /* Neuer Abschnittsname gefunden! */
1464                 if ( i > sectionnamemaxlen ) {
1465                     sectionnamemaxlen = i;
1466                     sectionname = INI_STRREALLOC( sectionname, sectionnamemaxlen );
1467                     if ( !sectionname ) return INI_ERR_NOMEMORY;
1468                 }
1469                 INI_COPYSTR( sectionname, keyptr, i );
1470                 isNewSection = 1;
1471             }
1472             keyptr = temp + 1;
1473             hadNamedSection = 1;
1474         }
1475     } else {
1476         *sectionname = '\0';
1477     }
1478
1479     if ( isNewSection ) {
1480         /* Neuer Abschnitt. Abschnittskennung muss geschrieben werden, falls
1481         * (i) benannter Abschnitt, oder (ii) unbenannter Abschnitt nach benanntem Abschnitt. */
1482         if ( INI_STRLen(sectionname) ) {
1483             if ( fprintf( inifile, "%s\n", sectionname ) < 0 ) isError = 1;
1484         } else {
1485             if (hadNamedSection) {
1486                 if ( fprintf( inifile, "[]\n" ) < 0 ) isError = 1;
1487             }
1488         }
1489     }
1490
1491     if ( !INI_STRLen(inilistptr->value) ) {
1492         /* Leerer Wert: */
1493         if ( fprintf( inifile, "%s=\n", keyptr ) < 0 ) isError = 1;
1494     } else {
1495         /* Nichtleerer Wert. */
1496         /* Prüfung auf nichtdruckbare Sonderzeichen unterhalb von \x20 (nur im Value): */

```



```

1497     hasNonprintables = ( strstr( inilistptr->value, nonprintables ) < strlen( inilistptr->value ) );
1498     if ( hasNonprintables && !resolve_escapes ) {
1499         if ( fprintf( inifile, "resolve_escapes: 1\n" ) < 0 ) isError = 1;
1500         resolve_escapes = 1;
1501     }
1502
1503     if ( hasNonprintables || ( resolve_escapes && strchr( inilistptr->value, '\\\' ) ) ) {
1504         /* Zeichen für Zeichen kopieren und dabei ggf. in Hex umsetzen: */
1505         for ( temp=inilistptr->value, temp2=minibuf; *temp; temp++, temp2++ ) {
1506             if ( *temp < ' ' ) {
1507                 if ( temp2-minibuf+4 >= MINIBUFSIZE ) break;
1508                 sprintf( temp2, "\\x%02x", *temp );
1509                 temp2 += 3;
1510             } else if ( *temp == '\\\' ) {
1511                 if ( temp2-minibuf+2 >= MINIBUFSIZE ) break;
1512                 *(temp2++) = *temp;
1513                 *temp2 = *temp;
1514             } else {
1515                 if ( temp2-minibuf+1 >= MINIBUFSIZE ) break;
1516                 *temp2 = *temp;
1517             }
1518         }
1519         *temp2 = '\\0';
1520     } else {
1521         i = strlen( inilistptr->value );
1522         if ( i >= MINIBUFSIZE ) i = MINIBUFSIZE-1;
1523         memcpy( minibuf, inilistptr->value, i+1 );
1524     }
1525
1526     if ( INI_ISSPACE( *(inilistptr->value) ) || INI_ISSPACE( *( inilistptr->value+ (
1527         strlen(inilistptr->value)-1 ) ) ) ) {
1528         templ = delims[ strchr( inilistptr->value, '"' ) ? 2 : 1 ];
1529         if ( fprintf( inifile, "%s=%s%s\n", keyptr, templ, minibuf, templ ) < 0 ) isError = 1;
1530     } else {
1531         if ( fprintf( inifile, "%s=%s\n", keyptr, minibuf ) < 0 ) isError = 1;
1532     }
1533 }
1534 fflush(inifile);
1535 if ( inifile != stdout ) fclose(inifile);
1536 INI_FREE(outbuffer);

```

8.1.3.22 ini_destroy() void ini_destroy (
void)

Löscht alle Schlüssel, löscht alle Strukturen.

Rückgabe

Nichts.

Definiert in Zeile 1544 der Datei IniFile.c.

```

1547     {
1548     TIniEntry *inilistptr = inilist;
1549     TIniEntry *nextptr;
1550
1551     while (inilistptr) {
1552         nextptr = inilistptr->next;
1553         INIS_STRFREE( inilistptr->key );
1554         INIS_STRFREE( inilistptr->value );
1555         INIS_FREE( inilistptr, sizeof(*inilistptr) );
1556         inilistptr = nextptr;
1557     }
1558     inilist = NULL;
1559
1560     inilistptr = comments;
1561     while (inilistptr) {
1562         nextptr = inilistptr->next;
1563         INIS_STRFREE( inilistptr->value );
1564         INIS_FREE( inilistptr, sizeof(*inilistptr) );
1565         inilistptr = nextptr;
1566     }
1567     comments = NULL;
1568
1569     INIS_STRFREE(logname);
1570     INIS_STRFREE(errname);

```

```

1571  INIS_STRFREE(erroriname);
1572  INIS_STRFREE(fallbacksection);
1573  INIS_STRFREE(tmpstr);
1574
1575  entrycount = sectioncount = commentcount = 0;

```

8.1.3.23 ini_set_fallbacksection() `int ini_set_fallbacksection (const char * sectionname )`

Setzt den Namen des Fallback-Abschnitts für im angegebenen Abschnitt nicht gefundene Einträge.

Üblicherweise wird ein Schlüssel nur im angegebenen Abschnitt der Ini-Datei (was der unbenannte Abschnitt sein kann) gesucht. Mit dieser Funktion kann jedoch der Name eines Fallback-Abschnitts gesetzt werden, in dem gesucht wird, falls ein Eintrag im angegebenen Abschnitt nicht gefunden wird. Der Wert kann leer oder [] sein, dann wird der unbenannte Abschnitt für den Fallback genutzt. Wenn der Wert jedoch NULL ist, wird kein Fallback genutzt.

Der Fallback wird nur für die Suche verwendet, nicht beim Löschen usw.

Parameter

in	<i>sectionname</i>	Name des Fallback-Abschnitts. "[]" für den Kann auch NULL oder leer sein, um Fallback abzuschalten. Begrenzende eckige Klammern dürfen fehlen (außer für den leeren Abschnitt).
----	--------------------	---

Rückgabe

0, falls OK. INI_ERR_NOMEMORY, falls nicht genug Speicher.

Siehe auch

[ini_get_fallbacksection\(\)](#), [ini_get\(\)](#)

Definiert in Zeile 1600 der Datei IniFile.c.

```

1603                                     {
1604
1605  if ( !sectionname || !*sectionname ) {
1606      /* Bisheriges Fallback-Segment killen */
1607      INIS_STRFREE(fallbacksection);
1608  } else {
1609      INIS_STRFREE(fallbacksection);
1610      fallbacksection = make_section_name(sectionname);
1611      if ( !fallbacksection ) return INI_ERR_NOMEMORY;
1612  }

```

8.1.3.24 ini_get_fallbacksection() `char* ini_get_fallbacksection (void )`

Holt den Namen des Fallback-Abschnitts für im angegebenen Abschnitt nicht gefundene Einträge.

Rückgabe

Pointer auf einen temporären statischen String, der den Namen des aktuellen Fallback-Segments enthält. Kann NULL sein, wenn kein Fallback-Segment benannt ist oder wenn nicht genug Speicher vorhanden ist. Wenn der unbenannte Abschnitt der Fallback-Abschnitt ist, wird ein leerer String zurückgegeben, sonst der Name des Fallback-Abschnitts in eckigen Klammern. Der String sollte für weitere Verwendung kopiert werden. Der Speicherbereich sollte nicht beschrieben oder freigegeben werden.

Siehe auch

[ini_set_fallbacksection\(\)](#), [ini_get\(\)](#)

Definiert in Zeile 1629 der Datei IniFile.c.

```
1632 {
1633     if ( !fallbacksection ) return NULL;
1634     tmpstr = INI_STRREALLOC( tmpstr, strlen(fallbacksection) );
1635     if (tmpstr) INI_COPYSTR( tmpstr, fallbacksection, strlen(fallbacksection) );
```

8.1.3.25 ini_get() char* ini_get (

```
    char * value,
    const char * key,
    const char *const defaultvalue,
    const size_t maxlen )
```

Holt den Wert, der zu einem gegebenen Konfigurations-Schlüssel gehört.

Ein Wert kann Referenzen auf andere Schlüssel enthalten, indem der Schlüsselname in Prozentzeichen eingeschlossen wird. Diese Referenzen werden dynamisch (d. h. mit dem jeweils aktuell vorliegenden Satz von Konfigurationseinträgen) und rekursiv aufgelöst. Eine Heuristik verhindert unendliche Rekursion.

Parameter

out	<i>value</i>	String, der den Ergebniswert aufnimmt. Muss mindestens Platz für maxlen Zeichen (inklusive NUL) bieten. Falls NULL, wird Speicherplatz dynamisch alloziert. Ist der leere String, wenn der Schlüssel nicht existiert.
in	<i>key</i>	Der Schlüssel (darf auch leer oder NULL sein). Der Schlüssel darf am Anfang in eckigen Klammern einen Abschnittsnamen enthalten.
in	<i>defaultvalue</i>	Defaultwert, falls der Schlüssel nicht existiert. Kann NULL sein.
in	<i>maxlen</i>	Maximale Länge des Werts. Wenn 0, wird als Ergebnis stets der leere String geliefert.

Rückgabe

Ein Pointer auf den Ergebnisstring. NULL, falls nicht genug Speicher vorhanden.

Siehe auch

[ini_set_fallbacksection\(\)](#)

Definiert in Zeile 1665 der Datei IniFile.c.

```
1668 {
1669     TIniEntry *inilistptr = inilist;
1670     char *tempkey, *temp = NULL, *temp2, *buffer = NULL;
```

```

1672     const char *temp3;
1673     size_t len, len2, len3;
1674     int maxchanges;
1675
1676     if ( !value ) value = INI_STRALLOC( (maxlen > 0) ? (maxlen-1) : 0 );
1677     if ( !value ) return NULL;
1678     *value = '\0';
1679     *(value+1) = '\1'; /* Optimistischer Marker, dass wir einen Wert gefunden haben werden */
1680     if ( maxlen == 0 ) return value;
1681
1682     JUMP_EMPTY_SECTION(key);
1683     tempkey = INI_STRALLOC(INI_STRLEN(key)); /* NULL-Schlüssel wie leeren Schlüssel behandeln. */
1684     if (!tempkey) return NULL;
1685     INI_COPYSTRUPPER( tempkey, key );
1686     temp2 = tempkey;
1687
1688     while ( inilistptr && ( strcmp( temp2, inilistptr->key ) > 0 ) ) inilistptr = inilistptr->next;
1689
1690     if ( ( !inilistptr || !INI_STREQUAL( temp2, inilistptr->key ) ) && fallbacksection ) {
1691         /* Wert nicht gefunden, aber ein Fallback-Abschnitt ist gesetzt.
1692          * Also dort suchen, falls wir nicht eh bereits dort gesucht haben.
1693          */
1694         buffer = temp2 = NULL;
1695         if ( ( strlen(fallbacksection) == 0 ) && ( *tempkey == '[' ) ) {
1696             /* In den unbenannten Abschnitt gucken, denn das ist der Fallback-Abschnitt,
1697              * aber wir waren bis jetzt in einem benannten Abschnitt.
1698              */
1699             temp2 = strchr( tempkey, ']' );
1700             if (temp2) temp2++;
1701         } else if ( ( strlen(fallbacksection) != 0 ) && ( strstr( tempkey, fallbacksection ) != tempkey ) )
1702         {
1703             /* Wir haben bisher in einem anderen als dem Fallback-Abschnittt gesucht,
1704              * also suchen wir dort weiter. Zunächst ggf. den benannten Abschnitt ausblenden: */
1705             temp2 = strchr( tempkey, ']' );
1706             if (temp2) {
1707                 temp2++;
1708             } else {
1709                 temp2 = tempkey;
1710             }
1711             buffer = INI_STRALLOC( strlen(fallbacksection)+strlen(temp2) );
1712             if (buffer) {
1713                 (void)strcpy( buffer, fallbacksection );
1714                 (void)strcat( buffer, temp2 );
1715                 temp2 = buffer;
1716             }
1717             if (temp2) {
1718                 /* temp2 zeigt jetzt auf einen neuen Namen, mit dem wir es versuchen können. */
1719                 inilistptr = inilist;
1720                 while ( inilistptr && ( strcmp( temp2, inilistptr->key ) > 0 ) ) inilistptr = inilistptr->next;
1721             }
1722         }
1723
1724         if ( inilistptr && temp2 && INI_STREQUAL( temp2, inilistptr->key ) ) {
1725             /* gefunden; also Wert kopieren. */
1726             len = INI_STRLEN(inilistptr->value);
1727             if ( len > maxlen ) len = maxlen;
1728             INI_COPYSTR( value, inilistptr->value, len );
1729         } else {
1730             /* Wert (immer noch) nicht gefunden. Also Defaultwerte abarbeiten. */
1731             if (defaultvalue) {
1732                 /* Expliziter Defaultwert. */
1733                 temp3 = defaultvalue;
1734             } else if ( INI_STREQUAL( tempkey, "LOGFILE" ) ) {
1735                 /* Spezialfall: Standard-Protokolldatei */
1736                 temp3 = logname ? logname : "IniFile.log";
1737             } else if ( INI_STREQUAL( tempkey, "ERRFILE" ) ) {
1738                 /* Spezialfall: Standard-Fehlerprotokolldatei */
1739                 temp3 = errname ? errname : "IniFile.err";
1740             } else {
1741                 temp3 = "";
1742                 *(value+1) = '\0';
1743                 /* Undokumentiertes Feature: wenn wir keinen Wert gefunden haben,
1744                  * folgt dem Rückgabewert (leerer String) ein weiteres NUL-Byte.
1745                  */
1746             }
1747             len = strlen(temp3);
1748             if ( len > maxlen ) len = maxlen;
1749             INI_COPYSTR( value, temp3, len );
1750         }
1751         INI_FREE(buffer);
1752
1753         if ( recursion > 0 ) {
1754             /* Im Rekursionsfall: Einfach Wert zurückliefern; weitere Referenzen werden dort behandelt. */
1755             INI_FREE(tempkey);
1756             return value;
1757         }

```

```

1758
1759 /* Vorbereiten für Auflösen von Referenzen. Heuristik: maximal so viele
1760 * Referenzen auflösen, wie der ursprüngliche Eintrag an Referenzen besitzt,
1761 * multipliziert mit der aktuellen Anzahl vorliegender Einträge insgesamt.
1762 * Also: jeder Eintrag bekommt statistisch gesehen zwei Chancen,
1763 * expandiert zu werden.
1764 */
1765 recursion = 1;
1766 maxchanges = 0;
1767 temp = value;
1768 while (temp) {
1769     temp = strchr( temp, '%' );
1770     if (temp) {
1771         maxchanges++;
1772         temp++;
1773     }
1774 }
1775 maxchanges = (maxchanges/2) * ini_nkeys( );
1776 buffer = INI_STRALLOC(maxlen);
1777 if ( !buffer ) return NULL;
1778
1779 /* Den gesamten Wert nach Prozentzeichen durchgehen. */
1780 temp = value;
1781 while ( ( ( temp = strchr( temp, '%' ) ) != NULL ) && ( maxchanges > 0 ) ) {
1782     /* Referenz gefunden. Schlüssel extrahieren. */
1783     if ( ( temp2 = strchr( temp+1, '%' ) ) == NULL ) break; /* Nee, war doch keine. */
1784     len = (size_t)( temp2 - temp - 1 );
1785     tempkey = INI_STRREALLOC( tempkey, len );
1786     if ( !tempkey ) return NULL;
1787     INI_COPYSTR( tempkey, temp+1, len );
1788     /* Wert der Referenz rekursiv bestimmen. */
1789     ini_get( buffer, tempkey, NULL, maxlen );
1790     if ( *buffer || *(buffer+1) ) {
1791         /* Referenz durch ihren Wert ersetzen, auch dann wenn der gefundene Wert der leere String ist
1792          * (aber nicht, wenn der Wert nicht gefunden wurde -- dazu benutzen wir den undokumentierten
1793          * Marker direkt hinter dem Ende des leeren Strings).
1794          */
1795         len3 = strlen(buffer);
1796         len2 = strlen(temp2+1);
1797         if ( len3+len2 > maxlen ) len2 = maxlen - len3;
1798         strncat( buffer, temp2+1, len2 );
1799         *temp = '\0';
1800         len = strlen(temp);
1801         len2 = strlen(buffer);
1802         if ( len+len2 > maxlen ) len2 = maxlen - len;
1803         strncat( temp, buffer, len2 );
1804
1805         if ( ( ( temp2 = strchr( temp+len+len3, '%' ) ) != NULL ) &&
1806             ( strchr( temp2+1, '%' ) != NULL )
1807             ) {
1808             /* Hinter unserer Ersetzung gibt es weitere Ersetzungskandidaten.
1809              * Die arbeiten wir zuerst ab, ehe wir unseren Rekursionszähler anpassen.
1810              */
1811             temp += len + len3;
1812         } else {
1813             /* Keine weiteren Ersetzungskandidaten gefunden. Dann noch mal von vorne an prüfen,
1814              * ob wir rekursiv tätig werden müssen; dabei aber buchführen, damit wir nicht ewig laufen.
1815              */
1816             maxchanges--;
1817             temp = value;
1818         }
1819     } else {
1820         temp++;
1821     }
1822 }
1823
1824 INI_FREE(buffer);
1825 INI_FREE(tempkey);
1826 recursion = 0;

```

8.1.3.26 ini_get_error_filename() char* ini_get_error_filename (void)

Liefert ggf.

den Namen der Ini-Datei, in der ein Fehler aufgetreten ist.

Wenn ein Fehler beim Verarbeiten einer Ini-Datei aufgetreten ist, wird hier ihr Name zurückgeliefert. Wegen des Include-Mechanismus ist das nicht unbedingt die Datei, mit der `ini_read()` explizit aufgerufen wurde.

Rückgabe

Der Name der Konfigurationsdatei, in der ein Fehler aufgetreten ist. NULL, wenn kein Fehler aufgetreten ist. Ein leerer String, wenn nicht genug Speicher für den Namen verfügbar war. Der Speicher sollte durch das aufrufende Programm nicht verändert oder freigegeben werden.

Siehe auch

[ini_read\(\)](#), [ini_get_error_linenummer\(\)](#)

Definiert in Zeile 1843 der Datei IniFile.c.

```
1846 {  
1847     static char *dummy = "";
```

8.1.3.27 ini_get_error_linenummer() int __inline ini_get_error_linenummer (
void)

Liefert ggf.

die Nummer der Zeile der Ini-Datei, in der ein Fehler aufgetreten ist.

Wenn ein Fehler beim Verarbeiten einer Ini-Datei aufgetreten ist, wird hier die Nummer der Zeile geliefert. Wegen des Include-Mechanismus ist das nicht unbedingt in der Datei, mit der [ini_read\(\)](#) explizit aufgerufen wurde.

Rückgabe

Die Zeilennummer. -1, wenn kein Fehler aufgetreten ist.

Siehe auch

[ini_read\(\)](#), [ini_get_error_filename\(\)](#)

Definiert in Zeile 1861 der Datei IniFile.c.

```
1864 { return errorlinenummer; }
```

8.1.3.28 ini_get_version() unsigned short __inline ini_get_version (
void)

Liefert die aktuelle Version der Library als Binärwert.

Liefert die aktuelle Version der Library.

Rückgabe

Die Versionsnummer der Library. Unteres Byte: Minor Version; oberes Byte: Major Version.

Definiert in Zeile 1869 der Datei IniFile.c.

```
1869 : Minor Version; oberes Byte: Major Version.
```

8.1.3.29 ini_get_version_string() `const char __inline* ini_get_version_string (void )`

Liefert die aktuelle Version der Library als String.

Liefert die aktuelle Version der Library.

Rückgabe

Die Versionsnummer der Library als Pointer auf einen statischen String..

Definiert in Zeile 1877 der Datei IniFile.c.

```
1880 { return VERSIONSTRING; }
```

8.1.3.30 ini_get_license() `char* ini_get_license (void )`

Liefert den Lizenz- und Copyright-Text.

Rückgabe

Pointer auf einen temporären statischen String, der den Text der Lizenz und des Copyrights enthält. Kann NULL sein, wenn nicht genug Speicher vorhanden ist. Der String sollte für weitere Verwendung kopiert werden. Der Speicherbereich sollte nicht beschrieben oder freigegeben werden.

Definiert in Zeile 1889 der Datei IniFile.c.

```
1892 {  
1893     tmpstr = INI_STRREALLOC( tmpstr, strlen(COPYRIGHT) );  
1894     if (tmpstr) INI_COPYSTR( tmpstr, COPYRIGHT, strlen(COPYRIGHT) );  
1895     got_license = 1;  
}
```

8.1.3.31 ini_get_dir_delimiter() `char ini_get_dir_delimiter (const unsigned char option )`

Gibt das Verzeichnistrennzeichen im Unix- oder DOS-Stil zurück.

Der Verzeichnistrenner wird nach Wunsch geliefert: fix im Unix- oder im DOS-/Windows-Stil, oder variabel (DOS-/Windows-Stil, falls ein Konfigurationseintrag mit dem Schlüssel DOS existiert und einen Wert ungleich 0 hat), oder variabel je nach aktuellem Betriebssystem.

Parameter

in	option	Falls 0: Verzeichnistrenner im Unix-Stil. Falls 2: Verzeichnistrenner im DOS-/Windows-Stil. Falls 1: variable Ermittlung (siehe oben).
----	--------	--

Rückgabe

Das Verzeichnistrennzeichen.

Definiert in Zeile 1912 der Datei IniFile.c.

```

1915                                     {
1916     char dosbuff[2];
1917     char delim = '/';
1918     #ifdef _WIN32
1919     #define DEFAULTTOPTION ("1")
1920     #else
1921     #define DEFAULTTOPTION ("0")
1922     #endif
1923
1924     /* Bestimmen, welches Verzeichnistrennerformat wir haben wollen. */
1925     if ( option == 1 ) {
1926         /* Variabel: Konfigurationswert mit dem Schlüssel DOS ansehen. */
1927         ini_get( dosbuff, "DOS", DEFAULTTOPTION, 1 );
1928         if ( ( *dosbuff != '\0' ) && ( !INI_ISSPACE(*dosbuff) ) && ( *dosbuff != '0' ) ) delim = '\\';
1929     } else if ( option == 2 ) {
1930         /* DOS-/Windows-Stil. */
1931         delim = '\\';
1932     }
1933

```

8.1.3.32 ini_catfile() char* ini_catfile (
char * components,
...)

Baut einen Dateipfad aus einzelnen Komponenten zusammen.

Hilfsfunktion für das Zusammenbauen von Dateipfaden. Die Eingabe besteht aus einer variablen Anzahl von Pfadkomponenten: Laufwerksangabe (optional als erste Komponente), Verzeichnisse und/oder Teilpfade, optional ein Dateiname am Ende. Die Funktion fügt die Komponenten mit für die aktuell vorliegende Plattform geeigneten Pfadtrennzeichen zusammen und entfernt dabei überflüssige Trennzeichen.

Parameter

in	<i>components</i>	Eine variable Anzahl von Pfadkomponenten. Die letzte Komponente muss (char *)NULL sein. (Typecast beachten!)
----	-------------------	--

Rückgabe

Pointer auf einen temporären Speicherbereich mit dem zusammengesetzten Pfad. NULL, wenn ein Fehler aufgetreten ist (z.B. Speichermangel). Der String sollte für weitere Verwendung kopiert werden. Der Speicherbereich sollte nicht beschrieben oder freigegeben werden.

Definiert in Zeile 1957 der Datei IniFile.c.

```

1960                                     {
1961     va_list argptr;
1962     char dirdelim, *comp;
1963     size_t n, k, i;
1964
1965     dirdelim = ini_get_dir_delimiter(1);
1966     tmpstr = INI_STRREALLOC( tmpstr, 0 );
1967     if ( !tmpstr ) return NULL;
1968     va_start( argptr, components );
1969     comp = components;
1970     while (comp) {
1971         n = k = strlen(tmpstr);
1972         i = strlen(comp);
1973         if ( n && ( *( tmpstr+n-1 ) != ':' ) && ( *( tmpstr+n-1 ) != dirdelim ) ) k++;
1974         /* tmpstr enthält schon etwas, an das ein Delimiter angehängt werden muss! */
1975         tmpstr = INI_STRREALLOC( tmpstr, k+i );
1976         if ( n != k ) *(tmpstr+n) = dirdelim;
1977         memmove( tmpstr+k, comp, i+1 );
1978         comp = va_arg( argptr, char * );
1979     }
1980     va_end(argptr);
1981

```


8.1.3.33 ini_fix_dir_delim() `char* ini_fix_dir_delim (`
`char * path,`
`const unsigned char option )`

Setzt alle Vorkommen von Verzeichnistrennern auf Unix- oder DOS-Stil um.

Hilfsfunktion für das Zusammenbauen von Dateipfaden. Entfernt doppelte Verzeichnistrenner (außer am Anfang); setzt Verzeichnistrenner fix auf Unix- oder auf DOS-/Windows-Stil um, oder variabel (DOS-/Windows-Stil, falls ein Konfigurationseintrag mit dem Schlüssel DOS existiert und einen Wert ungleich 0 hat), oder variabel je nach aktuellem Betriebssystem. Diese Aufgabe wird für die typischen Alltagsverwendungen von [ini_catfile\(\)](#) einfacher gelöst.

Parameter

<code>in, out</code>	<code>path</code>	Pfadangabe. Wird an Ort und Stelle modifiziert.
<code>in</code>	<code>option</code>	Falls 0: Umsetzung auf Unix-Stil. Falls 2: Umsetzung auf DOS-/Windows-Stil. Falls 1: variable Umsetzung (siehe oben).

Rückgabe

Pointer auf den modifizierten Pfad.

Siehe auch

[ini_catfile\(\)](#)

Definiert in Zeile 2004 der Datei IniFile.c.

```

2007
2008
2009 char *temp;
2010 char src, dst;
2011
2012 if ( !path ) return path;
2013
2014 dst = ini_get_dir_delimiter(option);
2015 src = ( dst == '/' ) ? '\\\' : '/';
2016
2017 while ( ( temp=strchr( path, src ) ) != NULL ) *temp = dst; /* Umsetzen. */
2018
2019 /* Jetzt nach doppelten Vorkommen von dst suchen. Am Anfang OK, sonst nicht. */
2020 temp = path;
2021 while ( *temp && ( *temp == dst ) ) temp++; /* Verzeichnistrenner am Anfang überspringen */
2022 src = 0;
2023 while (*temp) {
2024     if ( *temp == dst ) {
2025         if (src) {
2026             /* Mehrfaches Auftreten, also ab aktueller Stelle aufrücken lassen. */
2027             memmove( temp, temp+1, strlen(temp) );
2028         } else {
2029             src = 1;
2030             temp++;
2031         }
2032     } else {
2033         src = 0;
2034         temp++;
2035     }
2036 }
```

8.1.3.34 ini_add_dir_delim() `char* ini_add_dir_delim (`
`char * dirpath,`
`const unsigned char option )`

Hängt einen Verzeichnistrenner an, falls nicht schon vorhanden.

Parameter

<code>in, out</code>	<code>dirpath</code>	Pfadangabe. Wird an Ort und Stelle modifiziert. Muss genug Platz für ein weiteres Zeichen bieten.
<code>in</code>	<code>option</code>	Falls 0: Entfernen des Trenners am Ende (falls vorhanden). Falls 1: Anhängen eines Verzeichnistrenners, falls nicht bereits ein Trenner am Ende vorhanden ist.

Rückgabe

Pointer auf den modifizierten Pfad.

Siehe auch

[ini_catfile\(\)](#)

Veraltet Diese Aufgabe wird von [ini_catfile\(\)](#) besser gemacht.

Definiert in Zeile 2055 der Datei IniFile.c.

```
2058                                     {
2059     char *temp;
2060     if ( !INI_STRLen( dirpath ) ) return dirpath;
2061
2062     temp = dirpath + strlen( dirpath ) - 1;
2063
2064     if ( option ) {
2065         if ( ( *temp != '/' ) && ( *temp != '\\' ) ) {
2066             (void)strcat( dirpath, ( strchr( dirpath, '/' ) || !strchr( dirpath, '\\' ) ) ? "/" : "\\ " );
2067         }
2068     } else {
2069         if ( ( *temp == '/' ) || ( *temp == '\\' ) ) *temp = '\0';
2070     }
```

8.1.4 Variablen-Dokumentation

8.1.4.1 inilist `TIniEntry* inilist = NULL [static]`

Anfang der verketteten Liste von Ini-Einträgen.

Definiert in Zeile 280 der Datei IniFile.c.

8.1.4.2 comments `TIniEntry* comments = NULL [static]`

Anfang der verketteten Liste von Ini-Kommentar-Einträgen.

Definiert in Zeile 281 der Datei IniFile.c.

8.1.4.3 ininames `TIniEntry* ininames = NULL [static]`

Sammlung aktiver Include-Namen, um Rekursion zu vermeiden

Definiert in Zeile 282 der Datei IniFile.c.

8.1.4.4 logname `char* logname = NULL [static]`

Sonderfall: Name der Standard-Protokolldatei.

Definiert in Zeile 283 der Datei IniFile.c.

8.1.4.5 errname `char* errname = NULL [static]`

Sonderfall: Name der Standard-Fehlerprotokolldatei.

Definiert in Zeile 284 der Datei IniFile.c.

8.1.4.6 fallbacksection `char* fallbacksection = NULL [static]`

Fallback-Abschnitt für im angegebenen Abschnitt nicht gefundene Einträge.

Definiert in Zeile 285 der Datei IniFile.c.

8.1.4.7 tmpstr `char* tmpstr = NULL [static]`

Platz für temporär erzeugte Strings für Rückgabe aus Funktionen.

Definiert in Zeile 286 der Datei IniFile.c.

8.1.4.8 recursion `int recursion = -1 [static]`

Merker für Rekursion bei der Platzhalter-Ersetzung.

Definiert in Zeile 287 der Datei IniFile.c.

8.1.4.9 entrycount `int entrycount = 0 [static]`

Anzahl der Konfigurationseinträge (stets aktuell).

Definiert in Zeile 288 der Datei IniFile.c.

8.1.4.10 sectioncount `int sectioncount = 0 [static]`

Anzahl der Abschnitte (gecached).

Definiert in Zeile 289 der Datei IniFile.c.

8.1.4.11 commentcount `int commentcount = 0 [static]`

Anzahl der Kommentarzeilen (stets aktuell).

Definiert in Zeile 290 der Datei IniFile.c.

8.1.4.12 errorlinenumber `int errorlinenumber = -1 [static]`

Nummer der Konfigurationsdateizeile, bei der ein Fehler auftrat.

Definiert in Zeile 291 der Datei IniFile.c.

8.1.4.13 errorininame `char* errorininame = NULL [static]`

Name der Konfigurationsdatei, in der ein Fehler auftrat (oder NULL).

Definiert in Zeile 292 der Datei IniFile.c.

8.1.4.14 got_license `int got_license = 0 [static]`

Flag, ob Lizenz-/Copyright-Text schon abgefragt wurden.

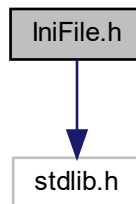
Definiert in Zeile 293 der Datei IniFile.c.

8.2 IniFile.h-Dateireferenz

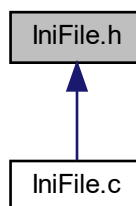
Header-Datei für den Konfigurationsdateihandler IniFile.

```
#include <stdlib.h>
```

Include-Abhängigkeitsdiagramm für IniFile.h:



Dieser Graph zeigt, welche Datei direkt oder indirekt diese Datei enthält:



Datenstrukturen

- struct [IniEntry](#)

Struktur zum Verwalten von Ini-Einträgen (Schlüssel-Wert-Paare) in einer verketteten Liste.

Makrodefinitionen

- `#define INI_CHAR_Toupper(c) ( (char)toupper( (uchar)(c) ) )`
Setzt ein einzelnes Zeichen auf Großschreibung um (char, nicht uchar).
- `#define INI_COPYSTR(dst, src, len)`
Sicheres Kopieren von Strings (so gut das in C eben geht).
- `#define INI_COPYSTRUPPER(dst, src)`
Hilfsfunktion zum Kopieren von Strings bei gleichzeitiger Umsetzung auf Großbuchstaben.
- `#define INI_ERR_NOFILE ( -2 )`
Rückgabecode, falls die angegebene Ini-Datei nicht gelesen bzw.
- `#define INI_ERR_NOMEMORY ( -3 )`
Rückgabecode, falls nicht genügend Speicherplatz vorhanden ist.
- `#define INI_ERR_NONAME ( -1 )`
Rückgabecode, falls keine Ini-Datei angegeben wurde.
- `#define INI_ERR_NOSUCHPRAGMA ( -4 )`
Rückgabecode, falls ein unbekanntes Pragma angegeben wurde.
- `#define INI_FREE(memp) { if (memp) free(memp); (memp) = NULL; }`
Sichere (safe) Speicherfreigabe.
- `#define INI_ISDIGIT(c) ( isdigit( (uchar)(c) ) )`
Testet auf Ziffer (char oder uchar).
- `#define INI_ISSPACE(c) ( isspace( (uchar)(c) ) )`
Testet auf Whitespace (char oder uchar).
- `#define INI_ISXDIGIT(c) ( isxdigit( (uchar)(c) ) )`
Testet auf Hexadezimalziffer (char oder uchar).
- `#define INI_MALLOC(size) ( ini_safemalloc( (size_t)(size) ) )`
Sichere (safe) Speicherallokation.
- `#define INI_REALLOC(memp, size) ( ini_saferealloc( (memp), (size_t)(size) ) )`
Sichere (safe) Speicher-Reallokation.
- `#define INI_STRALLOC(len) ( (char *)INI_MALLOC( ( (size_t)(len)+1 )*sizeof(char) ) )`
Sichere (safe) Speicherallokation für Strings (hängt Platz für NUL automatisch mit an).
- `#define INI_STREQUAL(str1, str2) ( (str1) ? ( (str2) ? ( strcmp((str1), (str2)) == 0 ) : 0 ) : ( (str2) ? 0 : 1 ) )`
Test zweier Strings auf Gleichheit (etwas sichererer und bequemerer Wrapper um strcmp()).
- `#define INI_STRLEN(src) ( (src) ? strlen(src) : 0 )`
Sichere Bestimmung von String-Längen (so gut das in C eben geht).
- `#define INI_STRREALLOC(memp, len) ( (char *)INI_REALLOC( (memp), ( (size_t)(len)+1 )*sizeof(char) ) )`
Sichere (safe) Speicher-Reallokation für Strings.
- `#define INI_STRUPPER(str)`
Hilfsfunktion zum Umsetzen eines Strings auf Großbuchstaben.
- `#define INIS0_FREE(memp, size) { if (memp) { inis_wipe_memory( (memp), (size) ); free(memp); (memp) = NULL; } }`
Sichere (safe & secure) Speicherfreigabe.
- `#define INIS_FREE(memp, size) { INIS0_FREE( (memp), (size) ); }`
Sichere (safe & secure) Speicherfreigabe (size wird nur genutzt auf Systemen ohne msize()).
- `#define INIS_REALLOC(memp, newsize, oldsize) ( inis_saferealloc( (memp), (size_t)(newsize), (size_t)(oldsize) ) )`
Sichere (safe & secure) Speicher-Reallokation.
- `#define INIS_STRFREE(memp) { INIS0_FREE( (memp), INI_STRLEN(memp) ); }`
Sichere (safe & secure) Speicherfreigaben für Strings.

Typdefinitionen

- typedef struct [IniEntry](#) [TIniEntry](#)

Struktur zum Verwalten von Ini-Einträgen (Schlüssel-Wert-Paare) in einer verketteten Liste.

Funktionen

- int [ini_add](#) (const char *key, const char *const value)
Fügt einen Konfigurationseintrag (Schlüssel/Wert) hinzu.
- int [ini_add_comment](#) (const char *text)
Fügt eine Kommentarzeile hinzu.
- char * [ini_add_dir_delim](#) (char *dirpath, const unsigned char option)
Hängt einen Verzeichnistrenner an, falls nicht schon vorhanden.
- char ** [ini_comments](#) (char **commentlist, int maxcomment)
Liefert eine Liste der Kommentare.
- int [ini_delete](#) (const char *key)
Löscht einen Schlüssel, falls es ihn gibt.
- void [ini_destroy](#) (void)
Löscht alle Schlüssel, löscht alle Strukturen.
- char * [ini_fix_dir_delim](#) (char *path, const unsigned char option)
Setzt alle Vorkommen von Verzeichnistrennern auf Unix- oder DOS-Stil um.
- char * [ini_get](#) (char *value, const char *key, const char *const defaultvalue, const size_t maxlen)
Holt den Wert, der zu einem gegebenen Konfigurations-Schlüssel gehört.
- char [ini_get_dir_delimiter](#) (const unsigned char option)
Gibt das Verzeichnistrennzeichen im Unix- oder DOS-Stil zurück.
- char * [ini_get_error_filename](#) (void)
Liefert ggf.
- int [ini_get_error_linenumber](#) (void)
Liefert ggf.
- char * [ini_get_fallbacksection](#) (void)
Holt den Namen des Fallback-Abschnitts für im angegebenen Abschnitt nicht gefundene Einträge.
- char * [ini_get_license](#) (void)
Liefert den Lizenz- und Copyright-Text.
- unsigned short [ini_get_version](#) (void)
Liefert die aktuelle Version der Library.
- const char * [ini_get_version_string](#) (void)
Liefert die aktuelle Version der Library.
- char ** [ini_keys](#) (char **keylist, int maxkeys)
Liefert eine Liste der Schlüssel.
- char ** [ini_keys_section](#) (char **keylist, int maxkeys, const char *sectionname)
Liefert eine Liste der Schlüssel in einem gegebenen Abschnitt.
- int [ini_ncomments](#) (void)
Liefert die Anzahl der Kommentare zurück.
- int [ini_nkeys](#) (void)
Liefert die Anzahl der Konfigurationseinträge zurück.
- int [ini_nkeys_section](#) (const char *sectionname)
Liefert die Anzahl der Konfigurationseinträge in einem gegebenen Abschnitt zurück.
- int [ini_nsections](#) (void)
Liefert die Anzahl der Konfigurationsabschnitte zurück.
- int [ini_read](#) (const char *const inifilename)

- Einlesen einer Konfigurationsdatei.*
 - void * [ini_safemalloc](#) (const size_t size)
Sichere (safe) Allokation von Speicherplatz.
 - void * [ini_saferealloc](#) (void *memp, const size_t size)
Sichere (safe) Re-Allokation von Speicherplatz.
 - char ** [ini_sections](#) (char **sectionlist, int maxsections)
Liefert eine Liste der Konfigurationsabschnitte zurück.
 - int [ini_set](#) (const char *key, const char *const value)
Synonym für [ini_add\(\)](#):
 - int [ini_set_fallbacksection](#) (const char *sectionname)
Setzt den Namen des Fallback-Abschnitts für im angegebenen Abschnitt nicht gefundene Einträge.
 - int [ini_write](#) (const char *const inifilename, const char *comment)
Schreibt die aktuellen Ini-Werte in eine Datei.
 - void __inline [inis_wipe_memory](#) (void *memp, const size_t size)
Sicheres (secure) Löschen von Speicherplatz.

8.2.1 Ausführliche Beschreibung

Header-Datei für den Konfigurationsdateihandler IniFile.

Autor

Gisbert W. Selke

Datum

2021-07-20

Version

1.20

Copyright © TapirSoft Gisbert und Harald Selke GbR

Kurzbeschreibung

Siehe auch

[IniFile.c](#)

\$Id: [IniFile.h](#) 426 2007-10-24 22:32:18Z gisbert \$

Änderungen:\n

Datum	Kürzel	Version	Beschreibung
2021-07-20	gws	1.20	Segmentnamen aus lauter Leerzeichen sind jetzt äquivalent zum unbenannten Segment. Ersetzung von Platzhaltern mit leerem Wert repariert. Funktionen ini_comments() , ini_ncomments() und ini_add_comment() hinzugefügt. Angabe nichtexistenter/nichtlesbarer/fehlender Ini-Dateien ist kein Fehler mehr. Dokumentation ergänzt. Testfälle systematisiert.
2021-03-12	gws	1.12	Fehlerhafte Behandlung bei fehlender Schlüsselangabe korrigiert. Probleme mit isspace() , toupper() usw. bei 8-Bit-Zeichen umgangen.
2021-02-01	gws	1.11	Fehlercode korrigiert, wenn Ini-Datei nicht gefunden Secure-Funktionen/-Makros eingeführt.
2007-10-25	gws	1.1	Pragma resolve_escapes eingeführt.
2007-06-25	gws	1.01	INI_COPYSTR verfeinert, INI_STRLEN eingeführt.
2007-04-02	gws	1.0	Dokumentation angepasst, Parametertypen genauer gefasst, ini_get_version() hinzugefügt.
2007-03-11	gws	1.0rc2	Unendliche Rekursion bei Include verhindert.
2007-02-05	gws	1.0rc1	Include-Mechanismus sowie 2 Pragmata implementiert.
2007-02-02	gws	0.95	Fallback-Abschnitt und ini_catfile eingeführt.
2006-04-25	gws	0.942	Deprecation Warnings von Visual Studio 2005 abgewürgt.
2005-09-27	gws	0.941	Prototyp für ini_destroy() korrigiert.
2005-05-08	gws	0.94	Minimale Erweiterungen.
2005-05-01	gws	0.2	Verarbeitung von Abschnitten voll integriert.
2004-11-04	gws	0.12	Minimale Funktionserweiterung.
1998-xx-xx	gws	0.1x	Erste Produktionsversion.

8.2.2 Makro-Dokumentation

8.2.2.1 INI_ERR_NONAME `#define INI_ERR_NONAME ( -1 )`

Rückgabecode, falls keine Ini-Datei angegeben wurde.

Definiert in Zeile 65 der Datei IniFile.h.

8.2.2.2 INI_ERR_NOFILE `#define INI_ERR_NOFILE ( -2 )`

Rückgabecode, falls die angegebene Ini-Datei nicht gelesen bzw.

geschrieben werden konnte.

Definiert in Zeile 68 der Datei IniFile.h.

8.2.2.3 INI_ERR_NOMEMORY `#define INI_ERR_NOMEMORY ( -3 )`

Rückgabecode, falls nicht genügend Speicherplatz vorhanden ist.

Definiert in Zeile 71 der Datei IniFile.h.

8.2.2.4 INI_ERR_NOSUCHPRAGMA `#define INI_ERR_NOSUCHPRAGMA ( -4 )`

Rückgabecode, falls ein unbekanntes Pragma angegeben wurde.

Definiert in Zeile 74 der Datei IniFile.h.

8.2.2.5 INI_MALLOC `#define INI_MALLOC(
size) (ini_safemalloc ((size_t)(size)))`

Sichere (safe) Speicherallokation.

Parameter

<i>in</i>	<i>size</i>	Angeforderte Speichergröße in Bytes.
-----------	-------------	--------------------------------------

Rückgabe

Pointer auf den Speicherplatz. NULL, falls nicht genügend Speicher vorhanden.

Definiert in Zeile 88 der Datei IniFile.h.

8.2.2.6 INI_STRALLOC `#define INI_STRALLOC(
len) ((char *)INI_MALLOC (((size_t)(len)+1) * sizeof(char)))`

Sichere (safe) Speicherallokation für Strings (hängt Platz für NUL automatisch mit an).

Parameter

<i>in</i>	<i>len</i>	Angeforderte maximale Stringlänge in Bytes (ohne Zählung des abschließenden NUL).
-----------	------------	---

Rückgabe

Pointer auf den Speicherplatz. NULL, falls nicht genügend Speicher vorhanden.

Definiert in Zeile 94 der Datei IniFile.h.

8.2.2.7 INI_REALLOC `#define INI_REALLOC(
 memp,
 size) (ini_saferealloc((memp), (size_t)(size)))`

Sichere (safe) Speicher-Reallokation.

Parameter

in	<i>memp</i>	Pointer auf den bisherigen Speicherplatz.
in	<i>size</i>	Angeforderte Speichergröße in Bytes.

Rückgabe

Pointer auf den Speicherplatz. NULL, falls nicht genügend Speicher vorhanden.

Definiert in Zeile 101 der Datei IniFile.h.

8.2.2.8 INIS_REALLOC `#define INIS_REALLOC(
 memp,
 newsize,
 oldsize) (inis_saferealloc((memp), (size_t)(newsize), (size_t)(oldsize)))`

Sichere (safe & secure) Speicher-Reallokation.

Parameter

in	<i>memp</i>	Pointer auf den bisherigen Speicherplatz.
in	<i>newsize</i>	Angeforderte Speichergröße in Bytes.
in	<i>oldsize</i>	Bisherige Speichergröße in Bytes.

Rückgabe

Pointer auf den Speicherplatz. NULL, falls nicht genügend Speicher vorhanden.

Definiert in Zeile 109 der Datei IniFile.h.

8.2.2.9 INI_STRREALLOC `#define INI_STRREALLOC(
 memp,
 len) ((char *)INI_REALLOC((memp), ((size_t)(len)+1)*sizeof(char)))`

Sichere (safe) Speicher-Reallokation für Strings.

Parameter

in	<i>memp</i>	Pointer auf den bisherigen Speicherplatz.
in	<i>len</i>	Angeforderte maximale Stringlänge in Bytes (ohne Zählung des abschließenden NUL).

Rückgabe

Pointer auf den Speicherplatz. NULL, falls nicht genügend Speicher vorhanden.

Definiert in Zeile 125 der Datei IniFile.h.

8.2.2.10 INI_FREE `#define INI_FREE(
 memp) { if (memp) free(memp); (memp) = NULL; }`

Sichere (safe) Speicherfreigabe.

Parameter

<i>in, out</i>	<i>memp</i>	Pointer auf den bisherigen Speicherplatz. Wird auf NULL gesetzt.
----------------	-------------	--

Definiert in Zeile 139 der Datei IniFile.h.

8.2.2.11 INISO_FREE `#define INISO_FREE(
 memp,
 size) { if (memp) { inis_wipe_memory((memp), (size)); free(memp); (memp) =
NULL; } }`

Sichere (safe & secure) Speicherfreigabe.

Parameter

<i>in, out</i>	<i>memp</i>	Pointer auf den bisherigen Speicherplatz. Wird auf NULL gesetzt.
<i>in</i>	<i>size</i>	Bisherige Speichergröße in Bytes; wird exakt so benutzt.

Definiert in Zeile 145 der Datei IniFile.h.

8.2.2.12 INIS_FREE `#define INIS_FREE(
 memp,
 size) { INISO_FREE((memp), (size)); }`

Sichere (safe & secure) Speicherfreigabe (*size* wird nur genutzt auf Systemen ohne *msize*()).

Parameter

<i>in, out</i>	<i>memp</i>	Pointer auf den bisherigen Speicherplatz. Wird auf NULL gesetzt.
<i>in</i>	<i>size</i>	Bisherige Speichergröße in Bytes (wird nur genutzt auf Systemen ohne <i>msize</i> ()).

Definiert in Zeile 154 der Datei IniFile.h.

8.2.2.13 INIS_STRFREE `#define INIS_STRFREE(
 memp) { INISO_FREE((memp), INI_STRLEN(memp)); }`

Sichere (safe & secure) Speicherfreigaben für Strings.

Wirklich sicher nur auf Systemen, die `msize()` kennen; sonst wird als Best Guess die String-Länge genommen, was zu kurz sein kann.

Parameter

<i>in, out</i>	<i>memp</i>	Pointer auf den bisherigen Speicherplatz. Wird auf NULL gesetzt.
----------------	-------------	--

Definiert in Zeile 172 der Datei `IniFile.h`.

8.2.2.14 INI_COPYSTR `#define INI_COPYSTR(
 dst,
 src,
 len)`

Wert:

```

{
if (dst) {
    size_t INI_len = (size_t)(len);
    if (src) {
        memcpy( (dst), (src), INI_len );
        *( (dst)+INI_len ) = '\0';
    } else {
        memset( (dst), '\0', INI_len+1 );
    }
}
}

```

Sicheres Kopieren von Strings (so gut das in C eben geht).

Parameter

<i>out</i>	<i>dst</i>	Pointer auf den Ziel-Speicherplatz.
<i>in</i>	<i>src</i>	Pointer auf den Quell-Speicherplatz.
<i>in</i>	<i>len</i>	Anzahl zu kopierender Bytes. Es werden genau so viele Bytes wie angegeben kopiert, selbst wenn der Quellstring kürzer ist. Im Ziel-Speicherplatz muss auf jeden Fall Platz für ein zusätzliches Zeichen sein, das auf jeden Fall mit NUL gefüllt wird. Die Funktion entspricht ungefähr einer etwas sichereren Variante von <code>strncpy()</code> , kopiert jedoch unter Umständen mehr!

Definiert in Zeile 185 der Datei `IniFile.h`.

8.2.2.15 INI_STRLEN `#define INI_STRLEN(
 src) ((src) ? strlen(src) : 0)`

Sichere Bestimmung von String-Längen (so gut das in C eben geht).

Parameter

in	src	Pointer auf den Quell-Speicherplatz.
----	-----	--------------------------------------

Definiert in Zeile 200 der Datei IniFile.h.

8.2.2.16 INI_STREQUAL `#define INI_STREQUAL(
 str1,
 str2) ((str1) ? ((str2) ? (strcmp((str1), (str2)) == 0) : 0) : ((str2) ?
0 : 1))`

Test zweier Strings auf Gleichheit (etwas sichererer und bequemerer Wrapper um strcmp()).

Parameter

in	str1	Pointer auf den ersten der beiden Strings. Kann NULL sein.
in	str2	Pointer auf den ersten der beiden Strings. Kann NULL sein.

Rückgabe

TRUE, wenn gleich; sonst FALSE.

Definiert in Zeile 207 der Datei IniFile.h.

8.2.2.17 INI_CHAR_Toupper `#define INI_CHAR_Toupper(
 c) ((char)toupper((uchar)(c)))`

Setzt ein einzelnes Zeichen auf Großschreibung um (char, nicht uchar).

Parameter

in	c	Das Zeichen (char; darf also auch < 0 sein).
----	---	--

Rückgabe

Wie toupper().

Definiert in Zeile 213 der Datei IniFile.h.

8.2.2.18 INI_ISSPACE `#define INI_ISSPACE(
 c) (isspace((uchar)(c)))`

Testet auf Whitespace (char oder uchar).

Parameter

in	c	Das Zeichen (char oder uchar; darf also auch < 0 sein).
----	---	---

Rückgabe

Wie isspace().

Definiert in Zeile 219 der Datei IniFile.h.

```
8.2.2.19 INI_ISDIGIT #define INI_ISDIGIT(  
    c ) ( isdigit( (uchar)(c) ) )
```

Testet auf Ziffer (char oder uchar).

Parameter

in	c	Das Zeichen (char oder uchar; darf also auch < 0 sein).
----	---	---

Rückgabe

Wie isdigit().

Definiert in Zeile 225 der Datei IniFile.h.

```
8.2.2.20 INI_ISXDIGIT #define INI_ISXDIGIT(  
    c ) ( isxdigit( (uchar)(c) ) )
```

Testet auf Hexadezimalziffer (char oder uchar).

Parameter

in	c	Das Zeichen (char oder uchar; darf also auch < 0 sein).
----	---	---

Rückgabe

Wie isxdigit().

Definiert in Zeile 231 der Datei IniFile.h.

8.2.2.21 INI_COPYSTRUPPER `#define INI_COPYSTRUPPER(
 dst,
 src)`

Wert:

```

{ char *SUC_temp1 = (char *) (dst), *SUC_temp2 = (char *) (src);
  if (SUC_temp2) {
    while ( *SUC_temp2 ) *(SUC_temp1++) = INI_CHAR_TOUPPER (
*(SUC_temp2++) ); \
  }
  *SUC_temp1 = '\0';
}
```

Hilfsfunktion zum Kopieren von Strings bei gleichzeitiger Umsetzung auf Großbuchstaben.

Parameter

out	<i>dst</i>	Pointer auf den Ziel-Speicherplatz. Muss mindestens so viel Platz wie <i>src</i> bieten.
in	<i>src</i>	Pointer auf den Quell-Speicherplatz.

Definiert in Zeile 237 der Datei IniFile.h.

8.2.2.22 INI_STRUPPER `#define INI_STRUPPER(
 str)`

Wert:

```

{ char *SU_temp;
  if (str) {
    for ( SU_temp=(str); *SU_temp; SU_temp++ ) *SU_temp = INI_CHAR_TOUPPER (
*SU_temp ); \
  }
}
```

Hilfsfunktion zum Umsetzen eines Strings auf Großbuchstaben.

Parameter

in, out	<i>str</i>	Pointer auf den Quell=Ziel-Speicherplatz.
---------	------------	---

Definiert in Zeile 247 der Datei IniFile.h.

8.2.3 Dokumentation der benutzerdefinierten Typen

8.2.3.1 TIniEntry `typedef struct IniEntry TIniEntry`

Struktur zum Verwalten von Ini-Einträgen (Schlüssel-Wert-Paare) in einer verketteten Liste.

8.2.4 Dokumentation der Funktionen

8.2.4.1 ini_read() int ini_read (
 const char *const inifilename)

Einlesen einer Konfigurationsdatei.

Die angegebene Konfigurationsdatei wird gelesen, ihre Einträge werden intern in einer sortierten verketteten Liste gespeichert. Konfigrierende Einträge aus ggf. vorher eingelesenen Konfigurationsdateien werden überschrieben. Wenn die angegebene Datei nicht zum Lesen geöffnet werden kann, ist dies kein Fehler.

Parameter

in	<i>inifilename</i>	Name der einzulesenden Konfigurationsdatei.
----	--------------------	---

Rückgabe

Anzahl der nach dem Einlesen verfügbaren Einträge. INI_ERR_NOMEMORY, falls nicht genügend Speicherplatz verfügbar. INI_ERR_NOSUCHPRAGMA, falls die Konfigurationsdatei ein unbekanntes Pragma enthält.

Definiert in Zeile 641 der Datei IniFile.c.

```

644                                     {
645
646     FILE *inifile = NULL;
647     TIniEntry *ininameentry = NULL;
648     char minibuf[MINIBUFSIZE];
649     char *sectionname = NULL;
650     char *inbuffer = NULL;
651     char *temp1, *temp2, *temp3;
652     size_t sectionnamemaxlen, i;
653     int npragma, pragmaaval, n, lineno = 0, resolve_escapes = 0;
654
655     errorlinenumber = -1;
656
657     if ( !INI_STRLen(inifilename) ) return ini_nkeys( );
658
659     /* Wir wollen keine unendlichen Include-Schleifen und müssen uns daher merken, wo wir schon waren.
660      * Mehrere Verarbeitungen derselben Ini-Datei sind zulässig, aber rekursive Aufrufe nicht.
661      * Wir speichern die gerade aktiven Namen in der Linked List ininames, wobei der neueste am Anfang
662      * steht.
663      */
664     ininameentry = ininames;
665     while ( ininameentry ) {
666         /* Checken, ob unser aktueller Dateiname uns bekannt vorkommt. */
667         if ( INI_STREQUAL( ininameentry->value, inifilename ) ) return ini_nkeys( );
668         /* Hier waren wir schon einmal, daher hier nicht mehr erneut rekurrieren. */
669         ininameentry = ininameentry->next;
670     }
671
672     /* Wir sind beruhigt: bisher keine Rekursion. Aber unseren aktuellen Namen merken: */
673     ininameentry = (TIniEntry *)INI_MALLOC( sizeof( *ininameentry ) );
674     if ( !ininameentry ) return INI_ERR_NOMEMORY;
675     ininameentry->key = NULL;
676     ininameentry->value = (char *)inifilename;
677     ininameentry->next = ininames;
678     ininames = ininameentry;
679
680     /* Spezialfälle vorbereiten: Name der Standard-Protokolldatei
681      * (wie Ini-Datei, aber mit Extension .log) und der Standard-Fehlerprotokolldatei
682      * (wie Ini-Datei, aber mit Extension .err).
683      */
684     if ( !logname || !errname ) {
685         /* Basisnamen bestimmen: */
686         temp1 = INI_STRALLOC( strlen(inifilename) );
687         if (!temp1) ERROR_RETURN( inifilename, lineno, INI_ERR_NOMEMORY );
688         (void)strcpy( temp1, inifilename );
689         temp2 = strrchr( temp1, '.' );

```

```

690     if (temp2) *temp2 = '\0';
691
692     MAKE_SPECIAL_FILE_NAME( logname, temp1, ".log" ); /* Protokolldateiname */
693     MAKE_SPECIAL_FILE_NAME( errname, temp1, ".err" ); /* Fehlerprotokolldateiname */
694
695     INI_FREE(temp1);
696 }
697
698 /* Ini-Datei öffnen, I/O-Buffer setzen, falls möglich: */
699 inifile = fopen( inifilename, "r" );
700 if ( !inifile ) {
701     /* Kein Fehler, einfach diese Datei ignorieren. Aber auch aus unserem Ini-Call-Stack entfernen. */
702     ininameentry = ininames;
703     ininames = ininames->next;
704     INIS_FREE( ininameentry, sizeof( *ininameentry ) );
705     return ini_nkeys( );
706 }
707 inbuffer = (char *)INI_MALLOC(BUFSIZE);
708 if (inbuffer) setvbuf( inifile, inbuffer, _IOFBF, BUFSIZE );
709
710 /* Abschnittsnamen initialisieren: */
711 sectionnamemaxlen = 0;
712 sectionname = INI_STRALLOC(sectionnamemaxlen);
713 if ( !sectionname ) ERROR_RETURN( inifilename, lineno, INI_ERR_NOMEMORY );
714 *sectionname = '\0';
715
716 /* Alle Zeilen durchgehen: */
717 while ( fgets( minibuf, MINIBUFSIZE, inifile ) ) {
718     lineno++;
719     for ( temp1=minibuf; INI_ISSPACE(*temp1) && *temp1; temp1++ ) /* Whitespace am Zeilenanfang
720     überspringen. */
721     if (!*temp1) continue;
722     for ( temp2=temp1+strlen(temp1)-1; INI_ISSPACE(*temp2); temp2-- ) *temp2 = '\0'; /* Whitespace am
723     Zeilenende ignorieren. */
724
725     if ( strchr( "##;", *temp1 ) ) {
726         /* Kommentar in einer gesonderten Liste speichern. */
727         n = ini_add_comment(temp1+1);
728         if ( n == INI_ERR_NOMEMORY ) ERROR_RETURN( inifilename, lineno, INI_ERR_NOMEMORY );
729         continue;
730     }
731
732     if ( ( *temp1 == '[' ) && ( *temp2 == ']' ) ) {
733         /* Abschnittsname gefunden. */
734         for ( temp3=temp1+1; INI_ISSPACE(*temp3) && ( temp3 < temp2 ); temp3++ ) ;
735         if ( temp3 == temp2 ) {
736             /* Sonderfall: leerer Abschnitt (oder Abschnittsname nur aus Blanks) => unbenannter Abschnitt */
737             *sectionname = '\0';
738         } else {
739             /* Normalfall: ein echter Abschnittsname */
740             i = (size_t)( temp2 - temp1 + 1 );
741             if ( i > sectionnamemaxlen ) {
742                 sectionnamemaxlen = i;
743                 sectionname = INI_STRREALLOC( sectionname, sectionnamemaxlen );
744                 if ( !sectionname ) ERROR_RETURN( inifilename, lineno, INI_ERR_NOMEMORY );
745             }
746             INI_COPYSTR( sectionname, temp1, i );
747         }
748     } else {
749         temp2 = strchr( temp1, '=' ); /* Prüfung auf normalen Ini-Eintrag */
750         temp3 = strchr( temp1, ';' ); /* Prüfung auf Pragma */
751         if ( temp3 && ( !temp2 || ( temp3 < temp2 ) ) ) {
752             /* Pragma gefunden */
753             *( temp3++ ) = '\0';
754             n = handle_pragma( temp1, isolate_string( temp3, 0 ), &npragma, &pragmaval );
755             /* In Pragmata keine Escape-Ersetzung! */
756             if ( n < 0 ) {
757                 /* Kann nur NOSUCHPRAGMA oder NOMEMORY sein. In beiden Fällen die weitere Bearbeitung
758                 * dieser Ini-Datei einstellen und das Problem nach oben melden. */
759                 /*
760                 if ( errorlinenumber < 1 ) {
761                     /* Markiere, wo der Fehler aufgefallen ist: */
762                     ERROR_RETURN( inifilename, lineno, n );
763                 } else {
764                     /* Wir reichen nur eine bereits eingetragene Meldung weiter nach oben. */
765                     return n;
766                 }
767             }
768             if ( npragma == 2 ) resolve_escapes = pragmaval; /* falls dies das resolve_escapes-Pragma war */
769         } else {
770             /* Kein Pragma. */
771             if ( temp2 != temp1 ) {
772                 /* kein leerer Schlüssel */
773                 if ( !temp2 ) { /* kein Gleichheitszeichen, also: leerer Wert */
774                     temp2 = temp1 + strlen(temp1);
775                 } else { /* Gleichheitszeichen gefunden; Wert kann immer noch leer sein. */
776                     /* Den Schlüssel isolieren: */

```

```

775         for ( temp3=temp2-1; INI_ISSPACE(*temp3) && (temp3>=temp1); temp3-- ) *temp3 = '\\0';
776         if ( *temp2 == '=' ) *temp2 = '\\0';
777         temp2++;
778     }
779     /* temp1 zeigt jetzt auf den Key (möglicherweise leerer String!),
780      * temp2 auf den Wert (möglicherweise leerer String!)
781      */
782     temp2 = isolate_string( temp2, resolve_escapes );
783     /* ggf. Anführungszeichen um Wert entfernen: */
784     if ( ( strlen(temp2) > 1 ) && ( ( *temp2 == '"' ) || ( *temp2 == '\\' ) ) &&
785         ( *(temp2+strlen(temp2)-1) == *temp2 ) ) {
786         temp2++;
787         *(temp2+strlen(temp2)-1) = '\\0';
788     }
789     if ( strlen(sectionname) ) {
790         /* Abschnittsname vorhanden: */
791         temp3 = INI_STRALLOC( strlen(sectionname)+strlen(temp1) );
792         if ( !temp3 ) ERROR_RETURN( inifilename, lineno, INI_ERR_NOMEMORY );
793         INI_COPYSTR( temp3, sectionname, strlen(sectionname) );
794         (void)strcat( temp3, temp1 );
795         if ( !ini_add( temp3, temp2 ) ) ERROR_RETURN( inifilename, lineno, INI_ERR_NOMEMORY );
796         INI_FREE(temp3);
797     } else {
798         /* Ein wahrscheinlich häufiger Sonderfall: kein benannter Abschnitt, daher schnelleres
799         Handling: */
800         if ( !ini_add( temp1, temp2 ) ) ERROR_RETURN( inifilename, lineno, INI_ERR_NOMEMORY );
801     }
802 }
803 }
804 }
805 fclose(inifile);
806 INI_FREE(inbuffer);
807 INI_FREE(sectionname);
808
809 /* Nun unseren Namen aus der ininames-Liste streichen, damit wir nicht fälschlich Rekursion vermeiden:
810 */
811 ininames = ininameentry->next;
812 INIS_FREE( ininameentry, sizeof( *ininameentry ) );

```

8.2.4.2 ini_get() char* ini_get (

char * value,

const char * key,

const char *const defaultvalue,

const size_t maxlen)

Holt den Wert, der zu einem gegebenen Konfigurations-Schlüssel gehört.

Ein Wert kann Referenzen auf andere Schlüssel enthalten, indem der Schlüsselname in Prozentzeichen eingeschlossen wird. Diese Referenzen werden dynamisch (d. h. mit dem jeweils aktuell vorliegenden Satz von Konfigurationseinträgen) und rekursiv aufgelöst. Eine Heuristik verhindert unendliche Rekursion.

Parameter

out	value	String, der den Ergebniswert aufnimmt. Muss mindestens Platz für maxlen Zeichen (inklusive NUL) bieten. Falls NULL, wird Speicherplatz dynamisch alloziert. Ist der leere String, wenn der Schlüssel nicht existiert.
in	key	Der Schlüssel (darf auch leer oder NULL sein). Der Schlüssel darf am Anfang in eckigen Klammern einen Abschnittsnamen enthalten.
in	defaultvalue	Defaultwert, falls der Schlüssel nicht existiert. Kann NULL sein.
in	maxlen	Maximale Länge des Werts. Wenn 0, wird als Ergebnis stets der leere String geliefert.

Rückgabe

Ein Pointer auf den Ergebnisstring. NULL, falls nicht genug Speicher vorhanden.

Siehe auch

[ini_set_fallbacksection\(\)](#)

Definiert in Zeile 1665 der Datei IniFile.c.

```

1668
1669
1670 TIniEntry *inilistptr = inilist;
1671 char *tempkey, *temp = NULL, *temp2, *buffer = NULL;
1672 const char *temp3;
1673 size_t len, len2, len3;
1674 int maxchanges;
1675
1676 if ( !value ) value = INI_STRALLOC( (maxlen > 0) ? (maxlen-1) : 0 );
1677 if ( !value ) return NULL;
1678 *value = '\0';
1679 *(value+1) = '\1'; /* Optimistischer Marker, dass wir einen Wert gefunden haben werden */
1680 if ( maxlen == 0 ) return value;
1681
1682 JUMP_EMPTY_SECTION(key);
1683 tempkey = INI_STRALLOC(INI_STRLen(key)); /* NULL-Schlüssel wie leeren Schlüssel behandeln. */
1684 if (!tempkey) return NULL;
1685 INI_COPYSTRUPPER( tempkey, key );
1686 temp2 = tempkey;
1687
1688 while ( inilistptr && ( strcmp( temp2, inilistptr->key ) > 0 ) ) inilistptr = inilistptr->next;
1689
1690 if ( ( !inilistptr || !INI_STREQUAL( temp2, inilistptr->key ) ) && fallbacksection ) {
1691     /* Wert nicht gefunden, aber ein Fallback-Abschnitt ist gesetzt.
1692      * Also dort suchen, falls wir nicht eh bereits dort gesucht haben.
1693      */
1694     buffer = temp2 = NULL;
1695     if ( ( strlen(fallbacksection) == 0 ) && ( *tempkey == '[' ) ) {
1696         /* In den unbenannten Abschnitt gucken, denn das ist der Fallback-Abschnitt,
1697          * aber wir waren bis jetzt in einem benannten Abschnitt.
1698          */
1699         temp2 = strchr( tempkey, ']' );
1700         if (temp2) temp2++;
1701     } else if ( ( strlen(fallbacksection) != 0 ) && ( strstr( tempkey, fallbacksection ) != tempkey ) )
1702     {
1703         /* Wir haben bisher in einem anderen als dem Fallback-Abschnitt gesucht,
1704          * also suchen wir dort weiter. Zunächst ggf. den benannten Abschnitt ausblenden: */
1705         temp2 = strchr( tempkey, ']' );
1706         if (temp2) {
1707             temp2++;
1708         } else {
1709             temp2 = tempkey;
1710         }
1711         buffer = INI_STRALLOC( strlen(fallbacksection)+strlen(temp2) );
1712         if (buffer) {
1713             (void)strcpy( buffer, fallbacksection );
1714             (void)strcat( buffer, temp2 );
1715             temp2 = buffer;
1716         }
1717     }
1718     if (temp2) {
1719         /* temp2 zeigt jetzt auf einen neuen Namen, mit dem wir es versuchen können. */
1720         inilistptr = inilist;
1721         while ( inilistptr && ( strcmp( temp2, inilistptr->key ) > 0 ) ) inilistptr = inilistptr->next;
1722     }
1723
1724     if ( inilistptr && temp2 && INI_STREQUAL( temp2, inilistptr->key ) ) {
1725         /* gefunden; also Wert kopieren. */
1726         len = INI_STRLen(inilistptr->value);
1727         if ( len > maxlen ) len = maxlen;
1728         INI_COPYSTR( value, inilistptr->value, len );
1729     } else {
1730         /* Wert (immer noch) nicht gefunden. Also Defaultwerte abarbeiten. */
1731         if (defaultvalue) {
1732             /* Expliziter Defaultwert. */
1733             temp3 = defaultvalue;
1734         } else if ( INI_STREQUAL( tempkey, "LOGFILE" ) ) {
1735             /* Spezialfall: Standard-Protokolldatei */
1736             temp3 = logname ? logname : "IniFile.log";
1737         } else if ( INI_STREQUAL( tempkey, "ERRFILE" ) ) {
1738             /* Spezialfall: Standard-Fehlerprotokolldatei */
1739             temp3 = errname ? errname : "IniFile.err";
1740         } else {
1741             temp3 = "";
1742             *(value+1) = '\0';
1743             /* Undokumentiertes Feature: wenn wir keinen Wert gefunden haben,
1744              * folgt dem Rückgabewert (leerer String) ein weiteres NUL-Byte.
1745              */
1746         }
1747     }
1748 }

```

```

1747     len = strlen(temp3);
1748     if ( len > maxlen ) len = maxlen;
1749     INI_COPYSTR( value, temp3, len );
1750 }
1751 INI_FREE(buffer);
1752
1753 if ( recursion > 0 ) {
1754     /* Im Rekursionsfall: Einfach Wert zurückliefern; weitere Referenzen werden dort behandelt. */
1755     INI_FREE(tempkey);
1756     return value;
1757 }
1758
1759 /* Vorbereiten für Auflösen von Referenzen. Heuristik: maximal so viele
1760 * Referenzen auflösen, wie der ursprüngliche Eintrag an Referenzen besitzt,
1761 * multipliziert mit der aktuellen Anzahl vorliegender Einträge insgesamt.
1762 * Also: jeder Eintrag bekommt statistisch gesehen zwei Chancen,
1763 * expandiert zu werden.
1764 */
1765 recursion = 1;
1766 maxchanges = 0;
1767 temp = value;
1768 while (temp) {
1769     temp = strchr( temp, '%' );
1770     if (temp) {
1771         maxchanges++;
1772         temp++;
1773     }
1774 }
1775 maxchanges = (maxchanges/2) * ini_nkeys();
1776 buffer = INI_STRALLOC(maxlen);
1777 if ( !buffer ) return NULL;
1778
1779 /* Den gesamten Wert nach Prozentzeichen durchgehen. */
1780 temp = value;
1781 while ( ( ( temp = strchr( temp, '%' ) ) != NULL ) && ( maxchanges > 0 ) ) {
1782     /* Referenz gefunden. Schlüssel extrahieren. */
1783     if ( ( temp2 = strchr( temp+1, '%' ) ) == NULL ) break; /* Nee, war doch keine. */
1784     len = (size_t)( temp2 - temp - 1 );
1785     tempkey = INI_STRREALLOC( tempkey, len );
1786     if ( !tempkey ) return NULL;
1787     INI_COPYSTR( tempkey, temp+1, len );
1788     /* Wert der Referenz rekursiv bestimmen. */
1789     ini_get( buffer, tempkey, NULL, maxlen );
1790     if ( *buffer || *(buffer+1) ) {
1791         /* Referenz durch ihren Wert ersetzen, auch dann wenn der gefundene Wert der leere String ist
1792         * (aber nicht, wenn der Wert nicht gefunden wurde -- dazu benutzen wir den undokumentierten
1793         * Marker direkt hinter dem Ende des leeren Strings).
1794         */
1795         len3 = strlen(buffer);
1796         len2 = strlen(temp+1);
1797         if ( len3+len2 > maxlen ) len2 = maxlen - len3;
1798         strncat( buffer, temp+1, len2 );
1799         *temp = '\0';
1800         len = strlen(temp);
1801         len2 = strlen(buffer);
1802         if ( len+len2 > maxlen ) len2 = maxlen - len;
1803         strncat( temp, buffer, len2 );
1804
1805         if ( ( ( temp2 = strchr( temp+len+len3, '%' ) ) != NULL ) &&
1806             ( strchr( temp2+1, '%' ) != NULL )
1807         ) {
1808             /* Hinter unserer Ersetzung gibt es weitere Ersetzungskandidaten.
1809             * Die arbeiten wir zuerst ab, ehe wir unseren Rekursionszähler anpassen.
1810             */
1811             temp += len + len3;
1812         } else {
1813             /* Keine weiteren Ersetzungskandidaten gefunden. Dann noch mal von vorne an prüfen,
1814             * ob wir rekursiv tätig werden müssen; dabei aber buchführen, damit wir nicht ewig laufen.
1815             */
1816             maxchanges--;
1817             temp = value;
1818         }
1819     } else {
1820         temp++;
1821     }
1822 }
1823
1824 INI_FREE(buffer);
1825 INI_FREE(tempkey);
1826 recursion = 0;

```

8.2.4.3 ini_add() int ini_add (

```
const char * key,
const char *const value )
```

Fügt einen Konfigurationseintrag (Schlüssel/Wert) hinzu.

Parameter

in	key	Der Schlüssel (darf auch leer oder NULL sein). Der Schlüssel darf am Anfang in eckigen Klammern einen Abschnittsnamen enthalten.
in	value	Der Wert (darf leer sein). Falls NULL, wird der entsprechende Eintrag gelöscht.

Rückgabe

Anzahl der nach dem Hinzufügen verfügbaren Einträge. INI_ERR_NOMEMORY, falls nicht genügend Speicherplatz verfügbar.

Definiert in Zeile 825 der Datei IniFile.c.

```
828                                     {
829     TIniEntry *inilistptr = inilist;
830     TIniEntry *previousinilistptr = NULL;
831     TIniEntry *newinilistentry;
832     char *tempkey;
833
834     if ( !got_license ) {
835         fprintf( stderr, "This software uses the IniFile library.\n%s", COPYRIGHT );
836         got_license = 1;
837     }
838
839     if ( !value ) return ini_delete(key); /* NULL-Wert: dann löschen. */
840
841     /* Eintrag grafieunabhängig suchen in der Liste: */
842     JUMP_EMPTY_SECTION(key);
843     tempkey = INI_STRALLOC( INI_STRLEN(key)); /* NULL-Schlüssel wie leeren Schlüssel behandeln. */
844     if ( !tempkey ) return INI_ERR_NOMEMORY;
845     INI_COPYSTRUPPER( tempkey, key );
846
847     while ( inilistptr && ( strcmp( tempkey, inilistptr->key ) > 0 ) ) {
848         previousinilistptr = inilistptr;
849         inilistptr = inilistptr->next;
850     }
851
852     if ( inilistptr && INI_STREQUAL( tempkey, inilistptr->key ) ) {
853         /* gefunden; also alten Wert überschreiben */
854         inilistptr->value = INI_STRREALLOC( inilistptr->value, strlen(value) );
855         if ( !inilistptr->value ) {
856             INI_FREE(tempkey);
857             return INI_ERR_NOMEMORY;
858         }
859         (void)strcpy( inilistptr->value, value );
860         INI_FREE(tempkey);
861     } else {
862         /* Neuen Eintrag hinzufügen. */
863         newinilistentry = (TIniEntry *)INI_MALLOC( sizeof(*newinilistentry) );
864         if ( !newinilistentry ) {
865             INI_FREE(tempkey);
866             return INI_ERR_NOMEMORY;
867         }
868         newinilistentry->key = tempkey;
869         newinilistentry->value = INI_STRALLOC( strlen(value) );
870         if ( !newinilistentry->value ) {
871             INI_FREE(tempkey);
872             INI_FREE(newinilistentry);
873             return INI_ERR_NOMEMORY;
874         }
875         (void)strcpy( newinilistentry->key, tempkey );
876         (void)strcpy( newinilistentry->value, value );
877         /* Nachfolger des neuen Eintrags wird der in der Sortierung nächstgrößere Alteintrag. */
878         newinilistentry->next = inilistptr;
879
880         if ( !previousinilistptr ) {
881             /* Einfügung am Anfang, also Anfangspointer neu setzen. */
882             inilist = newinilistentry;
883         } else {
884             /* Der in der Sortierung voran gehende Eintrag erhält den neuen als Nachfolger. */
885             previousinilistptr->next = newinilistentry;
886         }
887     }
```

```

887     entrycount++;
888 }
889
890 sectioncount = 0; /* Gecachten Wert invalidieren */

```

8.2.4.4 ini_set() `int ini_set (`
 `const char * key,`
 `const char *const value )`

Synonym für [ini_add\(\)](#):

Synonym für [ini_add\(\)](#):

Parameter

in	<i>key</i>	Der Schlüssel (darf auch leer oder NULL sein). Der Schlüssel darf am Anfang in eckigen Klammern einen Abschnittsnamen enthalten.
in	<i>value</i>	Der Wert (darf leer sein). Falls NULL, wird der entsprechende Eintrag gelöscht.

Rückgabe

Anzahl der nach dem Einlesen verfügbaren Einträge. INI_ERR_NOMEMORY, falls nicht genügend Speicherplatz verfügbar.

Siehe auch

[ini_add\(\)](#)

Definiert in Zeile 906 der Datei IniFile.c.

```
909 { return ini_add(key, value); }
```

8.2.4.5 ini_keys() `char** ini_keys (`
 `char ** keylist,`
 `int maxkeys )`

Liefert eine Liste der Schlüssel.

Liefert eine Liste aller definierten Schlüssel zurück.

Parameter

out	<i>keylist</i>	Pointer auf ein Array von Strings, das die Schlüssel aufnimmt. Schlüssel in benannten Abschnitten enthalten am Anfang den Abschnittsnamen in eckigen Klammern. Wird terminiert durch einen NULL-Pointer, sofern Platz dafür vorhanden ist. Platz für die Strings wird dynamisch alloziert. Kann NULL sein; dann wird Speicher dynamisch alloziert.
in	<i>maxkeys</i>	Maximal zurückzuliefernde Anzahl von Schlüssel. Kann ≤ 0 sein, dann keine Begrenzung. keylist muss (falls ungleich NULL) jedenfalls groß genug sein, um diese Einträge und den terminierenden NULL-Eintrag aufzunehmen.

Rückgabe

Pointer auf das Array von Strings; oder NULL, falls nicht genug Speicherplatz.

Siehe auch

[ini_nkeys\(\)](#)

Fehler Bei Abbruch wegen Speichermangel wird bereits allozierter Speicher nicht wieder freigegeben.

Definiert in Zeile 1031 der Datei IniFile.c.

```

1034                                     {
1035
1036     TIniEntry *inilistptr = inilist;
1037     char **keyptr;
1038     int keycount;
1039
1040     if ( !keylist ) {
1041         /* Speicherplatz allozieren. */
1042         keycount = ini_nkeys( ) + 1;
1043         if ( ( maxkeys > 0 ) && ( keycount > maxkeys ) ) keycount = maxkeys;
1044         if ( maxkeys <= 0 ) maxkeys = keycount;
1045         keylist = (char **)INI_MALLOC( ( (unsigned)keycount+1 ) * sizeof(char *) );
1046         if ( !keylist ) return NULL;
1047     }
1048
1049     /* Alle Einträge bis zur erlaubten Maximalzahl eintragen. */
1050     for ( keycount=0, keyptr=keylist; inilistptr && (keycount < maxkeys); keycount++,
1051         inilistptr=inilistptr->next ) {
1052         *keyptr = INI_STRALLOC( strlen(inilistptr->key) );
1053         if ( !*keyptr ) return NULL;
1054         (void)strcpy( *(keyptr++), inilistptr->key );
1055     }
1056     if ( keycount < maxkeys ) *keyptr = (char *)NULL; /* Terminierenden NULL-Pointer eintragen */

```

8.2.4.6 ini_keys_section() char** ini_keys_section (
 char ** keylist,
 int maxkeys,
 const char * sectionname)

Liefert eine Liste der Schlüssel in einem gegebenen Abschnitt.

Liefert eine Liste aller definierten Schlüssel in einem vorgegebenen Abschnitt zurück.

Parameter

out	<i>keylist</i>	Pointer auf ein Array von Strings, das die Schlüssel aufnimmt. Schlüssel in benannten Abschnitten enthalten am Anfang den Abschnittsnamen in eckigen Klammern. Wird terminiert durch einen NULL-Pointer, sofern Platz dafür vorhanden ist. Platz für die Strings wird dynamisch alloziert. Kann NULL sein; dann wird Speicher dynamisch alloziert.
in	<i>maxkeys</i>	Maximal zurückzuliefernde Anzahl von Schlüssel. Kann <= 0 sein, dann keine Begrenzung. keylist muss (falls ungleich NULL) jedenfalls groß genug sein, um diese Einträge und den terminierenden NULL-Eintrag aufzunehmen.
in	<i>sectionname</i>	Name des Abschnitts. Kann auch NULL, leer oder [] sein. Begrenzende eckige Klammern dürfen fehlen.

Rückgabe

Pointer auf das Array von Strings; oder NULL, falls nicht genug Speicherplatz.

Siehe auch

[ini_nkeys_section\(\)](#)

Fehler Bei Abbruch wegen Speichermangel wird bereits allozierter Speicher nicht wieder freigegeben.

Definiert in Zeile 1084 der Datei IniFile.c.

```

1087
1088
1089     TIniEntry *inilistptr = inilist;
1090     char **keyptr;
1091     char *mysectionname;
1092     size_t sectionnamelen;
1093     int keycount;
1094
1095     mysectionname = make_section_name(sectionname);
1096     if ( !mysectionname ) return NULL;
1097     sectionnamelen = strlen(mysectionname);
1098
1099     if ( !keylist ) {
1100         /* Speicherplatz allozieren. */
1101         keycount = ini_nkeys_section(mysectionname) + 1;
1102         if ( ( maxkeys > 0 ) && ( keycount > maxkeys ) ) keycount = maxkeys;
1103         if ( maxkeys <= 0 ) maxkeys = keycount;
1104         keylist = (char **)INI_MALLOC( ( (unsigned)keycount+1 ) * sizeof(char *) );
1105         if ( !keylist ) return NULL;
1106     }
1107
1108     /* Alle Einträge durchgehen, dabei nur die passenden berücksichtigen, und höchstens bis zur
1109     Maximalzahl. */
1109     for ( inilistptr=inilist, keyptr=keylist, keycount = 0; inilistptr && ( keycount < maxkeys );
1110           inilistptr=inilistptr->next ) {
1111         if ( ( ( sectionnamelen != 0 ) &&
1112               ( strlen(inilistptr->key) >= sectionnamelen ) &&
1113               ( memcmp( inilistptr->key, mysectionname, sectionnamelen ) == 0 )
1114               ) ||
1115             ( ( sectionnamelen == 0 ) && ( ( strlen(inilistptr->key) == 0 ) || ( *(inilistptr->key) != '['
1116           ) ) ) {
1117             *keyptr = INI_STRALLOC( strlen(inilistptr->key) );
1118             if ( !*keyptr ) return NULL;
1119             (void)strcpy( *(keyptr++), inilistptr->key );
1120             keycount++;
1121         }
1122     }
1123     if ( keycount < maxkeys ) *keyptr = (char *)NULL; /* Terminierenden NULL-Pointer eintragen */

```

8.2.4.7 ini_sections() char** ini_sections (
char ** sectionlist,
int maxsections)

Liefert eine Liste der Konfigurationsabschnitte zurück.

Liefert eine Liste aller definierten Abschnitte (Sections) zurück.

Parameter

out	<i>sectionlist</i>	Pointer auf ein Array von Strings, das die Abschnittsnamen aufnimmt. Die Abschnittsnamen enthalten begrenzende eckige Klammern, der unbenannte Abschnitt ist jedoch ein leerer String. Wird terminiert durch einen NULL-Pointer, sofern Platz dafür vorhanden ist. Platz für die Strings wird dynamisch alloziert. Kann NULL sein; dann wird Speicher dynamisch alloziert.
in	<i>maxsections</i>	Maximal zurückzuliefernde Anzahl von Abschnitten. Kann <= 0 sein, dann keine Begrenzung. sectionlist muss (falls ungleich NULL) jedenfalls groß genug sein, um diese Einträge und den terminierenden NULL-Eintrag aufzunehmen.

Rückgabe

Pointer auf das Array von Strings; oder NULL, falls nicht genug Speicherplatz.

Siehe auch

[ini_nsections\(\)](#)

Fehler Bei Abbruch wegen Speichermangel wird bereits allozierter Speicher nicht wieder freigegeben.

Definiert in Zeile 1149 der Datei IniFile.c.

```

1152                                     {
1153     TIniEntry *inilistptr;
1154     char **sectionptr;
1155     char *sectionname, *temp;
1156     size_t sectionnamemaxlen, i;
1157     int mysectioncount = 0;
1158     int foundunbenannt = 0;
1159     int isNewSection;
1160
1161     if ( !sectioncount ) {
1162         /* Nötigenfalls die Anzahl der Abschnitte neu ermitteln */
1163         if ( ini_nsections() < 0 ) return NULL;
1164     }
1165
1166     if ( !sectionlist ) {
1167         /* Speicherplatz allozieren. */
1168         mysectioncount = sectioncount + 1;
1169         if ( ( maxsections > 0 ) && ( mysectioncount > maxsections ) ) mysectioncount = maxsections;
1170         if ( maxsections <= 0 ) maxsections = mysectioncount;
1171         sectionlist = (char **)INI_MALLOC( ( (unsigned)mysectioncount+1 ) * sizeof(char *) );
1172         if ( !sectionlist ) return NULL;
1173     }
1174
1175     /* Abschnittsnamen initialisieren: */
1176     sectionnamemaxlen = 0;
1177     sectionname = INI_STRALLOC(sectionnamemaxlen);
1178     if ( !sectionname ) return NULL;
1179     *sectionname = '\0';
1180
1181     /* Alle Einträge durchgehen und bei jedem neuen Abschnittsnamen diesen in die Liste übernehmen.
1182      * Wir wissen: Die Einträge sind grafieunabhängig sortiert, brauchen also keine
1183      * Historie der gesehenen Einträge zu checken. */
1184     for ( inilistptr=inilist, sectionptr=sectionlist; inilistptr; inilistptr=inilistptr->next ) {
1185         if ( strlen(inilistptr->key) ) {
1186             isNewSection = 0;
1187             if ( *(inilistptr->key) == '[' ) {
1188                 /* Eintrag mit benamstem Abschnitt */
1189                 temp = strchr( inilistptr->key, '[' );
1190                 if (temp) { /* Sicherheitshalber; alles andere wäre sehr merkwürdig. */
1191                     i = (size_t)( temp - inilistptr->key + 1 );
1192                     if ( (i != strlen(sectionname) ) || ( memcmp( sectionname, inilistptr->key, i ) != 0 ) ) {
1193                         /* Neuer Abschnittsnamen gefunden! */
1194                         if ( i > sectionnamemaxlen ) {
1195                             sectionnamemaxlen = i;
1196                             sectionname = INI_STRREALLOC( sectionname, sectionnamemaxlen );
1197                             if ( !sectionname ) return NULL;
1198                         }
1199                         INI_COPYSTR( sectionname, inilistptr->key, i );
1200                         isNewSection = 1;
1201                     }
1202                 }
1203             } else {
1204                 /* Unbenannter Abschnitt. */
1205                 if ( !foundunbenannt ) {
1206                     foundunbenannt = 1;
1207                     isNewSection = 1;
1208                 }
1209                 *sectionname = '\0';
1210             }
1211             if (isNewSection) {
1212                 /* Neuen Eintrag in die Liste übernehmen */
1213                 *sectionptr = INI_STRALLOC( strlen(sectionname) );
1214                 if ( !*sectionptr ) return NULL;
1215                 (void)strcpy( *(sectionptr++), sectionname );
1216             }
1217         }
1218     }
1219
1220     if (mysectioncount < maxsections) *sectionptr = (char *)NULL; /* Terminierenden NULL-Pointer
eintragen */

```

8.2.4.8 ini_nkeys() int ini_nkeys (
 void)

Liefert die Anzahl der Konfigurationseinträge zurück.

Rückgabe

Die Anzahl der Konfigurationseinträge.

Definiert in Zeile 914 der Datei IniFile.c.

```
917 { return entrycount; }
```

8.2.4.9 ini_nkeys_section() int ini_nkeys_section (
 const char * sectionname)

Liefert die Anzahl der Konfigurationseinträge in einem gegebenen Abschnitt zurück.

Parameter

in	<i>sectionname</i>	Name des Abschnitts. Kann auch NULL, leer oder [] sein. Begrenzende eckige Klammern dürfen fehlen.
----	--------------------	--

Rückgabe

Die Anzahl der Konfigurationseinträge in dem genannten Abschnitt.

Definiert in Zeile 925 der Datei IniFile.c.

```
928                                     {
929     TIniEntry *inilistptr;
930     char *mysectionname;
931     size_t sectionnamelen;
932     int keycount = 0;
933
934     mysectionname = make_section_name(sectionname);
935     if ( !mysectionname ) return INI_ERR_NOMEMORY;
936     sectionnamelen = strlen(mysectionname);
937
938     /* Alle Einträge durchgehen, dabei nur die passenden berücksichtigen. */
939     for ( inilistptr=inilist; inilistptr; inilistptr=inilistptr->next ) {
940         if ( ( ( sectionnamelen != 0 ) &&
941             ( strlen( inilistptr->key ) >= sectionnamelen ) &&
942             ( memcmp( inilistptr->key, mysectionname, sectionnamelen ) == 0 )
943             ) ||
944             ( ( sectionnamelen == 0 ) &&
945             ( ( strlen(inilistptr->key) == 0 ) || ( *(inilistptr->key) != '[' ) )
946             )
947         ) keycount++;
948     }
949     INI_FREE(mysectionname);
```

8.2.4.10 ini_nsections() int ini_nsections (
 void)

Liefert die Anzahl der Konfigurationsabschnitte zurück.

Rückgabe

Die Anzahl der Abschnitte (inkl. des unbenannten Abschnitts).

Definiert in Zeile 957 der Datei IniFile.c.

```

960     {
961     TIniEntry *inilistptr;
962     char *sectionname, *temp;
963     size_t sectionnamemaxlen, i;
964     int foundunbenannt = 0;
965
966     if (sectioncount) return sectioncount; /* Gecacheter Wert liegt vor, alle sind froh. */
967
968     sectionnamemaxlen = 0;
969     sectionname = INI_STRALLOC(sectionnamemaxlen);
970     if ( !sectionname ) return INI_ERR_NOMEMORY;
971
972     *sectionname = '\0';
973
974     /* Alle Einträge durchgehen, dabei Sektionsnamen zählen.
975     * Wir wissen: Die Einträge sind grafieunabhängig sortiert, brauchen also keine
976     * Historie der gesehenen Einträge nachzuhalten. */
977     for ( inilistptr=inilist; inilistptr; inilistptr=inilistptr->next ) {
978         if ( strlen(inilistptr->key) ) {
979             if ( *(inilistptr->key) == '[' ) {
980                 /* Eintrag mit benamstem Abschnitt */
981                 temp = strchr( inilistptr->key, ']' );
982                 if (temp) { /* Sicherheitshalber; alles andere wäre sehr merkwürdig. */
983                     i = (size_t)( temp - inilistptr->key + 1 );
984                     if ( ( i != strlen(sectionname) ) || ( memcmp( sectionname, inilistptr->key, i ) != 0 ) ) {
985                         /* Neuer Abschnittsname gefunden! */
986                         sectioncount++;
987                         if ( i > sectionnamemaxlen ) {
988                             sectionnamemaxlen = i;
989                             sectionname = INI_STRREALLOC( sectionname, sectionnamemaxlen );
990                             if ( !sectionname ) return INI_ERR_NOMEMORY;
991                         }
992                         INI_COPYSTR( sectionname, inilistptr->key, i );
993                     }
994                 }
995             } else {
996                 /* Unbenannter Abschnitt. */
997                 if ( !foundunbenannt ) {
998                     foundunbenannt = 1;
999                     sectioncount++;
1000                 }
1001                 *sectionname = '\0';
1002             }
1003         }
1004     }
1005     INI_FREE(sectionname);

```

8.2.4.11 ini_ncomments() int ini_ncomments (
 void)

Liefert die Anzahl der Kommentare zurück.

Rückgabe

Die Anzahl der Kommentare.

Definiert in Zeile 1228 der Datei IniFile.c.

```

1231 { return commentcount; }

```

8.2.4.12 ini_comments() char** ini_comments (
 char ** commentlist,
 int maxcomment)

Liefert eine Liste der Kommentare.

Liefert eine Liste aller Kommentare zurück.

Parameter

out	<i>commentlist</i>	Pointer auf ein Array von Strings, das die Kommentare aufnimmt. Wird terminiert durch einen NULL-Pointer, sofern Platz dafür vorhanden ist. Platz für die Strings wird dynamisch alloziert. Kann NULL sein; dann wird Speicher dynamisch alloziert.
in	<i>maxcomment</i>	Maximal zurückzuliefernde Anzahl von Kommentaren. Kann ≤ 0 sein, dann keine Begrenzung. <i>commentlist</i> muss (falls ungleich NULL) jedenfalls groß genug sein, um diese Einträge und den terminierenden NULL-Eintrag aufzunehmen.

Rückgabe

Pointer auf das Array von Strings; oder NULL, falls nicht genug Speicherplatz.

Siehe auch

[ini_ncomments\(\)](#)

Fehler Bei Abbruch wegen Speichermangel wird bereits allozierter Speicher nicht wieder freigegeben.

Definiert in Zeile 1252 der Datei IniFile.c.

```

1255                                     {
1256
1257     TIniEntry *inicommentptr = comments;
1258     char **commentptr;
1259     int commentcount;
1260
1261     if ( !commentlist ) {
1262         /* Speicherplatz allozieren. */
1263         commentcount = ini_ncomments( ) + 1;
1264         if ( ( maxcomment > 0 ) && ( commentcount > maxcomment ) ) commentcount = maxcomment;
1265         if ( maxcomment <= 0 ) maxcomment = commentcount;
1266         commentlist = (char **)INI_MALLOC( ( (unsigned)commentcount+1 ) * sizeof(char * ) );
1267         if ( !commentlist ) return NULL;
1268     }
1269
1270     /* Alle Kommentare bis zur erlaubten Maximalzahl eintragen. */
1271     for ( commentcount=0, commentptr=commentlist; inicommentptr && (commentcount < maxcomment);
1272           commentcount++, inicommentptr=inicommentptr->next ) {
1273         *commentptr = INI_STRALLOC( strlen(inicommentptr->value) );
1274         if ( !*commentptr ) return NULL;
1275         (void)strcpy( *(commentptr++), inicommentptr->value );
1276     }
1277     if ( commentcount < maxcomment ) *commentptr = (char *)NULL; /* Terminierenden NULL-Pointer eintragen
1278     */

```

8.2.4.13 ini_add_comment() int ini_add_comment (
const char * text)

Fügt eine Kommentarzeile hinzu.

Parameter

in	<i>text</i>	Der Kommentar (ohne einleitendes Kommentarzeichen; darf leer sein).
----	-------------	---

Rückgabe

Anzahl der nach dem Hinzufügen verfügbaren Kommentarzeilen. `INI_ERR_NOMEMORY`, falls nicht genügend Speicherplatz verfügbar.

Definiert in Zeile 1288 der Datei `IniFile.c`.

```

1291                                     {
1292     TIniEntry *commentptr = comments;
1293     TIniEntry *previouscommentptr = NULL;
1294     TIniEntry *newcomment;
1295
1296     if ( !text ) return commentcount;
1297
1298     /* Neuen Eintrag hinzufügen. */
1299     newcomment = (TIniEntry *)INI_MALLOC( sizeof(*newcomment) );
1300     if ( !newcomment ) return INI_ERR_NOMEMORY;
1301     newcomment->key = NULL;
1302     newcomment->next = NULL;
1303     newcomment->value = INI_STRALLOC( strlen(text) );
1304     if ( !newcomment->value ) {
1305         INI_FREE(newcomment);
1306         return INI_ERR_NOMEMORY;
1307     }
1308     (void)strcpy( newcomment->value, text );
1309
1310     if ( commentptr ) {
1311         /* Wir haben schon mindesetns einen Kommentar.
1312          * Den neuen Eintrag ans Ende der Kommentarliste anhängen:
1313          */
1314         while ( commentptr->next ) commentptr = commentptr->next;
1315         commentptr->next = newcomment;
1316     } else {
1317         /* Dies ist unser erster Kommentar. */
1318         comments = newcomment;
1319     }

```

8.2.4.14 ini_delete() `int ini_delete (`
 `const char * key )`

Löscht einen Schlüssel, falls es ihn gibt.

Parameter

in	key	Der Schlüssel (darf auch leer oder NULL sein). Der Schlüssel darf am Anfang in eckigen Klammern einen Abschnittsnamen enthalten.
----	-----	--

Rückgabe

Anzahl der nach dem Einlesen verfügbaren Einträge. `INI_ERR_NOMEMORY`, falls nicht genügend Speicherplatz verfügbar.

Definiert in Zeile 1331 der Datei `IniFile.c`.

```

1334                                     {
1335     TIniEntry *inilistptr = inilist;
1336     TIniEntry *previousinilistptr = NULL;
1337     char *tempkey;
1338
1339     /* Eintrag grafieunabhängig suchen in der Liste: */
1340     JUMP_EMPTY_SECTION(key);
1341     tempkey = INI_STRALLOC( INI_STRLEN(key) ); /* NULL-Schlüssel wie leeren Schlüssel behandeln. */
1342     if ( !tempkey ) return INI_ERR_NOMEMORY;
1343     INI_COPYSTRUPPER(tempkey, key);
1344
1345     while ( inilistptr && ( strcmp( tempkey, inilistptr->key ) > 0 ) ) {
1346         previousinilistptr = inilistptr;
1347         inilistptr = inilistptr->next;
1348     }
1349

```

```

1350 if ( inilistptr && INI_STREQUAL( tempkey, inilistptr->key ) ) {
1351     /* gefunden; also löschen. */
1352     if ( !previousinilistptr ) {
1353         /* Es war der erste EWintrag der Liste; also Listenanfang neu setzen. */
1354         inilist = inilistptr->next;
1355     } else {
1356         /* Im voran gehenden Eintrag den Pointer auf das nächste Element umsetzen. */
1357         previousinilistptr->next = inilistptr->next;
1358     }
1359     /* Den von diesem Eintrag belegten Speicher freigeben. */
1360     INI_FREE(inilistptr->key);
1361     INI_FREE(inilistptr->value);
1362     INI_FREE(inilistptr);
1363     entrycount--;
1364 }
1365 INI_FREE(tempkey);
1366
1367 sectioncount = 0; /* Gecacheten Wert invalidieren */

```

8.2.4.15 ini_write() int ini_write (

const char *const inifilename,

const char * comment)

Schreibt die aktuellen Ini-Werte in eine Datei.

Schreibt den aktuellen Gesamtbestand der Schlüssel-/Werte-Paare in eine Datei, wahlweise mit einem Kommentar. Platzhalter werden nicht aufgelöst, sondern wörtlich ausgeschrieben.

Parameter

in	<i>inifilename</i>	Name der zu schreibenden Datei. Wenn NULL oder leer, wird nach stdout geschrieben.
in	<i>comment</i>	An den Anfang der Datei zu schreibender Kommentar. Wenn ein solcher angegeben ist, werden bisher vorhandene (eingesene oder mit ini_add_comment() hinzugefügte) Kommentare ignoriert. Kann mehrzeilig sein. Kann NULL sein.

Rückgabe

0, falls OK. INI_ERR_NOMEMORY, falls nicht genügend Speicherplatz verfügbar.

Siehe auch

[ini_nkeys\(\)](#)

Definiert in Zeile 1390 der Datei IniFile.c.

```

1393
1394
1395 FILE *inifile;
1396 TIniEntry *inilistptr;
1397 char *outbuffer = NULL;
1398 static const char * const delims[ ] = { " ", "\"", "'" };
1399 static const char * const nonprintables =
1400     "\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20\x21\x22\x23\x24\x25\x26\x27\x28\x29\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x40\x41\x42\x43\x44\x45\x46\x47\x48\x49\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x60\x61\x62\x63\x64\x65\x66\x67\x68\x69\x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x80\x81\x82\x83\x84\x85\x86\x87\x88\x89\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x00";
1401 char minibuf[MINIBUFSIZE];
1402 char *temp, *temp2, *keyptr, *sectionname;
1403 const char *templ;
1404 size_t sectionnamemaxlen, i;
1405 int isError, isNewSection, hadNamedSection, hasNonprintables, resolve_escapes=0;
1406
1407 isError = 0;
1408 if ( INI_STRLEN(inifilename) ) {
1409     inifile = fopen( inifilename, "w" );
1410     if (!inifile) return INI_ERR_NOFILE;
1411     outbuffer = (char *)INI_MALLOC(BUFSIZE);
1412     if (outbuffer) setvbuf( inifile, outbuffer, _IOFBF, BUFSIZE );

```

```

1412 } else {
1413     inifile = stdout;
1414 }
1415
1416 if ( INI_STRLLEN(comment) ) {
1417     /* Explizit angegebene neue Kommentare schreiben; bisher vorhandene Kommentare ignorieren. */
1418     while ( INI_STRLLEN(comment) ) {
1419         templ = strchr(comment, '\n');
1420         if (!templ) templ = comment + strlen(comment);
1421         if ( fprintf( inifile, "# " ) < 0 ) isError = 1;
1422         if ( !isError ) {
1423             if ( fwrite( comment, sizeof(char), (size_t)( templ-comment ), inifile ) < (size_t)(
templ-comment ) ) isError = 1;
1424         }
1425         if ( !isError ) {
1426             if ( fprintf( inifile, "\n" ) < 0 ) isError = 1;
1427         }
1428         if (isError) break;
1429         comment = ( *templ == '\0' ) ? templ : (templ+1);
1430     }
1431 } else {
1432     /* Ggf. bereits vorher vorhandene Kommentare schreiben: */
1433     inilistptr = comments;
1434     while (inilistptr) {
1435         if ( fprintf( inifile, "# %s\n", inilistptr->value ) < 0 ) isError = 1;
1436         inilistptr = inilistptr->next;
1437     }
1438 }
1439
1440 /* Abschnittsnamen initialisieren: */
1441 sectionnamemaxlen = 0;
1442 sectionname = INI_STRALLOC(sectionnamemaxlen);
1443 if ( !sectionname ) return INI_ERR_NOMEMORY;
1444 *sectionname = '\0';
1445 hadNamedSection = 0;
1446
1447 /* Alle Einträge schreiben. */
1448 for ( inilistptr=inilist; inilistptr && !isError; inilistptr=inilistptr->next ) {
1449     keyptr = inilistptr->key ? inilistptr->key : delims[0];
1450     /* Sicherheitshalber Zeilenende-Zeichen auf Blank umsetzen. (Inplace -- you get what you pay for.)
1451     */
1452     while ( ( temp = strchr( keyptr, '\n' ) ) != NULL ) *temp = ' ';
1453     while ( ( temp = strchr( keyptr, '\r' ) ) != NULL ) *temp = ' ';
1454
1455     /* Prüfung auf Abschnittswechsel: */
1456     isNewSection = 0;
1457     if ( *keyptr == '[' ) {
1458         /* Eintrag mit benamstem Abschnitt */
1459         temp = strchr( keyptr, ']' );
1460         if (temp) { /* Sicherheitshalber; alles andere wäre sehr merkwürdig. */
1461             i = (size_t)( temp - keyptr + 1 );
1462             if ( ( i != strlen(sectionname) ) || ( memcmp( sectionname, keyptr, i ) != 0 ) ) {
1463                 /* Neuer Abschnittsname gefunden! */
1464                 if ( i > sectionnamemaxlen ) {
1465                     sectionnamemaxlen = i;
1466                     sectionname = INI_STRREALLOC( sectionname, sectionnamemaxlen );
1467                     if ( !sectionname ) return INI_ERR_NOMEMORY;
1468                 }
1469                 INI_COPYSTR( sectionname, keyptr, i );
1470                 isNewSection = 1;
1471             }
1472             keyptr = temp + 1;
1473             hadNamedSection = 1;
1474         }
1475     } else {
1476         *sectionname = '\0';
1477     }
1478
1479     if ( isNewSection ) {
1480         /* Neuer Abschnitt. Abschnittskennung muss geschrieben werden, falls
1481         * (i) benannter Abschnitt, oder (ii) unbenannter Abschnitt nach benanntem Abschnitt. */
1482         if ( INI_STRLLEN(sectionname) ) {
1483             if ( fprintf( inifile, "%s\n", sectionname ) < 0 ) isError = 1;
1484         } else {
1485             if (hadNamedSection) {
1486                 if ( fprintf( inifile, "[]\n" ) < 0 ) isError = 1;
1487             }
1488         }
1489     }
1490
1491     if ( !INI_STRLLEN(inilistptr->value) ) {
1492         /* Leerer Wert: */
1493         if ( fprintf( inifile, "%s=\n", keyptr ) < 0 ) isError = 1;
1494     } else {
1495         /* Nichtleerer Wert. */
1496         /* Prüfung auf nichtdruckbare Sonderzeichen unterhalb von \x20 (nur im Value): */

```



```

1497     hasNonprintables = ( strstr( inilistptr->value, nonprintables ) < strlen( inilistptr->value ) );
1498     if ( hasNonprintables && !resolve_escapes ) {
1499         if ( fprintf( inifile, "resolve_escapes: 1\n" ) < 0 ) isError = 1;
1500         resolve_escapes = 1;
1501     }
1502
1503     if ( hasNonprintables || ( resolve_escapes && strchr( inilistptr->value, '\\\' ) ) ) {
1504         /* Zeichen für Zeichen kopieren und dabei ggf. in Hex umsetzen: */
1505         for ( temp=inilistptr->value, temp2=minibuf; *temp; temp++, temp2++ ) {
1506             if ( *temp < ' ' ) {
1507                 if ( temp2-minibuf+4 >= MINIBUFSIZE ) break;
1508                 sprintf( temp2, "\\x%02x", *temp );
1509                 temp2 += 3;
1510             } else if ( *temp == '\\\' ) {
1511                 if ( temp2-minibuf+2 >= MINIBUFSIZE ) break;
1512                 *(temp2++) = *temp;
1513                 *temp2 = *temp;
1514             } else {
1515                 if ( temp2-minibuf+1 >= MINIBUFSIZE ) break;
1516                 *temp2 = *temp;
1517             }
1518         }
1519         *temp2 = '\\0';
1520     } else {
1521         i = strlen( inilistptr->value );
1522         if ( i >= MINIBUFSIZE ) i = MINIBUFSIZE-1;
1523         memcpy( minibuf, inilistptr->value, i+1 );
1524     }
1525
1526     if ( INI_ISSPACE( *(inilistptr->value) ) || INI_ISSPACE( *( inilistptr->value+ (
1527         strlen(inilistptr->value)-1 ) ) ) ) {
1528         templ = delims[ strchr( inilistptr->value, '"' ) ? 2 : 1 ];
1529         if ( fprintf( inifile, "%s=%s%s\n", keyptr, templ, minibuf, templ ) < 0 ) isError = 1;
1530     } else {
1531         if ( fprintf( inifile, "%s=%s\n", keyptr, minibuf ) < 0 ) isError = 1;
1532     }
1533 }
1534 fflush(inifile);
1535 if ( inifile != stdout ) fclose(inifile);
1536 INI_FREE(outbuffer);

```

8.2.4.16 ini_destroy() void ini_destroy (
void)

Löscht alle Schlüssel, löscht alle Strukturen.

Rückgabe

Nichts.

Definiert in Zeile 1544 der Datei IniFile.c.

```

1547     {
1548     TIniEntry *inilistptr = inilist;
1549     TIniEntry *nextptr;
1550
1551     while (inilistptr) {
1552         nextptr = inilistptr->next;
1553         INIS_STRFREE( inilistptr->key );
1554         INIS_STRFREE( inilistptr->value );
1555         INIS_FREE( inilistptr, sizeof(*inilistptr) );
1556         inilistptr = nextptr;
1557     }
1558     inilist = NULL;
1559
1560     inilistptr = comments;
1561     while (inilistptr) {
1562         nextptr = inilistptr->next;
1563         INIS_STRFREE( inilistptr->value );
1564         INIS_FREE( inilistptr, sizeof(*inilistptr) );
1565         inilistptr = nextptr;
1566     }
1567     comments = NULL;
1568
1569     INIS_STRFREE(logname);
1570     INIS_STRFREE(errname);

```

```

1571  INIS_STRFREE (errorininame);
1572  INIS_STRFREE (fallbacksection);
1573  INIS_STRFREE (tmpstr);
1574
1575  entrycount = sectioncount = commentcount = 0;

```

8.2.4.17 ini_set_fallbacksection() int ini_set_fallbacksection (
 const char * sectionname)

Setzt den Namen des Fallback-Abschnitts für im angegebenen Abschnitt nicht gefundene Einträge.

Üblicherweise wird ein Schlüssel nur im angegebenen Abschnitt der Ini-Datei (was der unbenannte Abschnitt sein kann) gesucht. Mit dieser Funktion kann jedoch der Name eines Fallback-Abschnitts gesetzt werden, in dem gesucht wird, falls ein Eintrag im angegebenen Abschnitt nicht gefunden wird. Der Wert kann leer oder [] sein, dann wird der unbenannte Abschnitt für den Fallback genutzt. Wenn der Wert jedoch NULL ist, wird kein Fallback genutzt.

Der Fallback wird nur für die Suche verwendet, nicht beim Löschen usw.

Parameter

in	sectionname	Name des Fallback-Abschnitts. "[]" für den Kann auch NULL oder leer sein, um Fallback abzuschalten. Begrenzende eckige Klammern dürfen fehlen (außer für den leeren Abschnitt).
----	-------------	---

Rückgabe

0, falls OK. INI_ERR_NOMEMORY, falls nicht genug Speicher.

Siehe auch

[ini_get_fallbacksection\(\)](#), [ini_get\(\)](#)

Definiert in Zeile 1600 der Datei IniFile.c.

```

1603                                     {
1604
1605  if ( !sectionname || !*sectionname ) {
1606      /* Bisheriges Fallback-Segment killen */
1607      INIS_STRFREE (fallbacksection);
1608  } else {
1609      INIS_STRFREE (fallbacksection);
1610      fallbacksection = make_section_name (sectionname);
1611      if ( !fallbacksection ) return INI_ERR_NOMEMORY;
1612  }

```

8.2.4.18 ini_get_fallbacksection() char* ini_get_fallbacksection (
 void)

Holt den Namen des Fallback-Abschnitts für im angegebenen Abschnitt nicht gefundene Einträge.

Rückgabe

Pointer auf einen temporären statischen String, der den Namen des aktuellen Fallback-Segments enthält. Kann NULL sein, wenn kein Fallback-Segment benannt ist oder wenn nicht genug Speicher vorhanden ist. Wenn der unbenannte Abschnitt der Fallback-Abschnitt ist, wird ein leerer String zurückgegeben, sonst der Name des Fallback-Abschnitts in eckigen Klammern. Der String sollte für weitere Verwendung kopiert werden. Der Speicherbereich sollte nicht beschrieben oder freigegeben werden.

Siehe auch

[ini_set_fallbacksection\(\)](#), [ini_get\(\)](#)

Definiert in Zeile 1629 der Datei IniFile.c.

```
1632 {  
1633     if ( !fallbacksection ) return NULL;  
1634     tmpstr = INI_STRREALLOC( tmpstr, strlen(fallbacksection) );  
1635     if (tmpstr) INI_COPYSTR( tmpstr, fallbacksection, strlen(fallbacksection) );
```

8.2.4.19 ini_get_error_filename() `char* ini_get_error_filename (`
`void )`

Liefert ggf.

den Namen der Ini-Datei, in der ein Fehler aufgetreten ist.

den Namen der Ini-Datei, in der ein Fehler aufgetreten ist.

Wenn ein Fehler beim Verarbeiten einer Ini-Datei aufgetreten ist, wird hier ihr Name zurückgeliefert. Wegen des Include-Mechanismus ist das nicht unbedingt die Datei, mit der [ini_read\(\)](#) explizit aufgerufen wurde.

Rückgabe

Der Name der Konfigurationsdatei, in der ein Fehler aufgetreten ist. NULL, wenn kein Fehler aufgetreten ist. Ein leerer String, wenn nicht genug Speicher für den Namen verfügbar war. Der Speicher sollte durch das aufrufende Programm nicht verändert oder freigegeben werden.

Siehe auch

[ini_read\(\)](#), [ini_get_error_linenumber\(\)](#)

Definiert in Zeile 1843 der Datei IniFile.c.

```
1846 {  
1847     static char *dummy = "";
```

8.2.4.20 ini_get_error_linenumbers() `int ini_get_error_linenumbers (`
`void )`

Liefert ggf.

die Nummer der Zeile der Ini-Datei, in der ein Fehler aufgetreten ist.

die Nummer der Zeile der Ini-Datei, in der ein Fehler aufgetreten ist.

Wenn ein Fehler beim Verarbeiten einer Ini-Datei aufgetreten ist, wird hier die Nummer der Zeile geliefert. Wegen des Include-Mechanismus ist das nicht unbedingt in der Datei, mit der [ini_read\(\)](#) explizit aufgerufen wurde.

Rückgabe

Die Zeilennummer. -1, wenn kein Fehler aufgetreten ist.

Siehe auch

[ini_read\(\)](#), [ini_get_error_filename\(\)](#)

Definiert in Zeile 1861 der Datei IniFile.c.

```
1864 { return errorlinenumbers; }
```

8.2.4.21 ini_get_version() `unsigned short ini_get_version (`
`void )`

Liefert die aktuelle Version der Library.

Liefert die aktuelle Version der Library.

Rückgabe

Die Versionsnummer der Library. Unteres Byte: Minor Version; oberes Byte: Major Version.

Definiert in Zeile 1869 der Datei IniFile.c.

```
1869 : Minor Version; oberes Byte: Major Version.
```

8.2.4.22 ini_get_version_string() `const char* ini_get_version_string (`
`void )`

Liefert die aktuelle Version der Library.

Liefert die aktuelle Version der Library.

Rückgabe

Die Versionsnummer der Library als Pointer auf einen statischen String..

Definiert in Zeile 1877 der Datei IniFile.c.

```
1880 { return VERSIONSTRING; }
```

8.2.4.23 ini_get_license() char* ini_get_license (
 void)

Liefert den Lizenz- und Copyright-Text.

Rückgabe

Pointer auf einen temporären statischen String, der den Text der Lizenz und des Copyrights enthält. Kann NULL sein, wenn nicht genug Speicher vorhanden ist. Der String sollte für weitere Verwendung kopiert werden. Der Speicherbereich sollte nicht beschrieben oder freigegeben werden.

Definiert in Zeile 1889 der Datei IniFile.c.

```
1892 {
1893     tmpstr = INI_STRREALLOC( tmpstr, strlen(COPYRIGHT) );
1894     if (tmpstr) INI_COPYSTR( tmpstr, COPYRIGHT, strlen(COPYRIGHT) );
1895     got_license = 1;
```

8.2.4.24 ini_get_dir_delimiter() char ini_get_dir_delimiter (
 const unsigned char option)

Gibt das Verzeichnistrennzeichen im Unix- oder DOS-Stil zurück.

Der Verzeichnistrenner wird nach Wunsch geliefert: fix im Unix- oder im DOS-/Windows-Stil, oder variabel (DOS-/Windows-Stil, falls ein Konfigurationseintrag mit dem Schlüssel DOS existiert und einen Wert ungleich 0 hat), oder variabel je nach aktuellem Betriebssystem.

Parameter

in	option	Falls 0: Verzeichnistrenner im Unix-Stil. Falls 2: Verzeichnistrenner im DOS-/Windows-Stil. Falls 1: variable Ermittlung (siehe oben).
----	--------	---

Rückgabe

Das Verzeichnistrennzeichen.

Definiert in Zeile 1912 der Datei IniFile.c.

```
1915 {
1916     char dosbuff[2];
1917     char delim = '/';
1918     #ifdef _WIN32
1919     #define DEFAULTTOPTION ("1")
1920     #else
1921     #define DEFAULTTOPTION ("0")
1922     #endif
1923
1924     /* Bestimmen, welches Verzeichnistrennerformat wir haben wollen. */
1925     if ( option == 1 ) {
1926         /* Variabel: Konfigurationswert mit dem Schlüssel DOS ansehen. */
1927         ini_get( dosbuff, "DOS", DEFAULTTOPTION, 1 );
1928         if ( ( *dosbuff != '\0' ) && ( !INI_ISSPACE(*dosbuff) ) && ( *dosbuff != '0' ) ) delim = '\\';
1929     } else if ( option == 2 ) {
1930         /* DOS-/Windows-Stil. */
1931         delim = '\\';
1932     }
1933 }
```

8.2.4.25 ini_fix_dir_delim() `char* ini_fix_dir_delim (`
`char * path,`
`const unsigned char option )`

Setzt alle Vorkommen von Verzeichnistrennern auf Unix- oder DOS-Stil um.

Hilfsfunktion für das Zusammenbauen von Dateipfaden. Entfernt doppelte Verzeichnistrenner (außer am Anfang); setzt Verzeichnistrenner fix auf Unix- oder auf DOS-/Windows-Stil um, oder variabel (DOS-/Windows-Stil, falls ein Konfigurationseintrag mit dem Schlüssel DOS existiert und einen Wert ungleich 0 hat), oder variabel je nach aktuellem Betriebssystem. Diese Aufgabe wird für die typischen Alltagsverwendungen von [ini_catfile\(\)](#) einfacher gelöst.

Parameter

<code>in, out</code>	<code>path</code>	Pfadangabe. Wird an Ort und Stelle modifiziert.
<code>in</code>	<code>option</code>	Falls 0: Umsetzung auf Unix-Stil. Falls 2: Umsetzung auf DOS-/Windows-Stil. Falls 1: variable Umsetzung (siehe oben).

Rückgabe

Pointer auf den modifizierten Pfad.

Siehe auch

[ini_catfile\(\)](#)

Definiert in Zeile 2004 der Datei IniFile.c.

```

2007                                     {
2008
2009     char *temp;
2010     char src, dst;
2011
2012     if ( !path ) return path;
2013
2014     dst = ini_get_dir_delimiter(option);
2015     src = ( dst == '/' ) ? '\\\' : '/';
2016
2017     while ( ( temp=strchr( path, src ) ) != NULL ) *temp = dst; /* Umsetzen. */
2018
2019     /* Jetzt nach doppelten Vorkommen von dst suchen. Am Anfang OK, sonst nicht. */
2020     temp = path;
2021     while ( *temp && ( *temp == dst ) ) temp++; /* Verzeichnistrenner am Anfang überspringen */
2022     src = 0;
2023     while (*temp) {
2024         if ( *temp == dst ) {
2025             if (src) {
2026                 /* Mehrfaches Auftreten, also ab aktueller Stelle aufrücken lassen. */
2027                 memmove( temp, temp+1, strlen(temp) );
2028             } else {
2029                 src = 1;
2030                 temp++;
2031             }
2032         } else {
2033             src = 0;
2034             temp++;
2035         }
2036     }

```

8.2.4.26 ini_add_dir_delim() `char* ini_add_dir_delim (`
`char * dirpath,`
`const unsigned char option )`

Hängt einen Verzeichnistrenner an, falls nicht schon vorhanden.

Parameter

<code>in, out</code>	<code>dirpath</code>	Pfadangabe. Wird an Ort und Stelle modifiziert. Muss genug Platz für ein weiteres Zeichen bieten.
<code>in</code>	<code>option</code>	Falls 0: Entfernen des Trenners am Ende (falls vorhanden). Falls 1: Anhängen eines Verzeichnistrenners, falls nicht bereits ein Trenner am Ende vorhanden ist.

Rückgabe

Pointer auf den modifizierten Pfad.

Siehe auch

[ini_catfile\(\)](#)

Veraltet Diese Aufgabe wird von [ini_catfile\(\)](#) besser gemacht.

Definiert in Zeile 2055 der Datei IniFile.c.

```

2058                                     {
2059     char *temp;
2060     if ( !INI_STRLEN(dirpath) ) return dirpath;
2061
2062     temp = dirpath + strlen(dirpath) - 1;
2063
2064     if (option) {
2065         if ( ( *temp != '/' ) && ( *temp != '\\' ) ) {
2066             (void)strcat( dirpath, ( strchr( dirpath, '/' ) || !strchr( dirpath, '\\' ) ) ? "/" : "\\ " );
2067         }
2068     } else {
2069         if ( ( *temp == '/' ) || ( *temp == '\\' ) ) *temp = '\0';
2070     }

```

8.2.4.27 ini_safemalloc() `void* ini_safemalloc (`
`const size_t size )`

Sichere (safe) Allokation von Speicherplatz.

Der Speicherplatz (sofern genügend verfügbar) wird mit 0 initialisiert.

Parameter

<code>in</code>	<code>size</code>	Angeforderte Speichergröße in Bytes.
-----------------	-------------------	--------------------------------------

Rückgabe

Pointer auf den Speicherplatz. NULL, falls nicht genügend Speicher vorhanden.

Definiert in Zeile 304 der Datei IniFile.c.

```

307                                     {
308     char *memp = (char *)malloc(size);
309     if (memp) inis_wipe_memory( memp, size );

```

8.2.4.28 ini_saferealloc() `void* ini_saferealloc (`
 `void * memp,`
 `const size_t size )`

Sichere (safe) Re-Allokation von Speicherplatz.

Sichere (safe) Re-Allokation von Speicherplatz.

Falls die Größe 0 und der Pointer NULL ist, ist dies ein NOOP. Falls die Größe 0 ist, ist diese Funktion identisch mit `free( )`. Falls der angegebene Pointer NULL ist, ist diese Funktion identisch mit `ini_safemalloc( )`. Falls die neue Speichergröße größer als die alte ist, wird der neue Teil nicht initialisiert. Falls der neue Speicherbereich kleiner als der alte ist, wird der alte Restspeicher ebenfalls nicht ausgenullt.

Parameter

in, out	<i>memp</i>	Pointer auf den zu verändernden Speicher. Kann NULL sein.
in	<i>size</i>	Angeforderte Speichergröße in Bytes.

Rückgabe

Pointer auf den Speicherplatz. NULL, falls nicht genügend Speicher vorhanden.

Definiert in Zeile 328 der Datei IniFile.c.

```
331                                     {
332     if ( size == 0 ) {
333         if (memp) {
334             free(memp);
335             memp = NULL;
336         }
337     } else {
338         memp = (memp) ? realloc(memp, size) : INI_MALLOC(size);
339     }
```

8.2.4.29 inis_wipe_memory() `void __inline inis_wipe_memory (`
 `void * memp,`
 `const size_t size )`

Sicheres (secure) Löschen von Speicherplatz.

Sicheres (secure) Löschen von Speicherplatz.

Der angegebene Speicherbereich wird so sicher (secure) wie möglich gelöscht. Stark angelehnt an die ARGON2-Source.

Parameter

in	<i>memp</i>	Pointer auf den zu löschenden Speicher. Kann NULL sein.
in	<i>size</i>	Speichergröße in Bytes.

Definiert in Zeile 393 der Datei IniFile.c.

```
396                                     {
397     if ( memp && size ) {
398     #if defined(_MSC_VER) && ( _MSC_VER >= 1400 ) || defined(__MINGW32__)
399         SecureZeroMemory( memp, size );
400     #elif defined memset_s
```



```
401     memset_s( memp, size, 0, size );
402 #elif defined(__OpenBSD__) || ( defined(__GLIBC__) && defined(__GLIBC_PREREQ) && __GLIBC_PREREQ(2,25) )
403     explicit_bzero( memp, size );
404 #else
405     static void *( *const volatile memset_sec )( void *, int, size_t ) = &memset;
406     memset_sec( memp, 0, size );
```

Index

BUFSIZE
 IniFile.c, [11](#)

commentcount
 IniFile.c, [43](#)

comments
 IniFile.c, [41](#)

COPYRIGHT
 IniFile.c, [10](#)

DEFAULTOPTION
 IniFile.c, [12](#)

entrycount
 IniFile.c, [42](#)

errname
 IniFile.c, [42](#)

ERROR_RETURN
 IniFile.c, [11](#)

errorininame
 IniFile.c, [43](#)

errorlinenumber
 IniFile.c, [43](#)

fallbacksection
 IniFile.c, [42](#)

got_license
 IniFile.c, [43](#)

handle_pragma
 IniFile.c, [17](#)

ini_add
 IniFile.c, [20](#)
 IniFile.h, [60](#)

ini_add_comment
 IniFile.c, [28](#)
 IniFile.h, [68](#)

ini_add_dir_delim
 IniFile.c, [40](#)
 IniFile.h, [77](#)

ini_catfile
 IniFile.c, [39](#)

INI_CHAR_TOUPPER
 IniFile.h, [53](#)

ini_comments
 IniFile.c, [27](#)
 IniFile.h, [67](#)

INI_COPYSTR
 IniFile.h, [52](#)

INI_COPYSTRUPPER
 IniFile.h, [54](#)

ini_delete
 IniFile.c, [29](#)
 IniFile.h, [69](#)

ini_destroy
 IniFile.c, [32](#)
 IniFile.h, [72](#)

INI_ERR_NOFILE
 IniFile.h, [48](#)

INI_ERR_NOMEMORY
 IniFile.h, [48](#)

INI_ERR_NONAME
 IniFile.h, [48](#)

INI_ERR_NOSUCHPRAGMA
 IniFile.h, [49](#)

ini_fix_dir_delim
 IniFile.c, [39](#)
 IniFile.h, [76](#)

INI_FREE
 IniFile.h, [51](#)

ini_get
 IniFile.c, [34](#)
 IniFile.h, [58](#)

ini_get_dir_delimiter
 IniFile.c, [38](#)
 IniFile.h, [76](#)

ini_get_error_filename
 IniFile.c, [36](#)
 IniFile.h, [74](#)

ini_get_error_linenumber
 IniFile.c, [37](#)
 IniFile.h, [74](#)

ini_get_fallbacksection
 IniFile.c, [33](#)
 IniFile.h, [73](#)

ini_get_license
 IniFile.c, [38](#)
 IniFile.h, [75](#)

ini_get_version
 IniFile.c, [37](#)
 IniFile.h, [75](#)

ini_get_version_string
 IniFile.c, [37](#)
 IniFile.h, [75](#)

INI_ISDIGIT
 IniFile.h, [54](#)

INI_ISSPACE
 IniFile.h, [53](#)

INI_ISXDIGIT
 IniFile.h, [54](#)

ini_keys
 IniFile.c, [24](#)
 IniFile.h, [62](#)

ini_keys_section
 IniFile.c, [24](#)
 IniFile.h, [63](#)

INI_MALLOC
 IniFile.h, [49](#)

ini_ncomments
 IniFile.c, [27](#)

- IniFile.h, 67
- ini_nkeys
 - IniFile.c, 22
 - IniFile.h, 65
- ini_nkeys_section
 - IniFile.c, 22
 - IniFile.h, 66
- ini_nsections
 - IniFile.c, 23
 - IniFile.h, 66
- ini_read
 - IniFile.c, 18
 - IniFile.h, 56
- INI_REALLOC
 - IniFile.h, 49
- ini_safemalloc
 - IniFile.c, 13
 - IniFile.h, 78
- ini_saferealloc
 - IniFile.c, 13
 - IniFile.h, 78
- ini_sections
 - IniFile.c, 26
 - IniFile.h, 64
- ini_set
 - IniFile.c, 21
 - IniFile.h, 62
- ini_set_fallbacksection
 - IniFile.c, 33
 - IniFile.h, 73
- INI_STRALLOC
 - IniFile.h, 49
- INI_STREQUAL
 - IniFile.h, 53
- INI_STRLEN
 - IniFile.h, 52
- INI_STRREALLOC
 - IniFile.h, 50
- INI_STRUPPER
 - IniFile.h, 55
- ini_write
 - IniFile.c, 30
 - IniFile.h, 70
- IniEntry, 6
 - key, 6
 - next, 7
 - value, 6
- IniFile.c, 7
 - BUFSIZE, 11
 - commentcount, 43
 - comments, 41
 - COPYRIGHT, 10
 - DEFAULTOPTION, 12
 - entrycount, 42
 - errname, 42
 - ERROR_RETURN, 11
 - errorininame, 43
 - errorlinenumber, 43
 - fallbacksection, 42
 - got_license, 43
 - handle_pragma, 17
 - ini_add, 20
 - ini_add_comment, 28
 - ini_add_dir_delim, 40
 - ini_catfile, 39
 - ini_comments, 27
 - ini_delete, 29
 - ini_destroy, 32
 - ini_fix_dir_delim, 39
 - ini_get, 34
 - ini_get_dir_delimiter, 38
 - ini_get_error_filename, 36
 - ini_get_error_linenumber, 37
 - ini_get_fallbacksection, 33
 - ini_get_license, 38
 - ini_get_version, 37
 - ini_get_version_string, 37
 - ini_keys, 24
 - ini_keys_section, 24
 - ini_ncomments, 27
 - ini_nkeys, 22
 - ini_nkeys_section, 22
 - ini_nsections, 23
 - ini_read, 18
 - ini_safemalloc, 13
 - ini_saferealloc, 13
 - ini_sections, 26
 - ini_set, 21
 - ini_set_fallbacksection, 33
 - ini_write, 30
- inilist, 41
- ininames, 41
- inis_saferealloc, 14
- inis_wipe_memory, 15
- isolate_string, 15
- JUMP_EMPTY_SECTION, 11
- logname, 42
- make_section_name, 16
- MAKE_SPECIAL_FILE_NAME, 12
- MINIBUFSIZE, 11
- recursion, 42
- sectioncount, 43
- tmpstr, 42
- uchar, 13
- VERSION, 10
- VERSIONSTRING, 10
- IniFile.h, 44
 - ini_add, 60
 - ini_add_comment, 68
 - ini_add_dir_delim, 77
 - INI_CHAR_Toupper, 53
 - ini_comments, 67
 - INI_COPYSTR, 52
 - INI_COPYSTRUPPER, 54
 - ini_delete, 69
 - ini_destroy, 72

INI_ERR_NOFILE, 48
INI_ERR_NOMEMORY, 48
INI_ERR_NONAME, 48
INI_ERR_NOSUCHPRAGMA, 49
ini_fix_dir_delim, 76
INI_FREE, 51
ini_get, 58
ini_get_dir_delimiter, 76
ini_get_error_filename, 74
ini_get_error_linenumber, 74
ini_get_fallbacksection, 73
ini_get_license, 75
ini_get_version, 75
ini_get_version_string, 75
INI_ISDIGIT, 54
INI_ISSPACE, 53
INI_ISXDIGIT, 54
ini_keys, 62
ini_keys_section, 63
INI_MALLOC, 49
ini_ncomments, 67
ini_nkeys, 65
ini_nkeys_section, 66
ini_nsections, 66
ini_read, 56
INI_REALLOC, 49
ini_safemalloc, 78
ini_saferealloc, 78
ini_sections, 64
ini_set, 62
ini_set_fallbacksection, 73
INI_STRALLOC, 49
INI_STREQUAL, 53
INI_STRLEN, 52
INI_STRREALLOC, 50
INI_STRUPPER, 55
ini_write, 70
INIS0_FREE, 51
INIS_FREE, 51
INIS_REALLOC, 50
INIS_STRFREE, 52
inis_wipe_memory, 79
TIniEntry, 55
inilist
 IniFile.c, 41
ininames
 IniFile.c, 41
INIS0_FREE
 IniFile.h, 51
INIS_FREE
 IniFile.h, 51
INIS_REALLOC
 IniFile.h, 50
inis_saferealloc
 IniFile.c, 14
INIS_STRFREE
 IniFile.h, 52
inis_wipe_memory
 IniFile.c, 15
 IniFile.h, 79
isolate_string
 IniFile.c, 15
JUMP_EMPTY_SECTION
 IniFile.c, 11
key
 IniEntry, 6
logname
 IniFile.c, 42
make_section_name
 IniFile.c, 16
MAKE_SPECIAL_FILE_NAME
 IniFile.c, 12
MINIBUFSIZE
 IniFile.c, 11
next
 IniEntry, 7
recursion
 IniFile.c, 42
sectioncount
 IniFile.c, 43
TIniEntry
 IniFile.h, 55
tmpstr
 IniFile.c, 42
uchar
 IniFile.c, 13
value
 IniEntry, 6
VERSION
 IniFile.c, 10
VERSIONSTRING
 IniFile.c, 10